

Classical Obfuscation of Quantum Circuits via Publicly-Verifiable QFHE

James Bartusek*
Columbia

Aparna Gupte†
MIT

Saachi Mutreja‡
Columbia

Omri Shmueli§
NTT Research

January 11, 2026

Abstract

A classical obfuscator for quantum circuits is a classical program that, given the classical description of a quantum circuit Q , outputs the classical description of a functionally equivalent quantum circuit $\tilde{Q}$ that hides as much as possible about Q . Previously, the *only* known feasibility result for classical obfuscation of quantum circuits (Bartusek and Malavolta, ITC 2022) was limited to “null” security, which is only meaningful for circuits that always reject. On the other hand, if the obfuscator is allowed to compile the quantum circuit Q into a *quantum state* $|\tilde{Q}\rangle$, there exist feasibility results for obfuscating much more expressive classes of circuits: All pseudo-deterministic quantum circuits (Bartusek, Kitagawa, Nishimaki and Yamakawa, STOC 2023, Bartusek, Brakerski and Vaikuntanathan, STOC 2024), and even all unitaries (Huang and Tang, FOCS 2025).

We show that (relative to a classical oracle) there exists a *classical* obfuscator for all pseudo-deterministic quantum circuits. As our main technical step, we give the first construction of a compact quantum fully-homomorphic encryption (QFHE) scheme that supports public verification of (pseudo-deterministic) quantum evaluation, relative to a classical oracle.

To construct our QFHE scheme, we improve on an approach introduced by Bartusek, Kitagawa, Nishimaki and Yamakawa (STOC 2023), which previously required ciphertexts that are both *quantum* and *non-compact* due to a heavy use of quantum coset states and their publicly-verifiable properties. As part of our core technical contribution, we introduce new techniques for analyzing coset states that can be generated “on the fly”, by proving new cryptographic properties of the one-shot signature scheme of Shmueli and Zhandry (CRYPTO 2025). Our techniques allow us to produce QFHE ciphertexts that are purely classical, compact, and publicly-verifiable. This additionally yields the first classical verification of quantum computation protocol for BQP that simultaneously satisfies blindness and public-verifiability.

*bartusek.james@gmail.com

†agupte@mit.edu

‡sm5540@columbia.edu

§omri.shmueli1@gmail.com

Contents

1	Introduction	3
1.1	Classical Obfuscation via Publicly-Verifiable QFHE	4
1.2	Results	5
1.3	Discussion	6
2	Technical Overview	7
2.1	Publicly-Verifiable PFCs and Previous Work	7
2.2	Our Work: One-Shot Signatures with Delayed Collapsing	9
2.3	From PFCs to QFHE and Obfuscation	13
3	Preliminaries	14
3.1	Notation	14
3.2	Fully-homomorphic encryption	14
3.3	One-shot signatures (OSS)	15
3.4	Measure and re-program	17
3.5	The Compressed Oracle Technique	17
3.6	Useful Lemmas and Definitions	18
4	Pauli Functional Commitments with Classical Keys	18
4.1	Definition	18
4.2	Construction	20
4.3	Correctness	22
4.4	Binding	22
4.4.1	Hardness of the OSS delayed collapsing game	25
4.4.2	Coset-collapsing game	30
5	Post-quantum Succinct Ideal Obfuscation	34
5.1	SNARKs	34
5.2	Probabilistically Checkable Proofs (PCPs)	35
5.3	The construction of Micali	35
5.4	Construction	37
5.5	Correctness	37
5.6	Succinctness	37
5.7	Security	38
6	Publicly-verifiable QFHE	40
6.1	Definition	41
6.2	Upgradable privately-verifiable QFHE	42
6.3	Construction	44
6.4	Proofs	46
7	(Succinct) Classical Obfuscation of Pseudo-deterministic Quantum Circuits	49
7.1	Construction	50
8	Acknowledgements	52
A	Remaining proofs from Section 4	55
A.1	Correctness	55
A.2	Binding	57

1 Introduction

Program obfuscation disguises the inner workings of an algorithm while leaving its behavior unchanged. This concept traces back to the roots of modern cryptography [DH22], and ever since the formalization of *indistinguishability* obfuscation (iO) [BGI⁺01], constructing iO has been one of cryptography’s overarching goals. It took no less than a decade to come up with the first candidates for secure iO [GGH⁺16], and almost an additional decade to base classically-secure iO on well-studied assumptions [JLS21]. Current efforts continue along this trajectory, with an active line of work devoted to developing quantum-secure iO, eventually grounded in similarly well-understood assumptions (e.g. [GGH15, BGMZ18, CVW18, APM20, BDGM23, WW21],...).

The original notion of program obfuscation implicitly considers computation as classical, and accordingly, seeks to obfuscate classical algorithms. Now, as quantum computation increasingly emerges as a standard model of computation, it is natural to revisit the problem in this broader context: Can quantum algorithms be obfuscated as well?

Ideal Program Obfuscation: From Classical to Quantum. Our understanding of obfuscation in the quantum setting is still at a very early stage: In particular, we still don’t know how to base indistinguishability obfuscation for expressive classes of quantum circuits on (post-quantum) indistinguishability obfuscation for classical circuits.¹ Instead, recent work has turned to exploring the relationship between *idealized* notions of obfuscation in the classical and quantum worlds [BM22, BKNY23, BBV24, HT25]. In such an idealized model, an obfuscation of a circuit C is considered secure if everything that can be learned from the obfuscation $\tilde{C}$ can be simulated given only *oracle access* to C .

Of course, ideal obfuscation cannot be achieved in full generality, as there are well-known counterexamples demonstrating its impossibility [BGI⁺01]. Nevertheless, studying the connection between classical and quantum ideal obfuscation remains valuable for at least two reasons. First, it isolates a fundamental question: Does the existence of an ideal obfuscator for classical computation imply one for quantum? Second, indistinguishability obfuscation is widely regarded as a strong heuristic for ideal obfuscation (e.g., it is the best-possible obfuscation [GR07]). Thus, establishing that ideal classical obfuscation gives rise to ideal quantum obfuscation results in candidates for quantum obfuscation by instantiating the black-box with (post-quantum) indistinguishability obfuscation.

Prior work [BKNY23, BBV24, HT25] has established that, assuming a classical ideal obfuscator for classical circuits, one can construct a *quantum* ideal obfuscator for quantum circuits. In this setting, the obfuscator itself is a quantum algorithm, and on input a quantum circuit Q it outputs a quantum state $|\tilde{Q}\rangle$ serving as the obfuscated program.² A distinctive feature of these constructions is that the obfuscated object consists of *unclonable* states: Once $|\tilde{Q}\rangle$ is produced, it cannot be duplicated. As a consequence, it is impossible to create a single obfuscation of a quantum circuit and distribute it broadly. Moreover, these obfuscations cannot be transmitted over the classical channels that comprise the internet today.

However, the goal of constructing a classical obfuscator for quantum circuits (with security for non-null circuits³), even in the ideal setting, has so far remained out of reach. This raises our central motivating question:

*Is a quantum obfuscator necessary to obfuscate expressive quantum circuits?
Or can a classically-described quantum program be obfuscated by a classical obfuscator?*

We address this question by constructing a classical obfuscator for all pseudo-deterministic circuits.

¹The only existing result from indistinguishability obfuscation is limited to quantum circuits with logarithmically-many non-Clifford gates [BK21].

²We note that [BBV24, HT25] are able to do something slightly stronger—they obfuscate quantum circuits that also have an auxiliary quantum state. This is particularly relevant in some cryptographic applications, e.g. copy-protection.

³Bartusek and Malavolta [BM22] give a feasibility result for classical obfuscation of null quantum circuits, that is, circuits that always reject.

1.1 Classical Obfuscation via Publicly-Verifiable QFHE

Previous work [BK^{NY}23] identified *pseudo-deterministic* quantum circuits as a particularly useful class for obfuscation. A pseudo-deterministic quantum circuit Q from n to m qubits is one that, up to negligible error, computes a classical function: there exists a function $F : \{0, 1\}^n \rightarrow \{0, 1\}^m$ such that for every input $x \in \{0, 1\}^n$, the output of $Q(x)$ equals $F(x)$ with probability $1 - \text{negl}(n)$, where the probability is taken over the measurement of its output. This class is both expressive—capturing, for example, all quantum algorithms that solve problems in **NP**, including Shor’s factoring algorithm [Sho99]—and structurally well-suited for cryptographic applications like obfuscation. Indeed, pseudo-deterministic circuits have already served as the foundation for obfuscating increasingly general classes of quantum computation [BBV24, HT25]. Motivated by these properties, our work focuses on the obfuscation of pseudo-deterministic circuits. Crucially, we consider obfuscation through only classical means.

As also observed in previous work [BK^{NY}23], a natural route toward obfuscating pseudo-deterministic circuits relies on constructing quantum fully-homomorphic encryption (QFHE) with public verification of homomorphic evaluation for such circuits.

- A QFHE scheme is defined by algorithms (Gen, Enc, Eval, Dec), where all but the evaluation algorithm Eval can be regarded as classical. On input a ciphertext $\text{ct} = \text{Enc}(x)$ and a quantum circuit Q , the evaluation algorithm Eval produces an evaluated ciphertext $\tilde{\text{ct}}$ which encrypts the classical value $Q(x)$.
- In the publicly-verifiable variant of QFHE, Eval additionally outputs a succinct classical proof π , and correctness can be certified by an efficient classical verification algorithm Ver, which checks that the homomorphic evaluation has been honestly computed, and outputs $\text{Ver}(\text{ct}, Q, \tilde{\text{ct}}, \pi) \in \{\top, \perp\}$.

It is straightforward to see that, given such a powerful QFHE, obfuscation of pseudo-deterministic quantum circuits follows naturally. This can be accomplished by obfuscating the verifier and applying the standard “verify-then-decrypt” paradigm. Concretely, the obfuscation of a circuit Q consists of encrypting its classical description, i.e., $\text{ct}_Q \leftarrow \text{Enc}(Q)$. On input $x \in \{0, 1\}^n$ and the encrypted circuit ct_Q , the evaluator computes $(\tilde{\text{ct}}, \pi) \leftarrow \text{Eval}(\text{ct}_Q, U_x)$, where U_x denotes the universal quantum circuit with input x hard-wired. To recover the (classical) output $Q(x)$, the obfuscation makes use of a classical oracle implementing the circuit

$$C(x, \tilde{\text{ct}}, \pi) := \begin{cases} \text{Dec}(\tilde{\text{ct}}) & \text{if } \text{Ver}(\text{ct}_Q, U_x, \tilde{\text{ct}}, \pi) = \top \\ \perp & \text{otherwise} \end{cases}$$

Thus, only honestly evaluated ciphertexts are ever decrypted, and the obfuscation securely reproduces the functionality of Q .

The central role of Pauli Functional Commitments. However, it turns out that verification of QFHE evaluations is a particularly delicate problem in quantum cryptography.

- On the one hand, by definition, evaluating QFHE ciphertexts is a quantum *sampling* task. The quantum QFHE evaluation gets the classical ct and outputs a quantumly computed classical sample $\tilde{\text{ct}}$, which is going to have high entropy *even if* the underlying evaluated circuit Q is deterministic. In fact, this entropy was shown to be somewhat inherent in [Shm22a, Shm22b]. This seemingly presents a barrier, since it remains open how to classically verify quantum sampling problems with negligible soundness error ([CLLW22] has shown how to achieve inverse-polynomial error, which, if used in the template above, would allow a polynomial-time attack on the obfuscation scheme).
- On the other hand, the special structure of certain QFHE schemes, most notably Mahadev’s construction [Mah20], enables a partial solution. As shown in [Bar21], the use of Mahadev’s measurement protocol [Mah18] on Mahadev’s QFHE scheme allows for verification of QFHE evaluations with negligible soundness error. However, since the measurement protocol of Mahadev is privately verifiable, then so is the resulting QFHE scheme.

The technical centerpiece behind Mahadev’s measurement protocol was later abstracted by [BKNY23] as a Pauli functional commitment (PFC). Roughly speaking, a PFC allows a classical verifier to issue a public classical commitment key CK . A quantum prover can then commit to a quantum state in such a way that, later, measurements of Pauli observables (in particular, standard or Hadamard basis measurements) on the state can be verified through classical communication. Viewed through this lens, the state of the art can be summarized as follows: Mahadev [Mah18] constructs a privately verifiable PFC; Bartusek [Bar21] demonstrates that applying Mahadev’s PFCs to QFHE homomorphic evaluation yields privately verifiable evaluations; and finally, [BKNY23] show how to construct, relative to a classical oracle, PFCs with public verification, albeit with a *quantum* commitment key $|CK\rangle$.

Measurement protocols, or in their formal abstraction as PFCs, were originally devised to allow a classical client to force a quantum server to perform some prescribed computation. Existing results on public verification, however, require the commitment keys to remain quantum, which prevents them from being generated by a classical verifier. This leads us to what we view as a fascinating technical question in its own right: can one design classical measurement protocols that are publicly verifiable? As we have explained, resolving this question would not only advance the theory of quantum cryptography but would also translate directly to our applications, enabling public verification of QFHE evaluations and, ultimately, classical obfuscation of quantum circuits.

1.2 Results

We summarize our main results in this section.

Theorem 1 (Informal). *Assuming the quantum hardness of the Learning With Errors problem, there exists classical obfuscation for (classically-described) pseudo-deterministic quantum circuits in the classical oracle model.*

Along the way, as a tool for our obfuscation scheme, we build the first publicly-verifiable QFHE scheme for pseudo-deterministic circuits.

Theorem 2 (Informal). *Assuming the quantum hardness of the Learning With Errors problem (resp. plus an appropriate circular-security assumption), there exists a leveled (resp. unleveled) quantum fully-homomorphic encryption in the classical oracle model, with the following properties.*

- *Classical encryption of classical messages.*
- *Succinct, classical, and public verification of evaluation for pseudo-deterministic circuits.*

Main Technical Contribution. Put plainly, our main technical contribution is the first publicly-verifiable quantum measurement protocol in the classical oracle model. Before our work, a quantum prover could prove the integrity of a measurement (standard-basis or Hadamard) on a quantum state only to a single verifier holding a trapdoor. More formally, we construct the first publicly-verifiable Pauli functional commitment scheme with *classical keys*. As a central part of our construction and security proof, we construct one-shot signatures (and hash functions) with what we call *delayed collapsing*. Delayed collapsing was implicitly introduced in [BKNY23], in the context of random coset states (and we intuitively explain how it relates to Pauli functional commitments, in Section 2.1). We identify delayed collapsing as a fundamental property for hash functions; a delayed collapsing hash implies both collapsing hash functions and non-collapsing collision-resistant hash functions. We prove that the one-shot signatures construction of [SZ25] induces not only a non-collapsing collision-resistant hash function (which was previously known), but a delayed collapsing hash function. To prove our results, we show a new stronger analysis of one-shot signatures, and prove new quantum information theoretical properties of random coset states, which may be of independent interest. See more details in our Technical Overview (Section 2).

As an additional contribution that may be of independent interest, we show that post-quantum *succinct* ideal obfuscation for classical Turing machines exists in the classical oracle model (Section 5). We do this via

a natural “FHE + SNARK” approach, and combine techniques from [CMS19, Zha19a] in order to establish post-quantum security.

Finally, we note that our quantum obfuscation scheme is in fact *succinct*, meaning that the size of the obfuscated program grows only with the *description size* of the plaintext quantum program, as opposed to the number of physical gates in the circuit. We emphasize that, even though our construction includes a classical oracle, the classical circuit implemented by the oracle is itself succinct. This means that succinctness is preserved when the classical oracle is heuristically instantiated with post-quantum indistinguishability obfuscation.

As an immediate corollary, we obtain the first *succinct randomized encoding* scheme for pseudo-deterministic quantum computation. A succinct randomized encoding allows for compiling a circuit Q and an input x into a succinct representation $\widehat{Q(x)}$ in time much less than evaluating Q , such that $\widehat{Q(x)}$ reveals only $Q(x)$ and nothing else about x . Succinct randomized encodings have had several important applications in classical cryptography [BGL⁺15], and we expect them to be widely useful in the quantum setting as well.

1.3 Discussion

Comparison with prior work on CVQC. Our publicly-verifiable QFHE protocol naturally yields a new type of classical verification of quantum computation (CVQC) protocol for BQP. Indeed, given some quantum computation Q , the verifier simply encrypts their input x as $\text{Enc}(x)$, and the prover responds with $\text{Enc}(Q(x))$ along with a proof π of correct evaluation.

This protocol has several desirable properties: The prover remains blind to the verifier’s input x , the proof π is publicly-verifiable (implying for example that the prover cannot break security by repeatedly querying the verifier and learning whether they accept or reject), and the client’s work does not grow with the size of Q . In particular, we obtain the first CVQC protocol that is both *blind* and *publicly-verifiable*, and on top of that it is both non-interactive and succinct. We summarize a comparison with prior work in Table 1.

Open problems. Our work highlights the utility of classical-client publicly-verifiable QFHE, both for its applications in CVQC and in obfuscation of quantum computation. While we construct this primitive in a manner that supports pseudo-deterministic evaluation, we leave open some interesting questions for future work.

- Can we construct classical-client publicly-verifiable QFHE for *all* possible functionalities (i.e. quantum circuits that yield an arbitrary distribution over classical outputs)? This would be interesting to achieve even with non-negligible soundness (achieving negligible soundness for sampling problems is a well-known and difficult open problem), and even without compactness.
- Can we prove security of classical-client publicly-verifiable QFHE from post-quantum indistinguishability obfuscation? Unfortunately, it still remains open to establish the security of publicly-verifiable CVQC under iO even forgetting about blindness and compactness.

Another natural direction is to expand the class of quantum programs that can be classically obfuscated. While our approach builds on [BKNY23], an alternative approach of [BBV24, HT25] has yielded obfuscation for all *unitaries*, but with a quantum obfuscator. The reason that we build on [BKNY23] as opposed to [BBV24, HT25] is that the quantum component of the [BKNY23] obfuscated program contains “only” coset states, while the quantum component of the [BBV24, HT25] obfuscated programs contain more complicated states, in particular magic states encoded in the so-called coset state authentication code [BBV24]. It is a fascinating open question whether this approach can also be de-quantized, which may yield classical obfuscation of unitaries and beyond.

	Blind	Verification	Succinct	Interactive	Heuristic
[Mah18]	Yes*	Private	No	Yes	No
[BKL ⁺ 22]	Yes*	Private	Yes	Yes	No
[BM22]	No	Public	Yes	No	Yes
This work	Yes	Public	Yes	No	Yes

Table 1: A comparison among classical verification of quantum computation protocols for BQP. The asterisk in the Blind column refer to the fact that these protocols aren’t necessarily presented as blind, but can be made blind by running under QFHE. This cannot be done for [BM22] without sacrificing public verification. The Heuristic column refers to provable security (i.e. reduction to an assumption) vs. heuristic security (concretely, argued in the classical oracle model). Finally, we note that while the protocols listed above are the first to achieve the listed guarantees, there have been several other protocols proposed that improve along other metrics such as the assumption [NZ23, MNZ24, GKNV25] and circuit class [CLLW22].

2 Technical Overview

As mentioned above, the main technical workhorse behind our results can be seen as a *publicly-verifiable* measurement protocol with a classical verifier. To this end, we construct, relative to a classical oracle, *publicly-verifiable* Pauli functional commitments with classical commitment keys. We next recall what PFCs are, discuss previous work, and then present our new techniques.

2.1 Publicly-Verifiable PFCs and Previous Work

A publicly-verifiable PFC with classical keys (Definition 5) is given by algorithms ($\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX}$), where the classical algorithms here are $\text{Gen}, \text{DecZ}, \text{DecX}$. Our standard scenario is a classical verifier Ver that wants to make a Pauli measurement on some unknown n -qubit state $|\psi\rangle$ held by a quantum prover P . In this instance, the flow of a PFC goes as follows.

1. Ver samples classical keys $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$. We think of CK as a public commitment key in the form of an ideally-obfuscated classical circuit, and dk is a secret decoding key. Ver sends CK to the prover.
2. P commits to its state by computing $(c, \mathcal{B}, \mathcal{U}) \leftarrow \text{Com}^{\text{CK}}(|\psi\rangle_{\mathcal{B}})$, where c is a classical commitment string, $\mathcal{B} := (\mathcal{B}_i)_{i \in [n]}$ is the original n -qubit register that held $|\psi\rangle$ and $\mathcal{U} := (\mathcal{U}_i)_{i \in [n]}$ is an additional quantum register with some $\text{poly}(n)$ -qubits, now entangled with $\mathcal{B}$. P sends c to the verifier.
3. Ver chooses some measurement basis $h \in \{0, 1\}^n$, where $h_i = 0$ denotes measuring qubit i of $|\psi\rangle$ in the standard basis and $h_i = 1$ denotes Hadamard basis. Ver sends h to the prover.
4. P decommits to the Pauli observables of its state by applying $u_i \leftarrow \text{OpenZ}(\mathcal{B}_i, \mathcal{U}_i)$ for every $h_i = 0$ and $u_i \leftarrow \text{OpenX}(\mathcal{B}_i, \mathcal{U}_i)$ otherwise. P sends $u := (u_i)_{i \in [n]}$ to the verifier.
5. Ver can now privately decode the measurement outcomes by applying the corresponding decoding procedures $\text{DecZ}(\text{dk}, \cdot, \cdot)$ and $\text{DecX}(\text{dk}, \cdot, \cdot)$. For each $i \in [n]$, the corresponding algorithm takes as input the commitment c and the value u_i , and outputs a measurement result $b_i \in \{0, 1\}$.

In words, what happens above is that at Step 2, the prover generates an *encoding* of the state $|\psi\rangle_{\mathcal{B}}$, by embedding it into a larger space $(\mathcal{B}, \mathcal{U})$. Then, at Step 4, the prover generates encodings of standard basis measurements $u_i \leftarrow \text{OpenZ}(\mathcal{B}_i, \mathcal{U}_i)$ or Hadamard basis measurements $u_i \leftarrow \text{OpenX}(\mathcal{B}_i, \mathcal{U}_i)$, for each $i \in [n]$. The logical values of these encoded measurements are later extracted by Ver using dk .

PFC Security: Private vs Public Verification. Intuitively, security of a PFC ensures that the logical standard basis measurements cannot be swapped. A bit more formally, for each part $i \in [n]$ of the committed state on register $(\mathcal{B}_i, \mathcal{U}_i)$ at the end of Step 2, if we project the state onto encodings of standard basis measurements for some fixed value $b \in \{0, 1\}$ (i.e., strings u_i such that $\text{DecZ}(\text{dk}, c, u_i) = b$), then no adversary can map this state to encodings of standard basis measurements for value $1 - b$ (i.e., strings u_i such that $\text{DecZ}(\text{dk}, c, u_i) = 1 - b$). In fact, this is just one way to describe the traditional notion of post-quantum binding for classical messages (equivalent to collapse-binding and sum-binding).

In a privately-verifiable PFC, the verifier's decoding key dk must remain completely inaccessible to the prover throughout the protocol, and such a PFC was (implicitly) constructed in [Mah18]. In a publicly-verifiable scheme, as considered by [BKNY23], we aim to give the prover access to the verifier's decoding functionalities without compromising the binding security stated above. A natural attempt at such a definition would require the scheme to remain secure against a prover given quantum oracle access to the classical functions $\text{DecZ}(\text{dk}, \cdot, \cdot)$, $\text{DecX}(\text{dk}, \cdot, \cdot)$.

It turns out that, for our application (as well as for [BKNY23]), the security of publicly-verifiable PFC need not hold⁴ if the prover is given *uninterrupted* access to both $\text{DecZ}(\text{dk}, \cdot, \cdot)$, $\text{DecX}(\text{dk}, \cdot, \cdot)$. Instead, we consider the relaxation where the prover P is given quantum access to $\text{DecZ}(\text{dk}, \cdot, \cdot)$, $\text{DecX}(\text{dk}, \cdot, \cdot)$ only up to the collapse of the encoding register $(\mathcal{B}, \mathcal{U})$. That is, we consider a game where a malicious prover P^* prepares the commitment $(c, \mathcal{B}, \mathcal{U}) \leftarrow \text{Com}^{\text{CK}}(|\psi\rangle_{\mathcal{B}})$ as in Step 2, with additional access to both $\text{DecZ}(\text{dk}, \cdot, \cdot)$, $\text{DecX}(\text{dk}, \cdot, \cdot)$. Then, the PFC challenger "removes" access to the Hadamard decoding functionality $\text{DecX}(\text{dk}, \cdot, \cdot)$, and collapses registers $(\mathcal{B}_i, \mathcal{U}_i)_{i \in [n]}$ to encodings of classical measurement results $\{b_i\}_{i \in [n]}$. Then, given access to only $\text{DecZ}(\text{dk}, \cdot, \cdot)$, the prover P^* is required to map these encodings to encodings of classical measurement results for the opposing values $\{1 - b_i\}_{i \in [n]}$. To re-iterate, the prover has full access to the verification functionalities while preparing its *commitment*, but *loses* access to the Hadamard decoding functionality while attempting to map between standard basis openings. To satisfy the above notion of binding it is sufficient for our scheme to formally satisfy a property called *single-bit binding with public decodability*, which is made formal in Definition 7.

Public Verification via Delayed Collapsing. The only previous result on publicly-verifiable PFCs is due to the work of Bartusek, Kitagawa, Nishimaki and Yamakawa [BKNY23], which achieves the above (in a classical oracle model), though where the commitment key $|\text{CK}\rangle$ is *quantum*, and has size proportional to the number of qubits n in the state $|\psi\rangle$ to be committed.

Without going too deep into their construction, we remark that it makes use of a common tool in quantum cryptography: quantum coset states [CLLZ21]. A quantum coset state is defined by a random subspace $S \subset \mathbb{Z}_2^\lambda$ and shift $v \in \mathbb{Z}_2^\lambda$, and is defined as

$$|S + v\rangle := \frac{1}{\sqrt{|S|}} \sum_{x \in S} |x + v\rangle \quad .$$

As a major part of their security proof, [BKNY23] prove a delicate information-theoretic statement about coset states, which we refer to as the **delayed collapsing** property in this work. In a nutshell, they consider a classical oracle $\mathcal{O}$ such that quantum access to $\mathcal{O}$ enables implementing the projector onto the coset state, but when access to $\mathcal{O}$ is revoked, the coset state becomes indistinguishable from the coset state measured in the standard basis.

To elaborate, the ability to verify cosets is given by quantum oracle access to the classical membership check for the sets $S + v$ and $S^\perp$, where $S^\perp \subseteq \mathbb{Z}_2^\lambda$ is the dual subspace of S (i.e., vectors that have an inner product of 0 with all vectors inside S). It is a seminal result [AC12] and common knowledge by now that quantum access to $S + v$ and $S^\perp$ enable implementation of the projector $|S + v\rangle\langle S + v|$, while still preserving unclonability of the state $|S + v\rangle$. However, the delayed collapsing property required by [BKNY23] is *stronger* than unclonability of $|S + v\rangle$.

⁴In fact, there is good reason to believe that this definition is not satisfiable at all, since a Hadamard basis decoding functionality intuitively gives the committer the ability to perform a projection in the Hadamard basis and thus change their value in the standard basis.

This delayed collapsing property is established in [BKNY23, Claim 4.10],⁵ by building on the inner-product adversary method of [AC12]. To be precise, the statement is as follows. A quantum adversary, given $|S + v\rangle$ and quantum oracle access to both $S + v$, $S^\perp$, outputs some state on registers $\mathcal{R}_1, \mathcal{R}_2$. The first part $\mathcal{R}_1$ holds a superposition of elements in the coset $S + v$. The second part $\mathcal{R}_2$ is possibly entangled with the first part $\mathcal{R}_1$, and holds arbitrary quantum information that was learned from the oracle access to $S + v$ and $S^\perp$. Then, the challenger "takes away" oracle access to the membership check $S^\perp$, flips a coin $b \leftarrow \{0, 1\}$, and either measures $\mathcal{R}_1$ or not depending on the bit b . The task is for the adversary to now guess b . Note that this would have been easy given access to $S^\perp$. That is, during the first stage, the coset state is publicly-verifiable and non-collapsing, and during the second stage, access to $S^\perp$ is revoked and the state becomes indistinguishable from collapsed.

Circling back to our publicly-verifiable PFCs, our desired security notion (again, formalized in Definition 7) is reminiscent of the delayed collapsing property of coset states. Observe that the binding of PFCs can be interpreted as "revocable equivocality" as follows: The adversary has quantum access to $\text{DecZ}(\text{dk}, \cdot, \cdot)$, $\text{DecX}(\text{dk}, \cdot, \cdot)$ and makes a commitment. Having access to both oracles (and in particular the Hadamard decoding oracle $\text{DecX}(\text{dk}, \cdot, \cdot)$), it may be possible to equivocate from a state that has a superposition of encodings of measurement result $b \in \{0, 1\}$, to encodings of the opposite measurement result $1 - b$. However, the security definition ensures that once access to $\text{DecX}(\text{dk}, \cdot, \cdot)$ is removed, equivocation becomes computationally intractable.

Indeed, the above connection between delayed collapsing and public verification of PFCs was observed and formalized in [BKNY23]. However, as mentioned, the previous work deals only with quantum (and long) keys. In our work we seek to construct a scheme with classical (and potentially short) keys, and to this end we develop new techniques.

2.2 Our Work: One-Shot Signatures with Delayed Collapsing

Our work begins with the natural idea to employ *one-shot signatures* (OSS) in order to generate the coset states required by [CK]. OSS is a powerful primitive in quantum cryptography, first defined in [AGKZZ0] and recently constructed for the first time in [SZ25]. The OSS of [SZ25] is defined with respect to public parameters that enable anyone to sample their own *unclonable* coset states, and to publicly verify these states. That is, the sampled coset states cannot be cloned even by the sampler that produced them. Moreover, the public parameters can be used to sample an unbounded polynomial number of such unclonable states.

From Quantum to Classical Keys (and, from long to short keys). In principle, the use of OSS may eliminate two unwanted properties of the [BKNY23] PFC in a single swipe: OSS can enable commitment keys that are both classical and short. However, it turns out that the previous results on OSS are insufficient for our needs.

The OSS of [SZ25] is given by three classical oracles (P, P^{-1}, D) . In our context here, P, P^{-1} can be thought of as the oracles that allow for generation of coset states, and D is the oracle that allows for verification of the quantum tokens. In particular, D can be used to detect collapsing and distinguish between a uniform superposition over the coset and a collapsed classical state. [SZ25] prove that the oracles (P, P^{-1}, D) allow to generate coset states that are unclonable,⁶ and are verifiable (in particular, non-collapsing) given the oracle D . For our PFCs, we will want the stronger, delayed collapsing property: We will further ask that once we revoke access to D , the generated states become indistinguishable from collapsed. More formally, we consider a game in which an adversary, given full access to (P, P^{-1}, D) , first samples a quantum coset state (and potentially some side information). Then, a challenger either measures the state or not in the standard basis, and eliminates access to the D oracle. Then, the adversary is required to tell which of the options occurred (this is formalized in Definition 10).

Proving the delayed collapsing property for OSS requires a careful reexamination of the [SZ25] proof strategy, which we do in Section 4.4. Our proof technique rests upon a new and simpler proof of the delayed

⁵The statement in [BKNY23, Claim 4.10] is slightly different, but implies the statement we discuss here.

⁶In fact, an even stronger statement is proven by [SZ25] showing the one cannot produce two classical coset elements derived from the same token.

collapsing property of coset states that implicitly formed the heart of the [BK^{NY}23] proof strategy. In fact, the inner-product adversary method approach of [BK^{NY}23] appears ill-suited for use in the context of OSS,⁷ and hence we develop a different approach to establishing delayed collapsing that interfaces better with the [SZ25] proof technique.

Roadmap of our proofs. In the remainder of the overview, we explain the high-level structure for our proofs. Recall that our main goal is to construct a scheme that has single-bit binding with public decodability (Definition 7). We next go over the main steps for achieving this, how this connects to OSS with delayed collapsing, and the proof of the delayed collapsing of the OSS construction of [SZ25].

Reducing single-bit binding PFCs to collapse binding PFCs. The first observation we make is that, in order to construct publicly-verifiable PFC with single-bit binding, it suffices to construct PFC with a property we refer to as collapse binding (Definition 9). The scheme satisfies our notion of collapse binding if the following experiment has negligible advantage.

- The verifier samples $(CK, dk) \leftarrow \text{Gen}(1^\lambda)$.
- The adversary, with access to $CK, \text{DecZ}[dk], \text{DecX}[dk]$, outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U})$.
- The verifier samples a bit $b \leftarrow \{0, 1\}$ and either measures the registers $(\mathcal{B}, \mathcal{U})$ in the standard basis or not depending on the value of b .
- The adversary, given $CK, \text{DecZ}[dk]$, tries to guess the value of b .

To show the implication, we pass through a sequence of related notions of commitment binding. There are two main steps.

1. First, we show that any commitment scheme satisfying collapse binding (Definition 9) also satisfies *single-bit* collapse binding.⁸ Roughly, the setting of single-bit collapse binding is similar to that of collapse binding, but rather than measuring the registers $\mathcal{B}, \mathcal{U}$ in the computational basis, only the *first bit* of $(\mathcal{B}, \mathcal{U})$ is measured.
2. Second, it was established in [BK^{NY}23] that single-bit collapse binding implies a notion called "unique message" binding, which further implies a notion called string binding. Finally, string binding implies single-bit binding with public decodability.

Reducing collapse binding PFCs to OSS with delayed collapsing. It remains to construct a publicly-verifiable PFC with collapse binding. Our construction is given in Section 4.2 and is based on the OSS construction of [SZ25], which we denote by $\overline{\text{OSS}}$. In the body of the paper, we show that our construction is indeed a publicly-verifiable PFC with collapse binding, as long as $\overline{\text{OSS}}$ satisfies the property we informally discussed earlier, referred to as delayed collapsing. This game is formalized in Definition 10.

Delayed collapsing is captured by the advantage of query-bounded adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ in following experiment:⁹

1. Sample $(P, P^{-1}, D) \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda)$. Let $\{S_y\}_{y \in \{0, 1\}^r}$ be the cosets defined by P, P^{-1} .
2. Run $\mathcal{A}_1^{P, P^{-1}, D}(1^\lambda)$ until it outputs a string $y \in \{0, 1\}^r$ and a state on registers $(\mathcal{U}, \mathcal{R})$. Here, $\mathcal{U}$ is a k -qubit register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.

⁷This was the approach taken in the original but flawed attempt at proving security of OSS [AGKZ20].

⁸Note the names slightly defer, and "single-bit collapse binding" is not the exact same as our end goal security of "single-bit binding with public decodability."

⁹The following elaboration necessitates some familiarity with the the inner workings of $\overline{\text{OSS}}$, which can be found in Section 3.3.

3. Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $\mathcal{U}$ in the standard basis.
4. Run $\mathcal{A}_2^{P, P^{-1}}(\mathcal{U}, \mathcal{R})$ until it outputs a bit b' . The experiment outputs 1 if $b = b'$ and 0 otherwise.

Our reduction from collapse binding security of PFC to the delayed collapsing of $\overline{\text{OSS}}$ requires an *additional* use of OSS,¹⁰ though for this we only require standard unforgeability of OSS, as already shown by [SZ25].

Reducing OSS with delayed collapsing to a coset collapsing game. This reduction captures the heart of our technical contribution, as it proves the delayed collapsing property of $\overline{\text{OSS}}$. This is done in three main steps: (1) We propose a novel information-theoretic property of coset states, which we formalize as the *coset collapsing game*. (2) We perform a careful analysis of $\overline{\text{OSS}}$, which deviates from the previous security analysis of [SZ25] and ultimately reduces the delayed collapsing property of $\overline{\text{OSS}}$ to the hardness of the coset collapsing game. (3) We prove the information-theoretic hardness of the coset collapsing game.

The coset collapsing game (formalized in Definition 11) is a game between a challenger and a two-stage adversary $(\mathcal{A}_1, \mathcal{A}_2)$. There is a public subspace S known to all parties and the game executes as follows:

- The challenger samples a random subspace $T \subset S$, and grants $\mathcal{A}_1$ access to $O_{T^\perp}$, the membership checking oracle for $T^\perp$, which is a random superspace of $S^\perp$.
- $\mathcal{A}_1^{O_{T^\perp}}$ generates a state on register $\mathcal{R}$, to which the challenger applies a projective measurement onto a coset of T in S . That is, the left-over state will be in *some* superposition over *some* coset parallel to T (i.e., a set of the form $T + x$ for some x).
- With equal probability, the challenger either
 - Does nothing further to $\mathcal{R}$, or
 - Measures $\mathcal{R}$ in the standard basis.
- Finally $\mathcal{A}_2$ is given $\mathcal{R}$ but no access to $O_{T^\perp}$.

We say that "cosets of T are collapsing" if no $(\mathcal{A}_1, \mathcal{A}_2)$ has non-negligible advantage in guessing whether the final measurement was performed or not. In the proof of Theorem 6, we show that any adversary for the delayed collapsing of $\overline{\text{OSS}}$ can be turned into an adversary for the coset collapsing game. As mentioned above, this proof follows the security analysis of [SZ25] for some of the steps, but deviates in others. For example, while [SZ25] work towards a reduction to the collision-resistance of some "coset partition function", we reduce to the (non-delayed) collapsing property of the same affine partition function.

We will now give a proof sketch of the hybrids used in the reduction from the delayed collapsing property of $\overline{\text{OSS}}$ to the hardness of the coset collapsing game.

Step 1: Bloating the dual. Like [SZ25], we first bloat the dual oracle D and move from (P, P^{-1}, D) (see Definition 3) to (P, P^{-1}, D') , where D' checks membership for a random superspaces $T_y^\perp$ of $S_y^\perp$. A query bounded adversary will not notice the difference, since points in $T_y^\perp \setminus S_y^\perp$ are hard to find. Concretely, we bloat the dual oracle as follows

$$D'(y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^\top \mathbf{A}_y^{(1)} = \mathbf{0}^{n-r-s} \\ 0 & \text{otherwise.} \end{cases}$$

Here $\mathbf{A}_y^{(1)}$ refers to only the last $n - r - s$ columns of the matrix $\mathbf{A}_y$, which defines $S_y^\perp$. Identically to [SZ25], the superspaces $T_y^\perp$ are chosen such that they are sparse subsets of the entire space, and each $S_y^\perp$ is a sparse subset of $T_y^\perp$.

¹⁰In [BKNY23], this role was fulfilled by a signature token scheme with quantum keys.

Step 2: Simulating the dual to introduce a collapsing coset partition function. Like in [SZ25], we now eliminate the dual oracle altogether by embedding a rerandomization of the cosets defined by a smaller instance of the $(\tilde{P}, \tilde{P}^{-1}, \tilde{D})$ into the cosets defined by the actual oracles (P, P^{-1}, D) . This allows us to simulate *without access to $\tilde{D}$* the corresponding random superspace $T_y^\perp$ of the dual $S_y^\perp$. By the previous step, the adversary cannot tell the difference. Once we have eliminated the dual oracles, the adversary effectively has access only to $\tilde{P}, \tilde{P}^{-1}$ of these smaller instances, which we are now free to simulate using a collapsing coset-partition function Q .

Concretely, by combining Lemmas 31 and 34 of [SZ25], we simulate the oracles P, P^{-1}, D' ¹¹ using a coset partition function $Q : \{0, 1\}^{r+s} \rightarrow \{0, 1\}^s$ (see Definition 4) as follows:

1. Sample a random permutation $\Gamma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ and for every y , choose a random full rank matrix $\mathbf{C}_y \in \mathbb{Z}_2^{k \times n}$, and a random vector $\mathbf{d}_y \in \mathbb{Z}_2^k$.
2. Simulating $P(x \in \{0, 1\}^n)$.
 - $(x_0 \in \{0, 1\}^{r+s}, x_1 \in \{0, 1\}^{n-r-s}) \leftarrow \Gamma(x)$.
 - $y \leftarrow Q(x_0)$.
 - $\mathbf{u} \leftarrow (\mathbf{C}_y \cdot \Gamma(x) + \mathbf{d}_y)$.
 - Output $(y, \mathbf{u})$.
3. Simulating $P^{-1}(y \in \{0, 1\}^r, \mathbf{u} \in \mathbb{Z}_2^k)$.
 - $z \leftarrow \mathbf{C}_y^{-1} \cdot (\mathbf{u} - \mathbf{d}_y)$.
 - $(z_0 \in \{0, 1\}^{r+s}, z_1 \in \{0, 1\}^{n-r-s}) := z$.
 - If $Q(z_0) = y$, output $\Gamma^{-1}(z)$. Otherwise, output $\perp$.
4. Simulating $D'(y \in \{0, 1\}^r, \mathbf{v} \in \mathbb{Z}_2^k)$
 - $\mathbf{A}_y^{(1)\perp} :=$ last $n - r - s$ columns of $\mathbf{C}_y$.
 - Output 1 iff $\mathbf{v}^T \cdot \mathbf{A}_y^{(1)} = 0^{n-r-s}$.

Step 3: Utilizing the collapsing of coset partition function. In this part of the hybrid argument, we utilize the fact that our coset partition function Q is a collapsing hash function, which essentially means that, for any superposition of inputs (corresponding to a particular output) to Q that the adversary can produce, it is computationally infeasible to tell whether a challenger measured the superposition in the computational basis, or didn't measure the superposition at all. Now more concretely, we show that, for a fixed y , $\mathcal{A}_1^Q$ produces the following state on registers $(\mathcal{U}, \mathcal{R})$,

$$\sum_{\overline{\Gamma(x)} : Q(\overline{\Gamma(x)}) = y, \Gamma(x)} \mathbf{C}_y \begin{bmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{bmatrix} + \mathbf{d}_y$$

Since Q is collapsing, this state is indistinguishable from the state

$$\sum_{\Gamma(x)} \mathbf{C}_y \begin{bmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{bmatrix} + \mathbf{d}_y$$

where $\overline{\Gamma(x)}$ is some fixed value such that $Q(\overline{\Gamma(x)}) = y$.

¹¹The following elaboration necessitates some familiarity with the inner workings of $\overline{\text{OSS}}$, which can be found in Section 3.3.

Step 4: Unsimulating the bloated dual. Since Q is a $(r+s, r, s)$ coset partition function, the preimage set $Q^{-1}(y)$ has size 2^s and is a coset of a linear space of dimension s . Using this fact, crucially, we observe that collapsing the preimages of Q essentially projects the $\mathcal{U}$ register to a superposition over a coset of $\text{colspan}(\mathbf{A}_y^{(1)})$ in $\text{colspan}(\mathbf{A}_y)$. Therefore, using knowledge of $\mathbf{A}_y$, the reduction can discard Q , and actively collapse to cosets of $\text{colspan}(\mathbf{A}_y^{(1)})$. Thus, through a series of hybrids, we show that the state output by $\mathcal{A}_1$ is a superposition over a coset of a *specific* subspace of $\text{colspan}(\mathbf{A}_y)$, namely $\text{colspan}(\mathbf{A}_y^{(1)})$. This is the coset collapsing game with $T = \text{colspan}(\mathbf{A}_y^{(1)})$, and $S = \text{colspan}(\mathbf{A}_y)$. Now, the final few hybrids of our reduction serve to randomize T , at which point the game is exactly reduced to the coset collapsing game.

Information-theoretic hardness of the coset collapsing game. It remains to show hardness of the coset collapsing game. Intuitively, since T is a random subspace of S chosen by the challenger, it should be hard for any adversary to distinguish whether or not a measurement on a coset of T in S was performed, *after* access to $T^\perp$ has been revoked. This is established in three steps.

- Un-bloat the dual from $T^\perp$ to $S^\perp$. It is standard practice that access to $O_{T^\perp}$ for a random superspace $T^\perp$ can be shown to be indistinguishable from access to $O_{S^\perp}$. However, this only holds if $T^\perp$ is random and *independent of the rest of the experiment*. In our case, note that the challenger also depends on T to perform its intermediate coset measurement. Our saving grace is that we only need to argue about indistinguishability from $O_{S^\perp}$ *before* the challenger’s measurement, since the adversary’s access to this oracle is revoked afterwards. In fact, this is the **only** place in our proof where we make use of the crucial fact that the adversary’s access to the dual oracle is revoked after the challenge phase.
- Since S is public, access to $O_{S^\perp}$ gives no additional information to the adversary, so we can ignore it.
- Re-viewing our game in this modified setting, the adversary’s success in this game is exactly the diamond distance between the standard basis measurement channel and the coset-measurement channel over a random choice of subspace. This is something that can be directly analyzed, and we formally show this diamond distance to be exponentially small in Lemma 5.

2.3 From PFCs to QFHE and Obfuscation

Once we are equipped with publicly-verifiable PFCs with classical keys, our main results follow naturally from the outline proposed in [BK~~NY~~23], with one notable exception. In particular, we use PFCs to upgrade a privately-verifiable QFHE scheme satisfying certain properties to a publicly-verifiable scheme. This compiler is presented in Section 6. Then, given a publicly-verifiable QFHE (in the classical oracle model) we use a straightforward “verify-then-decrypt” construction to obtain classical obfuscation for pseudo-deterministic quantum circuits in the classical oracle model (Section 7).

The notable departure from [BK~~NY~~23] mentioned above is necessitated by our requirement of *succinctness*. While neither the privately-verifiable QFHE scheme we start with, nor the PFC-based compiler are themselves succinct, we show how to restore succinctness with a generic approach based on (post-quantum) succinct ideal obfuscation of *classical* circuits, in the classical oracle model. More concretely, given a Turing machine P (which may take in a long input, but has a short output), a post-quantum succinct ideal obfuscator outputs a circuit $\tilde{P}$ of size bounded by $|P|$, which is the size of the P ’s *code*, as opposed to its *run-time*. We provide the first construction of this object by combining (classical) FHE and post-quantum non-interactive succinct arguments of knowledge in the quantum random oracle model [CMS19]. The proof establishing ideal obfuscation of our construction requires a careful application of techniques from [CMS19], which we formalize in Section 5.

We describe some high level ideas below:

Succinct Ideal Obfuscation Classically, an input succinct obfuscator can be constructed through fully homomorphic encryption (FHE) and SNARKs (succinct non-interactive arguments of knowledge) as follows:

the obfuscated circuit $\tilde{P}$ consists of the FHE encryption of P , $\text{FHE.Enc}(P)$, along with an oracle O_{sk} , which has the FHE encryption secret key sk hardcoded. O_{sk} takes as input an evaluated ciphertext and a SNARK of its honest evaluation. It verifies the SNARK and outputs the decryption of the ciphertext (using sk) iff the verification procedure outputs 1.

In the setting where an adversary only makes classical queries to O_{sk} , prior work ([BCP14]) establishes a simulation strategy (and therefore establishes ideal obfuscation). However, in our setting, the adversary makes *superposition queries* to O_{sk} . In order to show a simulation strategy in this setting, we utilize the knowledge extractor for Micali's SNARK construction [Mic94] in the quantum random oracle model (QROM). In more detail, [CMS19] showed that the SNARK construction of Micali, which maps any probabilistically checkable proof (PCP) into a corresponding non-interactive argument, is unconditionally secure in the QROM. Along the way, they also prove that the Micali construction inherits proof of knowledge properties from the underlying PCP, by constructing an extractor $\mathcal{E}^*$, which, given a valid proof π (one that passes SNARK verification), outputs a valid witness with high probability. Our simulation strategy, which we call the **FindWitness** procedure, is essentially the witness extractor $\mathcal{E}^*$, with slight changes required to handle superposition query access. In more detail, on input a SNARK proof π and some evaluated ciphertext ct , our simulator runs the **FindWitness** procedure *coherently* iff π is a valid proof. Then, by utilizing the guarantees provided by $\mathcal{E}^*$, as well as the compressed oracle method of Zhandry [Zha19b], we show that the simulator can extract the intended (potentially large) input x and query its oracle, establishing ideal obfuscation security.

3 Preliminaries

3.1 Notation

Definition 1 (Pseudo-deterministic quantum circuit). *A family of pseudo-deterministic quantum circuits $\{Q_\lambda\}_{\lambda \in \mathbb{N}}$ is defined as follows. The circuit Q_λ takes as input a classical string $x \in \{0, 1\}^{n(\lambda)}$ and outputs a bit $b \leftarrow Q_\lambda(x)$. The circuit is pseudo-deterministic if for every sequence of classical inputs $\{x_\lambda\}_{\lambda \in \mathbb{N}}$, there exists a sequence of outputs $\{b_\lambda\}_{\lambda \in \mathbb{N}}$ such that*

$$\Pr[Q_\lambda(x_\lambda) \rightarrow b_\lambda] = 1 - \text{negl}(\lambda).$$

We will often leave the dependence on λ implicit, and just refer to pseudo-deterministic circuits Q with input x . In a slight abuse of notation, we will denote by $Q(x)$ the bit b such that $\Pr[Q(x) \rightarrow b] = 1 - \text{negl}(\lambda)$.

Sometimes, we will consider (potentially succinct) descriptions of pseudo-deterministic circuits, which we write as P_Q , and refer to as (pseudo-deterministic) programs. That is, while Q is understood to consist of the list of gates in the circuit, P_Q is understood to be an arbitrary (potentially short) description from which the entire list of gates Q can be derived. Thus, in general it may be the case that $|P_Q| \ll |Q|$.

3.2 Fully-homomorphic encryption

We will use fully-homomorphic encryption for classical circuits, defined as follows.

Definition 2 (FHE). *A fully-homomorphic encryption scheme consists of the following algorithms.*

- $\text{Gen}(1^\lambda, D) \rightarrow (\text{pk}, \text{sk})$: *The p.p.t. key generation algorithm takes as input the security parameter 1^λ and a circuit depth D , and returns a public key pk and secret key sk .*
- $\text{Enc}(\text{pk}, x) \rightarrow \text{ct}$: *The p.p.t. encryption algorithm takes as input the public key pk and a plaintext x , and outputs a ciphertext ct .*
- $\text{Eval}(\text{ct}, C) \rightarrow \tilde{\text{ct}}$: *The classical deterministic evaluation algorithm takes as input a ciphertext ct and the description of a classical deterministic computation C , and outputs an evaluated ciphertext $\tilde{\text{ct}}$.*

- $\text{Dec}(\text{sk}, \text{ct}) \rightarrow x$: The classical deterministic decryption algorithm takes as input the secret key sk and a ciphertext ct and outputs a plaintext x .

It should satisfy the following properties.

- **Correctness.** For any $(\text{pk}, \text{sk}) \in \text{Gen}(1^\lambda, D)$, $\text{ct} \in \text{Enc}(\text{pk}, x)$ and circuits C of depth at most D

$$\Pr \left[\text{Dec}(\text{sk}, \text{ct}') = C(x) : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x) \\ \text{ct}' \leftarrow \text{Eval}(\text{ct}, C) \end{array} \right] = 1 - \text{negl}(\lambda).$$

- **Privacy.** For any QPT adversary $\mathcal{A} = \{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$, polynomial D and messages x_0, x_1 ,

$$\left| \Pr \left[\mathcal{A}(\text{pk}, \text{ct}) = 1 : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, D) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x_0) \end{array} \right] - \Pr \left[\mathcal{A}(\text{pk}, \text{ct}) = 1 : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, D) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x_1) \end{array} \right] \right| = \text{negl}(\lambda).$$

- **Compactness.** There exists a fixed polynomial p such that any classical circuit C of depth at most D and it holds that

- $|\text{pk}|, |\text{ct}|, |\text{ct}'| \leq p(\lambda, D, |x|)$ and
- the runtimes $\text{Gen}(1^\lambda, D, |x|)$, $\text{Enc}(\text{pk}, x)$, $\text{Dec}(\text{sk}, \text{ct}')$ are bounded by $p(\lambda, D, |x|)$.

3.3 One-shot signatures (OSS)

Definition 3 (One-Shot Signature Scheme). A one-shot signature (OSS) scheme is a tuple of algorithms $\text{OSS} = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ satisfying the following:

- $\text{Setup}(1^\lambda) \rightarrow \mathcal{O}$: A classical probabilistic polynomial-time algorithm that given the security parameter 1^λ , outputs a classical deterministic circuit $\mathcal{O}$.
- $\text{Gen}^\mathcal{O}(\ell) \rightarrow (\text{vk}, |\text{sk}\rangle)$: A quantum polynomial-time algorithm that takes as input a message length ℓ , has oracle access to $\mathcal{O}$, and samples a classical public key vk and quantum secret key $|\text{sk}\rangle$.
- $\text{Sign}^\mathcal{O}(|\text{sk}\rangle, m) \rightarrow \sigma$: A quantum polynomial-time algorithm that has oracle access to $\mathcal{O}$, and, given the quantum key $|\text{sk}\rangle$ and message $m \in \{0, 1\}^\ell$ produces a classical signature σ .
- $\text{Ver}^\mathcal{O}(\text{vk}, m, \sigma) \rightarrow \{\perp, \top\}$: A classical deterministic polynomial-time algorithm that has oracle access to $\mathcal{O}$ and verifies a message m and signature σ relative to a public key vk .

Correctness: For any ℓ and $m \in \{0, 1\}^\ell$,

$$\Pr_{\substack{\mathcal{O} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ (\text{vk}, |\text{sk}\rangle) \leftarrow \text{Gen}^\mathcal{O} \\ \sigma \leftarrow \text{Sign}^\mathcal{O}(|\text{sk}\rangle, m)}} [\text{Ver}^\mathcal{O}(\text{vk}, m, \sigma) = \top] \geq 1 - \text{negl}(\lambda).$$

Security (Strong unforgeability): For any ℓ and QPT algorithm $\mathcal{A}$,

$$\Pr_{\substack{\mathcal{O} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ (\text{vk}, m_0, m_1, \sigma_0, \sigma_1) \leftarrow \mathcal{A}^\mathcal{O}}} [\text{Ver}^\mathcal{O}(\text{vk}, m_0, \sigma_0) = \top \wedge \text{Ver}^\mathcal{O}(\text{vk}, m_1, \sigma_1) = \top \wedge \sigma_0 \neq \sigma_1] \leq \text{negl}(\lambda).$$

Note that an adversary breaks the strong unforgeability security guarantee even if it produces two different signatures $\sigma_0 \neq \sigma_1$ of the same message $m_0 = m_1$.

We now describe the construction of one-shot signatures in [SZ25]. We will later use this construction in a non-black-box way in Section 4.2.

Parameters: Let $\lambda \in \mathbb{N}$ be the security parameter. Let $s = 16\lambda$ and let $n, r, k \in \mathbb{N}$ be such that $r = s \cdot (\lambda - 1)$, $n = r + \frac{3}{2} \cdot s$ and $k = n$.

- $\text{OSS.Setup}(1^\lambda)$

- Sample a random permutation $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$. Let $H(x)$ be the first r output bits of $\Pi(x)$ and let $J(x)$ denote the remaining $n - r$ bits. Interpret $J(x) \in \mathbb{Z}_2^{n-r}$.

$$\Pi(x) = H(x) \| J(x).$$

- Sample a random function $F : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$. Interpret $F(y) = (\mathbf{A}_y, \mathbf{b}_y)$ where $\mathbf{A}_y \in \mathbb{Z}_2^{k \times (n-r)}$ is a full-rank matrix and $\mathbf{b} \in \mathbb{Z}_2^n$.
Let $S_y = \{\mathbf{A}_y \mathbf{x} + \mathbf{b}_y \mid \mathbf{x} \in \mathbb{Z}_2^{n-r}\}$ be the coset defined by $(\mathbf{A}_y, \mathbf{b}_y)$. Let $S_{y,b} := \{\mathbf{u} \in S_y \mid u_1 = b\}$ be the affine subspace of S_y whose first coordinate entry is b .
- Define the functions P, P^{-1}, D as follows

$$\begin{aligned} P(x) &= (y, \mathbf{A}_y J(x) + \mathbf{b}_y) \in \{0, 1\}^n \rightarrow \{0, 1\}^r \times \mathbb{Z}_2^k \text{ where } y = H(x) \\ P^{-1}(y, \mathbf{u}) &= \begin{cases} \Pi^{-1}(y, \mathbf{z}) & \text{if } \exists \mathbf{z} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}_y \mathbf{z} + \mathbf{b}_y = \mathbf{u} \\ \perp & \text{otherwise.} \end{cases} \\ D(y, \mathbf{v}) &= \begin{cases} 1 & \text{if } \mathbf{v}^\top \mathbf{A}_y = \mathbf{0}^{n-r} \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

Let $S_y^\perp := \{\mathbf{v} \mid D(y, \mathbf{v}) = 1\}$ be the subspace dual to S_y .

- Output $\mathbf{O} := (P, P^{-1}, D)$.

- $\text{OSS.Gen}^{\mathbf{O}}(1^\lambda)$

- Parse $\mathbf{O} = (P, P^{-1}, D)$.¹²
- Prepare a uniform superposition over all strings $x \in \{0, 1\}^n$ on register $\mathcal{X}$, and using the P oracle, compute $P(x) := (y, \mathbf{u})$ onto registers $\mathcal{Y}, \mathcal{U}$. Measure register $\mathcal{Y}$ to get y .
- Use the P^{-1} oracle on registers $(\mathcal{Y}, \mathcal{U})$ to uncompute the $\mathcal{X}$ register.
- Output $\text{vk} := y$ and the state on $\mathcal{U}$ as $|\text{sk}\rangle$.

- $\text{OSS.Sign}^{\mathbf{O}}(|\text{sk}\rangle, m \in \{0, 1\})$

- Parse $\mathbf{O} = (P, P^{-1}, D)$.
- Let $\mathcal{U}$ be the register that $|\text{sk}\rangle$ lives on. Measure the first qubit $\hat{m}$ of the $\mathcal{U}$ register in the standard basis. If the result is $\hat{m} = m$, then measure the entire $\mathcal{U}$ register in the standard basis to get $\mathbf{u}$ and output $\mathbf{u}$.
- If $\hat{m} \neq m$, apply the rotation $H^{\otimes n} \circ \text{Phase}^D \circ H^{\otimes n}$ to the $\mathcal{U}$ register, where Phase^D is the map $|\mathbf{v}\rangle \mapsto (-1)^{D(\mathbf{v})} |\mathbf{v}\rangle$. Measure the register $\mathcal{U}$ to get $\mathbf{u}$. Abort if the first bit of $\mathbf{u}$ is 0, and otherwise output $\mathbf{u}$.

- $\text{OSS.Ver}^{\mathbf{O}}(\text{vk}, m, \sigma)$

- Parse $\text{vk} = y$ and $\sigma = \mathbf{u}$.
- If the first bit of $\mathbf{u}$ is not m and reject.
- Compute $P^{-1}(y, \mathbf{u})$ and reject if the output is $\perp$. Accept otherwise.

¹²In the construction of Pauli functional commitments in Section 4.2, it will be relevant that OSS.Gen does not actually need access to the D oracle, the oracles P and P^{-1} suffice.

3.4 Measure and re-program

Theorem 3 (Measure and re-program [DFMS19, DFM20]¹³). *Let A, B be finite non-empty sets, and let $q \in \mathbb{N}$. Let $\mathcal{A}$ be an oracle-aided quantum circuit that makes q queries to a uniformly random function $H : A \rightarrow B$ and then outputs classical strings (a, z) where $a \in A$. There exists a two-stage quantum circuit $\text{Sim}[\mathcal{A}]$ such that for any predicate V , it holds that*

$$\Pr \left[\begin{array}{l} (a, \text{state}) \leftarrow \text{Sim}[\mathcal{A}] \\ b \leftarrow B \\ z \leftarrow \text{Sim}[\mathcal{A}](b, \text{state}) \end{array} : V(a, b, z) = 1 \right] \geq \frac{\Pr [V(a, H(a), z) = 1 : (a, z) \leftarrow \mathcal{A}^H]}{(2q + 1)^2}.$$

Moreover, $\text{Sim}[\mathcal{A}]$ operates as follows.

- Sample $H : A \rightarrow B$ as a $2q$ -wise independent function and $(i, d) \leftarrow (\{0, \dots, q-1\} \times \{0, 1\}) \cup \{(q, 0)\}$.
- Run $\mathcal{A}$ until it has made i oracle queries, answering each query using H .
- When $\mathcal{A}$ is about to make its $(i+1)$ 'th oracle query, measure its query registers in the standard basis to obtain a . In the special case that $(i, d) = (q, 0)$, the simulator measures (part of) the final output register of $\mathcal{A}$ to obtain a .
- The simulator receives $b \leftarrow B$.
- If $d = 0$, answer $\mathcal{A}$'s $(i+1)$ 'th query using H , and if $d = 1$, answer $\mathcal{A}$'s $(i+1)$ 'th query using $H[a \rightarrow b]$, which is the function H except that $H(a)$ is re-programmed to b .
- Run $\mathcal{A}$ until it has made all q oracle queries. For queries $i+2$ through q , answer using $H[a \rightarrow b]$.
- Measure $\mathcal{A}$'s output z .

Note that the running time of $\text{Sim}[\mathcal{A}]$ is at most $\text{poly}(q, \log |A|, \log |B|)$ times the running time of $\mathcal{A}$.

3.5 The Compressed Oracle Technique

The compressed random oracle [Zha19a] acts on query and database registers $(\mathcal{Q}, \mathcal{D})$ as follows:

$$\text{CStO} = \text{Decomp} \circ \text{CStO}' \circ \text{Decomp} \circ \text{Increase},$$

where $\text{Increase} |x, y\rangle \otimes |D\rangle = |x, y\rangle \otimes |D\rangle |(\perp, 0^n)\rangle$ essentially adds another point in the database, and where Decomp and CStO' are defined as follows:

- Decomp is defined by $|x, u\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \mapsto |x, u\rangle_{\mathcal{Q}} \otimes \text{Decomp}_x |D\rangle_{\mathcal{D}}$, where Decomp_x is defined by its action on basis states as follows. Let D be such that $D(x) = \perp$:

$$\begin{aligned} |D\rangle &\mapsto \frac{1}{\sqrt{|\mathcal{Y}|}} \sum_{y \in \mathcal{Y}} |D \cup \{(x, y)\}\rangle \\ \frac{1}{\sqrt{|\mathcal{Y}|}} \sum_{y \in \mathcal{Y}} |D \cup \{(x, y)\}\rangle &\mapsto |D\rangle \\ \frac{1}{\sqrt{|\mathcal{Y}|}} \sum_{y \in \mathcal{Y}} (-1)^{y \cdot u} |D \cup \{(x, y)\}\rangle &\mapsto \frac{1}{\sqrt{|\mathcal{Y}|}} \sum_{y \in \mathcal{Y}} (-1)^{y \cdot u} |D \cup \{(x, y)\}\rangle \quad \text{for } u \neq 0 \end{aligned}$$

In words, Decomp_x swaps the states $|D\rangle$ and $\frac{1}{\sqrt{|\mathcal{Y}|}} \sum_{y \in \mathcal{Y}} |D \cup \{(x, y)\}\rangle$, and acts as the identity on the space orthogonal to these two states.

- CStO' maps $|x, u\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \mapsto |x, u \oplus D(x)\rangle \otimes |D\rangle$.

¹³This theorem was stated more generally in [DFMS19, DFM20] to consider the drop in expectation for each specific $a^* \in A$, and also to consider a more general class of quantum predicates.

3.6 Useful Lemmas and Definitions

Definition 4 (Coset Partition Functions). [SZ25] For $n, \ell \in \mathbb{N}$, such that $\ell \leq n$, we say a function $Q : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is a (n, m, ℓ) coset partition function if, for each y in the image of Q , the preimage set $Q^{-1}(y)$ has size 2^ℓ and is a coset of a linear space of dimension ℓ . Different preimage sets are allowed to be cosets of different linear spaces.

Lemma 1 (Lemma 3.6 of [BKNY23]). For each $\lambda \in \mathbb{N}$, let $\mathcal{K}_\lambda$ be a set of keys, and let $\{|\psi_k\rangle, O_k^0, O_k^1, B_k\}_{k \in \mathcal{K}_\lambda}$ be a set of state $|\psi_k\rangle$, classical functions O_k^0, O_k^1 , and sets of inputs B_k . Suppose that the following properties holds.

1. The oracles O_k^0, O_k^1 are identical on inputs outside of S_k .
2. For any oracle-aided unitary U , with $q = q(\lambda)$ queries, there is some $\varepsilon = \varepsilon(\lambda)$ such that¹⁴

$$\mathbb{E}_{k \leftarrow \mathcal{K}} \left[\|\Pi[B_k] U^{O_k^0} |\psi_k\rangle\|^2 \right] \leq \varepsilon$$

Then, for any oracle-aided unitary U with $q = q(\lambda)$ queries and for every distinguisher D ,

$$\left| \Pr_{k \leftarrow \mathcal{K}} \left[D(k, U^{O_k^0} |\psi_k\rangle) = 0 \right] - \Pr_{k \leftarrow \mathcal{K}} \left[D(k, U^{O_k^1} |\psi_k\rangle) = 0 \right] \right| \leq 4q\sqrt{\varepsilon}.$$

Small-range distribution. [Zha21] Given a distribution D on $\mathcal{Y}$, define $\text{SR}_r^D(\mathcal{X})$ as the following distribution on functions from $\mathcal{X}$ to $\mathcal{Y}$:

- For each $i \in [r]$, choose a random value $y_i \in \mathcal{Y}$ according to the distribution D .
- For each $x \in \mathcal{X}$, pick a random $i \in [r]$ and set $O(x) = y_i$.

Lemma 2 (Corollary 7.5 of [Zha21]). The output distributions of a quantum algorithm making q quantum queries to an oracle either drawn from $\text{SR}_r^D(\mathcal{X})$ or $D^{\mathcal{X}}$ are $\ell(q)/r$ -close, where $\ell(q) = \pi^2(2q)^3/3 < 27q^3$.

Lemma 3 ([Zha13], Corollary 2 of [Zha19a]). After making q quantum queries to a random oracle, the probability of finding a collision is at most $O(q^3/2^n)$.

4 Pauli Functional Commitments with Classical Keys

4.1 Definition

We recall the definition of Pauli functional commitments from [BKNY23]. The difference here is that, while [BKNY23] allow the commitment key to be a quantum state, we require the commitment key to be classical.

Definition 5 (Syntax). A Pauli functional commitment consists of six algorithms ($\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX}$) with the following syntax.

- **Key Generation:** $\text{Gen}(1^\lambda) \rightarrow (\text{CK}, \text{dk})$ is a p.p.t. algorithm that takes as input the security parameter 1^λ and outputs a classical deterministic circuit CK and a classical decoding key dk .
- **Commitment:** $\text{Com}^{\text{CK}}(b) \rightarrow (\mathcal{U}, c)$ is a QPT algorithm that takes as input a bit b and has oracle access to CK . It outputs registers $(\mathcal{U}, \mathcal{C})$ and then measures $\mathcal{C}$ in the standard basis to obtain a classical string $c \in \{0, 1\}^*$ and a left-over state on register $\mathcal{U}$. We then write

$$\text{Com}^{\text{CK}} := |0\rangle\langle 0| \otimes \text{Com}^{\text{CK}}(0) + |1\rangle\langle 1| \otimes \text{Com}^{\text{CK}}(1)$$

¹⁴ $\Pi[B_k]$ is the projector onto subspace spanned by B_k . Informally what this means is that the probability of that U given query access to O_k^0 outputs $b \in B_k$ is at most ε .

to refer to the map that applies $\text{Com}^{\text{CK}}(\cdot)$ classically controlled on a single-qubit register $\mathcal{B}$ to produce a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{C})$, and then measures $\mathcal{C}$ in the standard basis to obtain a classical string c along with a left-over quantum state on registers $(\mathcal{B}, \mathcal{U})$.

- **Opening in Z, X bases:** q.p.t. measurements on registers $(\mathcal{B}, \mathcal{U})$ that outputs a classical string u :

$$\text{OpenZ}(\mathcal{B}, \mathcal{U}) \rightarrow u \quad \text{and} \quad \text{OpenX}(\mathcal{B}, \mathcal{U}) \rightarrow u.$$

- **Decoding in Z, X bases:** classical deterministic polynomial-time algorithm that takes as input the decoding key dk , a string c , and a string u , and outputs either a bit b or a $\perp$ symbol:

$$\text{DecZ}(\text{dk}, c, u) \rightarrow \{0, 1\} \cup \{\perp\} \quad \text{and} \quad \text{DecX}(\text{dk}, c, u) \rightarrow \{0, 1\} \cup \{\perp\}.$$

Definition 6 (Correctness). A Pauli functional commitment scheme $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ is correct if for any single-qubit (potentially mixed) state on register $\mathcal{B}$, it holds that

$$\text{TV}(Z(\mathcal{B}), \text{PFCZ}(1^\lambda, \mathcal{B})) = \text{negl}(\lambda), \quad \text{and} \quad \text{TV}(X(\mathcal{B}), \text{PFCX}(1^\lambda, \mathcal{B})) = \text{negl}(\lambda),$$

where the distributions are defined as follows.

- $Z(\mathcal{B})$ measures $\mathcal{B}$ in the standard basis.
- $X(\mathcal{B})$ measures $\mathcal{B}$ in the Hadamard basis.
- $\text{PFCZ}(1^\lambda, \mathcal{B})$ samples $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$, $(\mathcal{B}, \mathcal{U}, c) \leftarrow \text{Com}^{\text{CK}}(\mathcal{B})$, $u \leftarrow \text{OpenZ}(\mathcal{B}, \mathcal{U})$, and outputs $\text{DecZ}(\text{dk}, c, u)$.
- $\text{PFCX}(1^\lambda, \mathcal{B})$ samples $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$, $(\mathcal{B}, \mathcal{U}, c) \leftarrow \text{Com}^{\text{CK}}(\mathcal{B})$, $u \leftarrow \text{OpenX}(\mathcal{B}, \mathcal{U})$, and outputs $\text{DecX}(\text{dk}, c, u)$.

Definition 7 (Single-bit Binding with public decodability). A Pauli functional commitment $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies single-bit binding with public decodability if the following holds. Given dk , c , and $b \in \{0, 1\}$, define the following projective measurement

$$\Pi_{\text{dk}, c, b} := \sum_{d: \text{DecZ}(\text{dk}, c, d) = b} |d\rangle\langle d|.$$

Consider any adversary $\mathcal{A} = \{(\mathcal{A}_1, \mathcal{A}_2)_\lambda\}_{\lambda \in \mathbb{N}}$, where each $\mathcal{A}_1$ is an oracle-aided quantum operation, each $\mathcal{A}_2$ is an oracle-aided unitary, and each $(\mathcal{A}_1, \mathcal{A}_2)_\lambda$ make at most $\text{poly}(\lambda)$ oracle queries. Then for any $b \in \{0, 1\}$,

$$\mathbb{E} \left[\left\| \Pi_{\text{dk}, c, 1-b} \mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]} \Pi_{\text{dk}, c, b} |\psi\rangle \right\| : (|\psi\rangle, c) \leftarrow \mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda) \right] = \text{negl}(\lambda),$$

where the expectation is taken over $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$. Here, $\text{DecZ}[\text{dk}]$ is the oracle implementing the classical functionality $\text{DecZ}(\text{dk}, \cdot, \cdot)$, and $\text{DecX}[\text{dk}]$ is the oracle implementing the classical functionality $\text{DecX}(\text{dk}, \cdot, \cdot)$.

Our application to publicly-verifiable QFHE will in fact rely on the following notion of string binding, which was shown in [BKNY23] to be implied by single-bit binding.

Definition 8 (String binding with public decodability). A Pauli functional commitment $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies string binding with public decodability if the following holds for any polynomial $m = m(\lambda)$ and two disjoint sets $W_0, W_1 \subset \{0, 1\}^m$ of m -bit strings. Given a set of m decoding keys $\text{dk} = (\text{dk}_1, \dots, \text{dk}_m)$, m strings $\mathbf{c} = (c_1, \dots, c_m)$, and $b \in \{0, 1\}$, define

$$\Pi_{\text{dk}, \mathbf{c}, W_b} := \sum_{w \in W_b} \left(\bigotimes_{i \in [m]} \Pi_{\text{dk}_i, c_i, w_i} \right).$$

Consider any adversary $\{(\mathcal{A}_1, \mathcal{A}_2)_\lambda\}_{\lambda \in \mathbb{N}}$, where each $\mathcal{A}_1$ is an oracle-aided quantum operation, each $\mathcal{A}_2$ is an oracle-aided unitary, and each $(\mathcal{A}_1, \mathcal{A}_2)_\lambda$ make at most $\text{poly}(\lambda)$ oracle queries. Then,

$$\mathbb{E} \left[\left\| \Pi_{\mathbf{dk}, \mathbf{c}, W_1} \mathcal{A}_2^{\mathbf{CK}, \text{DecZ}[\mathbf{dk}]} \Pi_{\mathbf{dk}, \mathbf{c}, W_0} |\psi\rangle \right\| : (|\psi\rangle, \mathbf{c}) \leftarrow \mathcal{A}_1^{\mathbf{CK}, \text{DecZ}[\mathbf{dk}], \text{DecX}[\mathbf{dk}]}(1^\lambda) \right] = \text{negl}(\lambda),$$

where the expectation is over $\{\text{CK}_i, \text{dk}_i \leftarrow \text{Gen}(1^\lambda)\}_{i \in [m]}$. Here, $\mathbf{CK}$ is the collection of oracles $\text{CK}_1, \dots, \text{CK}_m$, $\text{DecZ}[\mathbf{dk}]$ is the collection of oracles $\text{DecZ}[\text{dk}_1], \dots, \text{DecZ}[\text{dk}_m]$, and $\text{DecX}[\mathbf{dk}]$ is the collection of oracles $\text{DecX}[\text{dk}_1], \dots, \text{DecX}[\text{dk}_m]$.

Lemma 4 (Lemma 4.5 of [BKNY23]). *Any Pauli functional commitment that satisfies single-bit binding with public decodability (Definition 7) also satisfies string binding with public decodability (Definition 8).*

Since string-binding when $m = 1$ is exactly the definition of single-bit binding, we have the following corollary.

Corollary 1. *The single-bit binding definition (Definition 7) and string binding definition (Definition 8) are equivalent.*

4.2 Construction

In this section, we will present our construction of a Pauli Functional Commitment scheme. Let $n = n(\lambda) \geq \lambda$. Our construction makes use of the following ingredients:

1. A general one-shot signature scheme $\text{OSS} = (\text{OSS.Setup}, \text{OSS.Gen}, \text{OSS.Sign}, \text{OSS.Ver})$.
2. The particular construction of one-shot signatures from [SZ25], which we will denote as $\overline{\text{OSS}}$.
3. PRF F .

Now, we are ready to present the construction.

- $\text{Gen}(1^\lambda; R)$:
 - Parse randomness $R = (R_1, R_2)$.
 - Run $\text{O} \leftarrow \text{OSS.Setup}(1^\lambda; R_1)$.
 - Sample a random PRF key k using randomness R_2 .
 - Define oracle $\text{CK}_P(\text{vk}, x)$ as follows:
 - * Run $\overline{\text{O}} \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda; F_k(\text{vk}))$ and parse $\overline{\text{O}} = (P, P^{-1}, D)$.
 - * Output $P(x)$.
 - Define oracle $\text{CK}_{P^{-1}}(\text{vk}, \overline{\text{vk}}, u)$ as follows:
 - * Run $\overline{\text{O}} \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda; F_k(\text{vk}))$ and parse $\overline{\text{O}} = (P, P^{-1}, D)$.
 - * Output $P^{-1}(\overline{\text{vk}}, u)$.
 - Define the oracle $\text{CK}_D(\text{vk}, \sigma, \overline{\text{vk}}, u)$ as follows:
 - * Run $\overline{\text{O}} \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda; F_k(\text{vk}))$ and parse $\overline{\text{O}} = (P, P^{-1}, D)$.
 - * If $\text{OSS.Ver}^{\text{O}}(\text{vk}, 0, \sigma) = \perp$, output $\perp$ and abort.
 - * Otherwise, output 0 if $D(\overline{\text{vk}}, u) = 1$ and 1 if $D(\overline{\text{vk}}, u) = 0$.
 - Output $\text{CK} = (\text{O}, \text{CK}_P, \text{CK}_{P^{-1}}, \text{CK}_D)$ and $\text{dk} = (R_1, k)$.
- $\text{Com}^{\text{CK}}(b)$:
 - Parse $\text{CK} = (\text{O}, \text{CK}_P, \text{CK}_{P^{-1}}, \text{CK}_D)$.

- Run $(vk, |sk\rangle) \leftarrow \text{OSS.Gen}^O(1^\lambda)$. Coherently apply $\text{OSS.Sign}^O(|sk\rangle, 0)$ to get a superposition over signatures σ on bit 0 on register $\mathcal{S}$. This signature register $\mathcal{S}$ will allow us to (coherently) access the oracle CK_D .
- Run $(\overline{vk}, |\overline{sk}\rangle) \leftarrow \overline{\text{OSS.Gen}}^{\text{CK}_P(vk, \cdot), \text{CK}_{P^{-1}}(vk, \cdot, \cdot)}(1^\lambda)$,¹⁵ where $\overline{vk} = y$ and $|\overline{sk}\rangle = |S_y\rangle$ on register $\mathcal{S}'$ for some y as determined by P . Measure the first qubit of $|S_y\rangle$ in the standard basis to get result b' . If $b = b'$ then the state has collapsed to $|S_{y,b}\rangle$ and we continue. Otherwise, perform a rotation from $|S_{y,b'}\rangle$ to $|S_{y,b}\rangle$ by applying the operation

$$(\text{H}^{\otimes n} \otimes \text{I}) \circ \text{Phase}^{\text{CK}_D(vk, \cdot, \cdot)} \circ (\text{H}^{\otimes n} \otimes \text{I})$$

to register $(\mathcal{S}', \mathcal{S})$, where $\text{Phase}^{\text{CK}_D(vk, \cdot, \cdot)}$ is the map

$$|u\rangle_{\mathcal{S}'} |\sigma\rangle_{\mathcal{S}} \mapsto (-1)^{\text{CK}_D(vk, \sigma, u)} |u\rangle_{\mathcal{S}'} |\sigma\rangle_{\mathcal{S}}.$$

- Reverse the $\text{OSS.Sign}^O(\cdot, 0)$ operation on $\mathcal{S}$ to recover $|sk\rangle$.
 - Sample $\sigma \leftarrow \text{OSS.Sign}^O(|sk\rangle, 1)$, and output $c = (vk, \sigma, \overline{vk})$ along with the final state on register $\mathcal{U} := \mathcal{S}'$.
- $\text{OpenZ}(\mathcal{B}, \mathcal{U})$: Measure all registers in the standard basis.
 - $\text{OpenX}(\mathcal{B}, \mathcal{U})$: Measure all registers in the Hadamard basis.
 - $\text{DecZ}(dk, c, d)$:
 - Parse $dk = (R, k)$, $c = (vk, \sigma, \overline{vk})$ and $d = (b, u)$, where $vk = y$, $b \in \{0, 1\}$ and $u \in \{0, 1\}^n$.
 - Run $\text{O} \leftarrow \text{OSS.Setup}(1^\lambda; R)$.
 - Run $\overline{\text{O}} \leftarrow \overline{\text{OSS.Setup}}(1^\lambda; F_k(vk))$ and parse $\overline{\text{O}} = (P, P^{-1}, D)$.
 - Check that $\text{OSS.Ver}^O(vk, 1, \sigma) = \top$. Otherwise output $\perp$.
 - If $u \in S_{y,b}$, output b , and otherwise output $\perp$. This can be checked given oracle access to $\overline{\text{O}}$, by checking that $P^{-1}(y, u) \neq \perp$.
 - $\text{DecX}(dk, c, d)$:
 - Parse $dk = (R, k)$, $c = (vk, \sigma, \overline{vk})$ and $d = (b', u)$, where $\overline{vk} = y$, $b' \in \{0, 1\}$ and $u \in \{0, 1\}^n$.
 - Run $\text{O} \leftarrow \text{OSS.Setup}(1^\lambda; R)$.
 - Run $\overline{\text{O}} \leftarrow \overline{\text{OSS.Setup}}(1^\lambda; F_k(vk))$ and parse $\overline{\text{O}} = (P, P^{-1}, D)$.
 - Check that $\text{OSS.Ver}^O(vk, 1, \sigma) = \top$, and if not output $\perp$.
 - Compute the bit r as follows and abort if $r = \perp$

$$r := \begin{cases} 0 & \text{if } u \in S_y^\perp \\ 1 & \text{if } u \oplus (1, 0, \dots, 0) \in S_y^\perp \\ \perp & \text{otherwise.} \end{cases}$$

This sets $r = 0$ if $u \in S_y^\perp$ and $r = 1$ if $u \in (S_{y,0})^\perp \setminus S_y^\perp$. Output $b' \oplus r$. This can be computed using $\overline{\text{O}}$.

¹⁵Note that $\overline{\text{OSS.Gen}}$ does not need the entire O oracle in the OSS construction of SZ25, the oracles P and P^{-1} suffice.

4.3 Correctness

Theorem 4. *The Pauli Functional commitment scheme described in Section 4.2 satisfies correctness as in Definition 6.*

The proof of this statement is almost identical to that of Theorem 4.6 of [BKNY23], except that we are using one-shot signatures instead of signature tokens. We defer the proof to Appendix A.

4.4 Binding

Theorem 5. *The construction in Section 4.2 satisfies single-bit binding with public decodability as in Definition 7.*

Proof. It will be convenient for us to work with a different but sufficient (in fact, equivalent) definition of binding. By Lemma 8, it suffices to show that the scheme satisfies the following definition of binding.

Definition 9 (Collapse-binding). *For any adversary $\mathcal{A} := \{(\mathcal{A}_1, \mathcal{A}_2)_\lambda\}_{\lambda \in \mathbb{N}}$, where each of $\mathcal{A}_1$ and $\mathcal{A}_2$ are oracle-aided quantum operations that make at most $\text{poly}(\lambda)$ oracle queries, define the collapse-binding experiment $\text{CollapseBindingExpt}^{\mathcal{A}}(1^\lambda)$ as follows.*

- *Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.*
- *Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.*
- *Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.*
- *Run $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' .*
- *The experiment outputs 1 if $b = b'$.*

We say that adversary $\mathcal{A}$ is valid if the state on $(\mathcal{B}, \mathcal{U})$ output by $\mathcal{A}_1$ is in the image of $|0\rangle\langle 0| \otimes \Pi_{\text{dk}, c, 0} + |1\rangle\langle 1| \otimes \Pi_{\text{dk}, c, 1}$. Then, we say that a Pauli functional commitment $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies publicly-decodable collapse-binding if it holds that for all valid adversaries $\mathcal{A}$,

$$\left| \Pr \left[\text{CollapseBindingExpt}^{\mathcal{A}}(1^\lambda) = 1 \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

We will now show, through a sequence of hybrids, that our construction satisfies the collapse-binding with public decodability property. For any valid q -query¹⁶ adversary $\mathcal{A}$, let $\text{Hyb}_i^{\mathcal{A}}$ be the advantage achieved by $\mathcal{A}$ in Hybrid i .

Hybrid 0. This hybrid is the collapse-binding game defined above.

Hybrid 1. We now move to a hybrid where we replace the PRF F_k in the scheme in Section 4.2 with a random oracle $H : \mathcal{X} \rightarrow \mathcal{R}$, so that we are effectively choosing a random instance of the one-shot signature oracles $\bar{\mathcal{O}}$ independently for every choice of verification key vk .

Hybrid 0 and Hybrid 1 are indistinguishable because of the post-quantum security of the PRF, so for every valid $\mathcal{A}$, $|\text{Hyb}_0^{\mathcal{A}} - \text{Hyb}_1^{\mathcal{A}}| \leq \text{negl}(\lambda)$.

¹⁶The bound q on the number of queries made by $\mathcal{A}$ will be important in Hybrid 3, when we apply the small-range distribution argument of [Zha21].

Hybrid 2. Observe that once $\mathcal{A}_1$ has committed to $c^* = (\text{vk}^*, \sigma_1^*, \overline{\text{vk}})$, where σ_1^* is a valid signature of 1 under vk^* , we can replace the CK_D oracle given to the second-stage adversary $\mathcal{A}_2$ with $\text{CK}_D\{\text{vk}^*\}$, which works as follows.

$\text{CK}_D\{\text{vk}^*\}(\text{vk}, \sigma, \overline{\text{vk}}, u)$:

- If $\text{vk} = \text{vk}^*$, output $\perp$.
- Otherwise, output $\text{CK}_D(\text{vk}, \sigma, \overline{\text{vk}}, u)$.

Formally, the experiment in this hybrid is as follows:

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment $c^* = (\text{vk}^*, \sigma_1^*, \overline{\text{vk}})$ and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^{\text{CK}\{\text{vk}^*\}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' , where $\text{CK}\{\text{vk}^*\} = (\text{O}, \text{CK}_P, \text{CK}_{P-1}, \text{CK}_D\{\text{vk}^*\})$.
- The experiment outputs 1 if $b = b'$.

Claim 1. For every valid adversary $\mathcal{A}$, $|\text{Hyb}_1^{\mathcal{A}} - \text{Hyb}_2^{\mathcal{A}}| = \text{negl}(\lambda)$.

Proof. By Lemma 1, if an adversary is able to distinguish between these two hybrids, there must be an adversary $\mathcal{A}'$ that produces an input on which CK and $\text{CK}\{\text{vk}^*\}$ differ. The only points on which CK_D and $\text{CK}_D\{\text{vk}^*\}$ differ are of the form $(\text{vk}^*, \sigma_0, \cdot, \cdot)$, where σ_0 is a valid signature of 0 under vk^* . By the strong unforgeability of the one-shot signatures scheme OSS, once the adversary has committed to a valid signature σ_1^* of 1 under vk^* , it cannot produce a valid σ_0 , and therefore cannot notice the difference between the oracles CK_D and $\text{CK}_D\{\text{vk}^*\}$. $\square$

Hybrid 3. This hybrid is the same as Hybrid 2, except that we define the punctured CK_D oracle differently, as follows. $\text{CK}_D\{H(\text{vk}^*)\}(\text{vk}, \sigma, \overline{\text{vk}}, u)$:

- If $H(\text{vk}) = H(\text{vk}^*)$, output $\perp$.
- Otherwise, output $\text{CK}_D(\text{vk}, \sigma, \overline{\text{vk}}, u)$.

Formally, the experiment in this hybrid is as follows:

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment $c^* = (\text{vk}^*, \sigma_1^*, \overline{\text{vk}})$ and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^{\text{CK}\{H(\text{vk}^*)\}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' , where $\text{CK}\{H(\text{vk}^*)\} = (\text{O}, \text{CK}_P, \text{CK}_{P-1}, \text{CK}_D\{H(\text{vk}^*)\})$.
- The experiment outputs 1 if $b = b'$.

Claim 2. For every valid adversary $\mathcal{A}$, $|\text{Hyb}_2^{\mathcal{A}} - \text{Hyb}_3^{\mathcal{A}}| = \text{negl}(\lambda)$.

Proof. By Lemma 1, if an adversary is able to distinguish between these two hybrids, there must be an adversary $\mathcal{A}'$ that produces an input on which $\text{CK}\{\text{vk}^*\}$ and $\text{CK}\{H(\text{vk}^*)\}$ differ. The only points on which CK_D and $\text{CK}_D\{\text{vk}^*\}$ differ are of the form $(\text{vk}, \cdot, \cdot, \cdot)$, where $H(\text{vk}) = H(\text{vk}^*)$ but $\text{vk} \neq \text{vk}^*$. By the collision-resistance of random oracles (Lemma 3), no query-efficient adversary should be able to find such a collision with greater than negligible probability. $\square$

Hybrid 4. This hybrid is the same as Hybrid 3, except that use a small-range distribution argument to move to a hybrid where we **replace the random oracle H introduced in Hybrid 1 with a random function H_{SR} of bounded range**. This will allow us to rephrase the collapse-binding game in terms of the oracles of the inner one-shot signature scheme $\overline{\text{OSS}}$, which we then later analyze.

Let $R = 300q^3/\varepsilon$, where $\varepsilon(\lambda) = \text{Hyb}_3^A$. We sample R independent and uniformly random values $r_i \leftarrow \mathcal{R}$, for $i \in [R]$, and for each $x \in \mathcal{X}$, we sample a random $i_x \leftarrow [R]$ and set $H_{\text{SR}}(x) := r_{i_x}$.

Claim 3. For every valid q -query adversary $\mathcal{A}$, $\text{Hyb}_4^A \geq \frac{4}{5}\text{Hyb}_3^A$.

Proof. By Lemma 2, we know that the output distributions of $\mathcal{A}$ in Hybrids 3 and 4 are at most $27q^3/R$ -far in statistical distance, and by our choice of R , this means that $|\text{Hyb}_3^A - \text{Hyb}_4^A| \leq \varepsilon/5$, which in turn implies that $\text{Hyb}_4^A \geq 4\varepsilon/5$. $\square$

We now show that no query-efficient adversary can win the game in Hybrid 4 with non-negligible advantage. Towards that end, consider the following game, which is similar in spirit to the collapse-binding game in Definition 9, but rephrased in terms of the underlying oracles of $\overline{\text{OSS}}$ in the [SZ25] construction.

Definition 10 (OSS delayed collapsing experiment). Let $\overline{\text{OSS}}$ be the one-shot signature scheme in Section 3.3. For any adversary $\mathcal{A} = \{(\mathcal{A}_1, \mathcal{A}_2)_\lambda\}_{\lambda \in \mathbb{N}}$, where each of $\mathcal{A}_1$ and $\mathcal{A}_2$ are quantum operations that make at most $\text{poly}(\lambda)$ many oracle queries, define the experiment $\text{OSSDelayedCollapsingExpt}^A(1^\lambda)$ as follows.

- Sample $P, P^{-1}, D \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda)$. Let $\{S_y\}_{y \in \{0,1\}^r}$ be the cosets defined by P, P^{-1} .
- Run $\mathcal{A}_1^{P, P^{-1}, D}(1^\lambda)$ until it outputs a string $y \in \{0,1\}^r$ and a state on registers $(\mathcal{U}, \mathcal{R})$. Here $\mathcal{U}$ is k -qubit register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Sample $b \leftarrow \{0,1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $\mathcal{U}$ in the computational basis.
- Run $\mathcal{A}_2^{P, P^{-1}}(\mathcal{U}, \mathcal{R})$ until it outputs a bit b' . The experiment outputs 1 if $b = b'$ and 0 otherwise.

The adversary is valid if the state on $\mathcal{U}$ output by $\mathcal{A}_1$ is in the image of $\Pi_y = \sum_{u \in S_y} |u\rangle\langle u|$. We say that the adversary $\mathcal{A}$ wins the experiment with advantage

$$\left| \Pr[\text{OSSDelayedCollapsingExpt}^A(1^\lambda) = 1] - \frac{1}{2} \right|.$$

Given an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ that wins the game in Hybrid 4 with non-negligible advantage, we give an adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ that wins the OSS delayed collapsing game with non-negligible advantage. The first-stage adversary $\mathcal{B}_1$ works as follows:

- $\mathcal{B}_1$ receives oracles $P^*, (P^{-1})^*, D^*$ from the challenger.
- Let $\text{CK}_4 = (\text{O}_4, \text{CK}_{P,4}, \text{CK}_{P^{-1},4}, \text{CK}_{D,4}), \text{DecZ}_4, \text{DecX}_4$ be oracles as defined in Hybrid 4 (using H_{SR} in place of the PRF), and let $\mathcal{R}_{\text{SR}} \subseteq \mathcal{R}$ be the range of H_{SR} , such that $|\mathcal{R}_{\text{SR}}| = R$.
- Sample a random $r^* \leftarrow \mathcal{R}_{\text{SR}}$.
- Define oracles $\text{CK}_P^*, \text{CK}_{P^{-1}}^*, \text{CK}_D^*$ and $\text{DecZ}^*, \text{DecX}^*$ to be identical to $\text{CK}_{P,4}, \text{CK}_{P^{-1},4}, \text{CK}_{D,4}$ and $\text{DecZ}_4, \text{DecX}_4$, except that on inputs vk such that $H_{\text{SR}}(\text{vk}) = r^*$, we use oracles $P^*, (P^{-1})^*, D^*$ instead of the oracles you get by running $\text{OSS.Setup}(1^\lambda; H_{\text{SR}}(\text{vk}))$. For example, $\text{CK}_P^*(\text{vk}, x)$ is defined as follows:
 - If $H_{\text{SR}}(\text{vk}) = r^*$, output $P^*(x)$.
 - Otherwise, output $\text{CK}_P(\text{vk}, x)$.

The oracles $\text{CK}_{P^{-1}}^*, \text{CK}_D^*$ and $\text{DecZ}^*, \text{DecX}^*$ are similarly defined.

- Run $\mathcal{A}_1$ with $\text{CK}^* = (\text{O}_4, \text{CK}_P^*, \text{CK}_{P^{-1}}^*, \text{CK}_D^*)$.

- Receive $c = (\tilde{vk}, \sigma, \overline{vk})$ and registers $\mathcal{B}, \mathcal{U}$ from $\mathcal{A}_1$.
- If $H_{SR}(\tilde{vk}) = r^*$, send $y = \overline{vk}$ and $\mathcal{U}$ to the challenger.¹⁷
- Otherwise output a random y , where $(y, \mathcal{U}) \leftarrow \text{OSS.Gen}^{P^*, (P^{-1})^*, D^*}(1^\lambda)$.

The second stage $\mathcal{B}_2$ adversary works as follows.

- $\mathcal{B}_2$ receives the oracles $P^*, (P^{-1})^*$ from the challenger.
- If $H_{SR}(\tilde{vk}) \neq r^*$ then output $b' \leftarrow \{0, 1\}$ uniformly at random.
- If $H_{SR}(\tilde{vk}) = r^*$, run $\mathcal{A}_2$ with oracles $\text{CK}_D^*\{H(\tilde{vk})\}$ and DecZ^* , where $\text{CK}_D^*\{H(\tilde{vk})\}(\tilde{vk}, \sigma, \overline{vk}, u)$ is defined just like $\text{CK}_D\{H(\tilde{vk})\}$ from Hybrid 4
 - If $H(\tilde{vk}) = H(\tilde{vk})$, then output $\perp$ and abort.
 - Otherwise, output $\text{CK}_D^*(\tilde{vk}, \sigma, \overline{vk}, u)$.
- Output the output of $\mathcal{A}_2$.

Observe that the oracles $\text{CK}^*, \text{DecZ}^*, \text{DecX}^*$ and $\text{CK}^*\{\cdot\}$ given to $\mathcal{A}_1, \mathcal{A}_2$ will have the same distribution as in Hybrid 4. Since $|\mathcal{R}_{SR}| \leq R$, $\mathcal{B}_1$ correctly guesses $r^* = H_{SR}(\tilde{vk})$ with probability at least $1/R$. So $\mathcal{B}$ wins the OSS delayed collapsing game in Definition 10 with advantage at least

$$\frac{1}{R} \cdot \text{Hyb}_4^A \geq \Omega\left(\frac{\varepsilon^2}{q^3}\right),$$

where $\varepsilon = \text{Hyb}_3^A$, and the inequality follows from Claim 3. Finally, Theorem 6 shows that no query-efficient adversary can win the OSS delayed collapsing game with non-negligible advantage, which completes the proof. $\square$

4.4.1 Hardness of the OSS delayed collapsing game

Theorem 6. *Let $\lambda \in \mathbb{N}$ denote the security parameter. Let q, s, n, r be polynomials in λ , and let $s \leq n - r$ such that $\frac{k^9 \cdot q^7 \cdot \frac{1}{\varepsilon^4}}{\sqrt{2^{n-r-s}}} \leq o(1)$. Then, for every q query adversary $\mathcal{A}$,*

$$\left| \Pr[\text{OSSDelayedCollapsingExpt}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

Proof. Suppose for contradiction that there exists an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ with non negligible advantage in the OSS delayed collapsing game. Then,

$$\left| \Pr[\text{OSSDelayedCollapsingExpt}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| \geq \varepsilon(\lambda).$$

for some non-negligible function ε . Consider the following sequence of hybrids.

H₁: This corresponds to $\text{OSSDelayedCollapsingExpt}^{\mathcal{A}}(1^\lambda)$

¹⁷For valid commitments the state on $(\mathcal{B}, \mathcal{U})$ is supported on basis states $|b, u\rangle$ where $b = u[1]$, so measuring $\mathcal{U}$ (or not) is the same as measuring both $\mathcal{B}, \mathcal{U}$.

H₂: In this hybrid, we use the standard small-range distribution technique to simulate the oracles P, P^{-1}, D using a polynomially bounded number of underlying cosets $(\mathbf{A}_y, \mathbf{b}_y)$ sampled using a function $F' : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$ as follows:

- Let $R := (300 \cdot q^3) \cdot \frac{2^7 \cdot k^2}{\varepsilon}$
- for every $y \in \mathbb{Z}_2^r$, sample $i_y \leftarrow [R]$.
- Sample a random function $F' : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$. For every $i \in [R]$, interpret $F(i) = (\mathbf{A}_i \in \mathbb{Z}_2^{k \times (n-r)}, \mathbf{b}_i \in \mathbb{Z}_2^k)$.
- For $y \in \mathbb{Z}_2^r$, define $F'(y) := (\mathbf{A}_{i_y}, \mathbf{b}_{i_y})$.

Claim 4. For any q query quantum algorithm $\mathcal{A}$,

$$|\Pr[\mathbf{H}_2^{\mathcal{A}} = 1] - \Pr[\mathbf{H}_1^{\mathcal{A}} = 1]| \leq \frac{\varepsilon}{8}$$

Proof. From Theorem A.6 in [AGQY22], it follows that for every quantum algorithm making at most q queries, the distinguishing advantage between F and F' is bounded by $\frac{300 \cdot q^3}{R} < \frac{\varepsilon}{8}$. $\square$

H₃: Bloating the dual Same as **H₂**, except the oracle D is replaced with D' , which on input y , accepts an evasive superspace of the dual subspace corresponding to the coset of y ,

$$D'(y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^\top \mathbf{A}_y^{(1)} = \mathbf{0}^{n-r-s} \\ 0 & \text{otherwise.} \end{cases}$$

, where $\mathbf{A}_y^{(1)} \in \mathbb{Z}_2^{n-r-s}$ denotes the last $n-r-s$ columns of $\mathbf{A}_y \in \mathbb{Z}_2^{k \times (n-r)}$.

Claim 5. For any q query quantum algorithm $\mathcal{A}$,

$$|\Pr[\mathbf{H}_2^{\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{H}_3^{\mathcal{A}}(1^\lambda) = 1]| \leq \frac{\varepsilon}{8}.$$

Proof. By Lemma 19 in [SZ25], due to $k - (k - (n-r)) - s = n-r-s$ for every $i \in [R]$, changing D to check for membership in $\mathbf{A}_y^{(1)\perp}$ instead of $\mathbf{A}_y^\perp$ is $O\left(\frac{q \cdot s}{\sqrt{2^{n-r-s}}}\right)$ indistinguishable, for any q query algorithm. Since we use the above indistinguishability R times, it follows that the distinguishing advantage in the current hybrid is $:= \Pr[\mathbf{H}_2^{\mathcal{A}}(1^\lambda) = 1] - R \cdot O\left(\frac{q \cdot s}{\sqrt{2^{n-r-s}}}\right) \geq \frac{7\varepsilon}{8} - O\left(\frac{k^2 \cdot q^4 \cdot s \cdot \frac{1}{\varepsilon}}{\sqrt{2^{n-r-s}}}\right) \geq \frac{3\varepsilon}{4}$, when $\frac{k^9 \cdot q^7 \cdot \frac{1}{\varepsilon^4}}{\sqrt{2^{n-r-s}}} \leq o(1)$. This implies the distinguishing advantage $\leq \frac{\varepsilon}{8}$. $\square$

H₄: In this hybrid, we simulate the oracles P, P^{-1}, D' from **H₃** using a coset partition function (Definition 4) that operates on a smaller input domain. Formally, given oracle access to any $(r+s, r, s)$ coset partition function $Q : \{0, 1\}^{r+s} \rightarrow \{0, 1\}^r$, the oracles P, P^{-1}, D' are simulated as follows:

1. Sample a random permutation $\Gamma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ and for every y , choose a random full rank matrix $\mathbf{C}_y \in \mathbb{Z}_2^{k \times n}$, and a random vector $\mathbf{d}_y \in \mathbb{Z}_2^k$.
2. Simulating $P(x \in \{0, 1\}^n)$.
 - $(x_0 \in \{0, 1\}^{r+s}, x_1 \in \{0, 1\}^{n-r-s}) \leftarrow \Gamma(x)$.
 - $y \leftarrow Q(x_0)$.
 - $u \leftarrow (\mathbf{C}_y \cdot \Gamma(x) + \mathbf{d}_y)$.
 - Output (y, u) .

3. Simulating $P^{-1}(y \in \{0, 1\}^r, u \in \{0, 1\}^k)$.
 - $z \leftarrow \mathbf{C}_y^{-1} \cdot (u - \mathbf{d}_y)$.
 - $(z_0 \in \{0, 1\}^{r+s}, z_1 \in \{0, 1\}^{n-r-s}) := z$.
 - If $Q(z_0) = y$, output $\Gamma^{-1}(z)$. Otherwise, output $\perp$.
4. Simulating $D'(\mathbf{v}, y)$
 - $\mathbf{A}_y^{(1)\perp} :=$ last $n - r - s$ columns of $\mathbf{C}_y$.
 - Output 1 iff $\mathbf{v}^T \cdot \mathbf{A}_y^{(1)} = 0^{n-r-s}$.

Claim 6.

$$\Pr[\mathbf{H}_4^A(1^\lambda) = 1] = \Pr[\mathbf{H}_3^A(1^\lambda) = 1]$$

Proof. We will argue that $\Gamma, \{\mathbf{C}_y, \mathbf{d}_y\}_y$ and Q perfectly simulate P, P^{-1}, D' . Observe that simulating P, P^{-1}, D' using Q is implicitly setting the following parameters:

- $\mathbf{A}_y$: The preimage set $Q^{-1}(y)$ is a coset, which can be described as the set $\{\mathbf{B}_y \cdot r + r_y\}$ as r ranges over $\mathbb{Z}_2^s$. Thus $\mathbf{A}_y := \mathbf{C}_y \cdot K_y$, where $K_y = \begin{pmatrix} \mathbf{B}_y \in \mathbb{Z}^{(r+s) \times s} & & \\ & \ddots & \\ & & \mathbf{I}_{n-r-s} \end{pmatrix}$. Since K_y is full rank and $\mathbf{C}_y$ is random, this implies that $\mathbf{A}_y$ is random.
- $\mathbf{b}_y := \mathbf{d}_y + \mathbf{C}_y \cdot (r_y || 0^{n-r-s})$. This follows by construction, now since d_y is random, this implies that b_y is also random.
- Π : Define an augmented function $Q' : \{0, 1\}^n \rightarrow \{0, 1\}^n$. On input $\mathbf{z}$, the first r bits of $Q'(\mathbf{z})$ are set to $y = Q(\mathbf{z})$. Define the function $\bar{J}(z)$ that outputs the unique vector in $\mathbb{Z}_2^{n-r}$ such that $\mathbf{z} = K_y \cdot \bar{J}(z) + r_y$. Then define $Q'(\mathbf{z}) = (Q(\mathbf{z}), \bar{J}(\mathbf{z}))$. This implies that $\Pi = Q' \circ \Gamma$.

□

H₅: This hybrid describes the following experiment

- Simulate P, P^{-1}, D' given oracle access to $Q : \{0, 1\}^{r+s} \rightarrow \{0, 1\}^r$ as in **H₃**.
- Run $\mathcal{A}_1^Q(1^\lambda)$ until it outputs a string $y \in \{0, 1\}^r$ and a state on registers $(\mathcal{U}, \mathcal{R})$. Here $\mathcal{U}$ is k -qubit register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Initialize a register T in the state $|0\rangle^n$ and apply the map $(u, 0) \rightarrow (u, \mathbf{C}_y^{-1}(u - \mathbf{d}_y))$ on the joint system $\mathcal{U} \otimes T$.
- Measure the first $r + s$ bits of register T in the computational basis.
- Apply the map $(x, 0) \rightarrow (x, \mathbf{C}_y \cdot x + \mathbf{d}_y)$ on the resulting state in register $T \otimes \mathcal{U}$.
- Uncompute the register T .
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^Q(\mathcal{U}, \mathcal{R})$ until it outputs a bit b' . The experiment outputs 1 if $b = b'$ and 0 otherwise.

Claim 7. For all QPT algorithms $\mathcal{A}$,

$$|\Pr[\mathbf{H}_5^A(1^\lambda) = 1] - \Pr[\mathbf{H}_4^A(1^\lambda) = 1]| \leq \text{negl}(\lambda).$$

Proof. Observe that $\mathcal{P}(x) = \left(y = Q(\overline{\Gamma(x)}), \mathbf{u} = \mathbf{C}_y \left[\begin{smallmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{smallmatrix} \right] + \mathbf{d}_y \right)$ where $\overline{\Gamma(x)}$ picks out the first $r + s$ bits and $\Gamma(x)$ picks out the last $(n - r - s)$ bits. Now, for a fixed y , the state on register $\mathcal{U}$ has the following form:

$$\sum_{\overline{\Gamma(x)}: Q(\overline{\Gamma(x)})=y, \Gamma(x)} \mathbf{C}_y \left[\begin{smallmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{smallmatrix} \right] + \mathbf{d}_y$$

Measuring the first $r + s$ bits of register T in the computational basis and then applying the map $(x, 0) \rightarrow (x, C_y(x) + d_y)$ on the resulting state in register $T \otimes \mathcal{U}$ yields the following state:

$$\sum_{\overline{\Gamma(x)}} \mathbf{C}_y \left[\begin{smallmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{smallmatrix} \right] + \mathbf{d}_y$$

, where $\overline{\Gamma(x)}$ is such that $Q(\overline{\Gamma(x)}) = y$.

Now, [SZ25] construct collision resistant coset partition functions by composing 2 to 1 functions together. More concretely, let $H : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ be a random 2 to 1 function. Given this, the coset partition function $H^\ell : \{0, 1\}^{n\ell} \rightarrow \{0, 1\}^{(n-1)\ell}$ is the following:

$$H^\ell(x_1, \dots, x_\ell) := H(x_1), \dots, H(x_\ell)$$

In particular, constructing Q in this manner, we can conclude that, since it is a composition of 2 to 1 functions which are known to be collapsing from [Zha22] and composition preserves the collapsing property, for any computationally bounded adversary $\mathcal{A}$, the above state is indistinguishable from

$$\sum_{\overline{\Gamma(x)}: Q(\overline{\Gamma(x)})=y, \Gamma(x)} \mathbf{C}_y \left[\begin{smallmatrix} \overline{\Gamma(x)} \\ \Gamma(x) \end{smallmatrix} \right] + \mathbf{d}_y$$

and the claim then follows immediately. $\square$

H₆: In this hybrid, we “unsimulate” the oracles, and revert to defining the oracles P, P^{-1}, D' as in **H₃**. This hybrid describes the following experiment:

- Sample $P, P^{-1}, D' \leftarrow \overline{\text{OSS}}.\text{Setup}(1^\lambda)$.
- Run $\mathcal{A}_1^{P, P^{-1}, D'}(1^\lambda)$ until it outputs a string $y \in \{0, 1\}^r$ and a state on registers $(\mathcal{U}, \mathcal{R})$. Here $\mathcal{U}$ is k -qubit register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Initialize a register T in the state $|0\rangle^n$ and apply the map $(u, 0) \rightarrow (u, (\mathbf{A}_y^{-1}(u - \mathbf{b}_y)))$ on the joint system $\mathcal{U} \otimes T$.
- Uncompute $\mathcal{U}$ and measure the first s bits of register T in the computational basis, and then apply the map $(x, 0) \rightarrow (x, \mathbf{A}_y \cdot x + \mathbf{b}_y)$ on the resulting state in register $T \otimes \mathcal{U}$.
- Uncompute the register T .
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^{P, P^{-1}}(\mathcal{U}, \mathcal{R})$ until it outputs a bit b' . The experiment outputs 1 if $b = b'$ and 0 otherwise.

Claim 8.

$$\Pr[\mathbf{H}_6^{\mathcal{A}}(1^\lambda) = 1] = \Pr[\mathbf{H}_5^{\mathcal{A}}(1^\lambda) = 1]$$

Proof. This hybrid distributes identically to the previous hybrid by the simulation of P, P^{-1}, D' by Q in **H₄**. $\square$

H₇: Same as **H₆** except, $\forall y \in [R]$, $\mathbf{A}_y$ is sampled as follows: sample a random function $F : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$ and a random permutation $M : \{0, 1\}^{n-r} \rightarrow \{0, 1\}^{n-r}$. Interpret $F(y) = (\mathbf{A}'_y, \mathbf{b}_y)$ where $\mathbf{A}'_y \in \mathbb{Z}_2^{k \times (n-r)}$ is a full-rank matrix and $\mathbf{b} \in \mathbb{Z}_2^n$, and interpret $M(y) = M_y$, where $M_y \in \mathbb{Z}_2^{(n-r) \times (n-r)}$ is a full-rank matrix. Set $\mathbf{A}_y = \mathbf{A}'_y \circ M_y$.

Claim 9.

$$\Pr[\mathbf{H}_7^A(1^\lambda) = 1] = \Pr[\mathbf{H}_6^A(1^\lambda) = 1]$$

Proof. The distribution of $\mathbf{A}_y$ is identical in both hybrids, **H₇** and **H₈**. $\square$

H₈: Same as **H₇** except we change how $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is sampled in $\overline{\text{OSS}}$:

1. Sample a random permutation $\Pi' : \{0, 1\}^n \rightarrow \{0, 1\}^n$.
 - Let $\sigma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be the following permutation: $\sigma(x)$: Parse input as $(y \in \{0, 1\}^r, z \in \{0, 1\}^{n-r})$, and output $(y, M_y^{-1}(z))$.
2. $\Pi(x) = \Pi' \circ \sigma(x)$.

Claim 10.

$$\Pr[\mathbf{H}_8^A(1^\lambda) = 1] = \Pr[\mathbf{H}_7^A(1^\lambda) = 1]$$

Proof. The 2 hybrids are distributed identically. $\square$

Claim 11. *If there exists an adversary $(\mathcal{A}_1, \mathcal{A}_2)$ that wins the experiment in **H₈** with non negligible advantage, then there exists an adversary which wins the coset-collapsing game with non negligible advantage.*

Proof. First, observe that the oracles P, P^{-1}, D' in **H₈** are the following:

- Sample a random permutation $\Pi' : \{0, 1\}^n \rightarrow \{0, 1\}^n$.
 - Let $\sigma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be the following permutation: $\sigma(x)$: Parse input as $(y \in \{0, 1\}^r, z \in \{0, 1\}^{n-r})$, and output $(y, M_y^{-1}(z))$.
- $\Pi(x) = \Pi' \circ \sigma(x)$. Let $H(x)$ be the first r output bits of $\Pi(x)$, and let $J'(x)$ denote the remaining $n-r$ bits.
- Sample a random function $F : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$ and a random permutation $M : \{0, 1\}^{n-r} \rightarrow \{0, 1\}^{n-r}$. For every $y \in [R]$, Interpret $F(y) = (\mathbf{A}'_y, \mathbf{b}_y)$ where $\mathbf{A}'_y \in \mathbb{Z}_2^{k \times (n-r)}$ is a full-rank matrix and $\mathbf{b} \in \mathbb{Z}_2^n$, and interpret $M(y) = M_y$, where $M_y \in \mathbb{Z}_2^{(n-r) \times (n-r)}$ is a full-rank matrix. Set $\mathbf{A}_y = \mathbf{A}'_y \circ M_y$. Let $S_y = \{\mathbf{A}'_y \mathbf{x} + \mathbf{b}_y \mid \mathbf{x} \in \mathbb{Z}_2^{n-r}\}$ be the coset defined by $(\mathbf{A}'_y, \mathbf{b}_y)$.
- Now, the functions P, P^{-1}, D' are defined as follows:

$$\begin{aligned}
P(x) &= (y, \mathbf{A}'_y \cdot J(x) + \mathbf{b}_y) \in \{0, 1\}^n \rightarrow \{0, 1\}^r \times \mathbb{Z}_2^k \text{ where } y = H(x) \\
P^{-1}(y, \mathbf{u}) &= \begin{cases} \Pi'(y, \mathbf{z}) & \text{if } \exists \mathbf{z} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}'_y \mathbf{z} + \mathbf{b}_y = \mathbf{u} \\ \perp & \text{otherwise.} \end{cases} \\
D'(y, \mathbf{v}) &= \begin{cases} 1 & \text{if } \mathbf{v}^\top \mathbf{A}'_y = \mathbf{0}^{n-r-s} \\ 0 & \text{otherwise.} \end{cases}
\end{aligned}$$

The experiment in **H₈** is the following:

- Run $\mathcal{A}_1^{P, P^{-1}, D'}(1^\lambda)$ until it outputs a string $y \in \{0, 1\}^r$ and a state on registers $(\mathcal{U}, \mathcal{R})$. Here $\mathcal{U}$ is k -qubit register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.

- Initialize a register T in the state $|0\rangle^n$ and apply the map $(u, 0) \rightarrow (u, (\mathbf{A}_y^{-1}(u - \mathbf{b}_y))$ on the joint system $\mathcal{U} \otimes T$.
- Measure the first s bits of register T in the computational basis, and then apply the map $(x, 0) \rightarrow (x, \mathbf{A}_y \cdot x + \mathbf{b}_y)$ on the resulting state in register $T \otimes \mathcal{U}$.
- Uncompute the register T .
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^{P, P^{-1}}(\mathcal{U}, \mathcal{R})$ until it outputs a bit b' . The experiment outputs 1 if $b = b'$ and 0 otherwise.

Now, consider the following experiment.

Definition 11 (Coset-collapsing game with Dual Access). *This is a game between a challenger and a two-stage adversary $(\mathcal{A}_1, \mathcal{A}_2)$. There is a public subspace $S \subset \mathbb{F}_2^n$ that is known to all parties.*

1. The challenger samples a random subspace $T \subset S$, and sends to $\mathcal{A}_1$ the oracle $O_{T^\perp}$. Note that $T^\perp$ is a random superspace of $S^\perp$.
2. The adversary $\mathcal{A}_1^{O_{T^\perp}}$ outputs a quantum state on two registers $\mathcal{R}, \mathcal{U}$, and sends $\mathcal{U}$ to the challenger and $\mathcal{R}$ to the adversary $\mathcal{A}_2$.
3. The challenger does the following:
 - (a) Challenger checks in superposition $O_S(\mathcal{U}) = \text{Accept}$; if the check fails, the adversary automatically loses the game.
 - (b) Perform a measurement that projects $\mathcal{U}$ to a coset of T in S . Concretely, let $\text{co}(T)$ be the set of coset representatives of T . Then, the measurement is described by the projectors $\{\Pi_u\}_{\text{co}(T)}$, where $\Pi_u := \sum_{t \in T} |t + u\rangle\langle t + u|$ for $u \in \text{co}(T)$.
 - (c) The challenger samples random $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing and send the register $\mathcal{U}$ to $\mathcal{A}_2$. If $b = 1$, measure the $\mathcal{U}$ register in the computational basis and then send it to $\mathcal{A}_2$.
4. The second stage adversary, $\mathcal{A}_2$, does not receive access to $O_{T^\perp}$, and outputs a bit b' and wins if $b' = b$.

Now by construction, $\mathbf{H}_8$ is exactly the coset-collapsing game with dual access when $T = \text{Colspan}(\mathbf{A}_y^{(1)})$, $S = \text{Colspan}(\mathbf{A}'_y)$. Assume for contradiction that there exists an adversary $(\mathcal{B}_1, \mathcal{B}_2)$ which has distinguishing advantage ε in $\mathbf{H}_8$, where ε is a non negligible function. Now consider an adversary $(\mathcal{A}_1, \mathcal{A}_2)$ for the coset collapsing game with dual access, with knowledge of the public subspace $S = \text{ColSpan}(\mathbf{A}'_y)$ for $y \in [R]$ and access to a membership oracle for $T^\perp = \text{Colspan}(\mathbf{A}_y^{(1)\perp})$. With probability $1/R$, $\mathcal{A}$ guesses correctly the value of y that $\mathcal{B}$ commits to in the game. Conditioned on guessing correctly, $(\mathcal{A}_1, \mathcal{A}_2)$ can run $(\mathcal{B}_1, \mathcal{B}_2)$, so the distinguishing advantage of $(\mathcal{A}_1, \mathcal{A}_2)$ is $\frac{1}{R} \cdot \varepsilon$, which is still non negligible. We will establish next that the coset collapsing game with dual access is indistinguishable from the coset-collapsing game (Definition 12), and prove that the coset-collapsing game is information theoretically hard (Theorem 7). Assuming this, we can complete the proof by invoking Theorem 7. $\square$

$\square$

4.4.2 Coset-collapsing game

We will first consider the following hybrids.

$\mathbf{H}_0$: This is the Coset collapsing game with dual access (Definition 11).

$\mathbf{H}_1$: Same as $\mathbf{H}_0$, but $\mathcal{A}_1$ is given access to $O_{S^\perp}$ instead of $O_{T^\perp}$.

Claim 12. For any q query algorithm $(\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr[1 \leftarrow \mathbf{H}_0^{(\mathcal{A}_1, \mathcal{A}_2)}] - \Pr[1 \leftarrow \mathbf{H}_1^{(\mathcal{A}_1, \mathcal{A}_2)}] \leq \text{negl}(\lambda)$$

Proof. We unbloated the dual in $\mathbf{H}_1$, and the proof is identical to the proof of Claim 5. $\square$

$\mathbf{H}_2$: Same as $\mathbf{H}_1$, but $(\mathcal{A}_1, \mathcal{A}_2)$ are not given access to $O_{S^\perp}$.

Claim 13.

$$\Pr[1 \leftarrow \mathbf{H}_1^{(\mathcal{A}_1, \mathcal{A}_2)}] = \Pr[1 \leftarrow \mathbf{H}_2^{(\mathcal{A}_1, \mathcal{A}_2)}]$$

Proof. This claim follows because $(\mathcal{A}_1, \mathcal{A}_2)$ know the description of S , since its a public subspace. $\square$

Now, $\mathbf{H}_2$ is the coset collapsing game, which we define more concretely below.

Definition 12 (Coset-collapsing game). *This is a game between a challenger and a two-stage adversary $(\mathcal{A}_1, \mathcal{A}_2)$. There is a public subspace $S \subset \mathbb{F}_2^n$ that is known to all parties. Let $n \geq k$.*

1. *The adversary $\mathcal{A}_1$ outputs a quantum state on two registers $\mathcal{R}, \mathcal{U}$, and sends $\mathcal{U}$ to the challenger and $\mathcal{R}$ to the adversary $\mathcal{A}_2$.*
2. *The challenger does the following:*
 - (a) *Sample a random subspace $T \subset S$ of $\dim(T) = k$.*
 - (b) *Challenger checks in superposition $O_S(\mathcal{U}) = \text{Accept}$; if the check fails, the adversary automatically loses the game.*
 - (c) *Perform a measurement that projects $\mathcal{U}$ to a coset of T in S . Concretely, let $\text{co}(T)$ be the set of coset representatives of T . Then, the measurement is described by the projectors $\{\Pi_u\}_{u \in \text{co}(T)}$, where $\Pi_u := \sum_{t \in T} |t + u\rangle\langle t + u|$ for $u \in \text{co}(T)$.*
 - (d) *The challenger samples random $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing and send the register $\mathcal{U}$ to $\mathcal{A}_2$. If $b = 1$, measure the $\mathcal{U}$ register in the computational basis and then send it to $\mathcal{A}_2$.*
3. *Adversary $\mathcal{A}_2$ outputs a bit b' and wins if $b' = b$.*

Theorem 7. *Every (potentially unbounded) quantum adversary wins the coset-collapsing game in Definition 12 with advantage at most $2^k/2^n$.*

Proof. Since the subspace S is public and known to the adversary, we can assume that the subspace-check in Item 2b of Definition 12 always passes, and can perform a change-of-basis so that $S = \mathbb{F}_2^{n'}$ where $n' = \dim(S)$.

After this change of basis, if the challenger samples $b = 1$, the channel it applies is simply Φ_1 as defined in Lemma 5, and if $b = 0$, the challenger applies the Φ_2 channel. Then, the theorem follows from Lemma 5. $\square$

Lemma 5. *The diamond distance between the following two channels Φ_1, Φ_2 is*

$$\|\Phi_1 - \Phi_2\|_\diamond \leq \frac{2^k}{2^n}.$$

- Φ_1 is defined as follows: *Sample a random subspace $T \subset \mathbb{F}_2^n$ of $\dim(T) = k$, and project onto the cosets of T . Concretely, let $\text{co}(T)$ be the set of coset representatives of T , and let $\#T$ be the number of subspaces T such that $\dim(T) = k$.*

$$\Phi_1(\rho) := \frac{1}{\#T} \sum_T \sum_{u \in \text{co}(T)} \Pi_u \rho \Pi_u,$$

where $\Pi_u := \sum_{t \in T} |t + u\rangle\langle t + u|$ for any $u \in \text{co}(T)$.

- Φ_2 is defined as simply measuring in the computational basis

$$\Phi_2(\rho) := \sum_{x \in \mathbb{F}_2^n} \Pi_x \rho \Pi_x,$$

where $\Pi_x := |x\rangle\langle x|$.

Proof. First let us define the following pairs of channels:

- Let Φ'_1 (resp. Φ'_2) be channels defined as follows: append an EPR pair $|\text{EPR}\rangle_{\mathcal{C},\mathcal{D}} \propto \sum_x |x\rangle_{\mathcal{C}} \otimes |x\rangle_{\mathcal{D}}$ on registers $\mathcal{C}, \mathcal{D}$ and teleport the state on input register $\mathcal{A}$ to $\mathcal{D}$. Then, apply Φ_1 (resp. Φ_2) on register $\mathcal{D}$ and output the resulting state. Trace out registers $\mathcal{A}, \mathcal{C}$.
- Let Φ''_1 (resp. Φ''_2) be channels defined as follows: append an EPR pair on registers $\mathcal{C}, \mathcal{D}$. Apply the channel Φ_1 (resp. Φ_2) to register $\mathcal{D}$. Perform the "teleportation step": apply a Bell basis measurement on registers $\mathcal{A}, \mathcal{C}$ and then apply the Z-gate teleportation corrections on register $\mathcal{D}$. Trace out registers $\mathcal{A}, \mathcal{C}$.

Since Φ'_1, Φ'_2 simply teleports the input state and then applies Φ_1, Φ_2 , it is clear that

$$\|\Phi_1 - \Phi_2\|_{\diamond} = \|\Phi'_1 - \Phi'_2\|_{\diamond}.$$

The teleportation step consists of a Bell basis measurement on registers $\mathcal{A}, \mathcal{C}$ followed by a Pauli correction on register $\mathcal{D}$. Claim 14 shows that Φ_1 and Φ_2 absorb the Pauli X corrections and commute with the Pauli Z corrections, which implies that

$$\|\Phi_1 - \Phi_2\|_{\diamond} = \|\Phi''_1 - \Phi''_2\|_{\diamond}.$$

Claim 14. *Let Φ'_1, Φ'_2 and Φ''_1, Φ''_2 be channels as defined above. Then, $\Phi'_1 = \Phi''_1$ and $\Phi'_2 = \Phi''_2$.*

Proof. Observe that Φ_1, Φ_2 are both channels of the form $\Phi(\rho) = \sum_i p_i \Pi_i \rho \Pi_i$ for some $\{(p_i, \Pi_i, S_i)\}_{i \in \mathcal{I}}$ such that

$$\Pi_i = \sum_{x \in S_i} |x\rangle\langle x|,$$

where $\mathcal{I}$ is a finite index set, $\sum_i p_i = 1$ are probability masses, Π_i are projections¹⁸ and $S_i \in \{0, 1\}^n$ are (not necessarily disjoint) sets that cover all of $\{0, 1\}^n$. When convenient, we will alternate between the following two notations: $\Pi_i = \Pi_{S_i}$.

- For Φ_1 , the index set is $\mathcal{I} = \{(T, u)\}$ where T is a k -dimensional subspace of $\mathbb{F}_2^n$ and $u \in \text{co}(T)$ is a representative of one of the cosets of T . The probabilities $p_i = \frac{1}{\#T}$ and $S_{T,u} = T + u$.
- For Φ_2 , the index set is particular simple $\mathcal{I} = \{x \mid x \in \mathbb{F}_2^n\}$ and $S_x = \{x\}$.

The channels Φ_1, Φ_2 have the further property that the probability distribution $\{p_i\}_{i \in \mathcal{I}}$ is uniform, and for every $\bar{x} \in \{0, 1\}^n$, shifting by $\bar{z}$ simply rotates the sets S_i . That is, $\{S_i\}_{i \in \mathcal{I}} = \{S_i + \bar{x}\}_{i \in \mathcal{I}}$.

For any Pauli corrections $\bar{x}, \bar{z}$, and for any input state ρ , we have that

$$\begin{aligned} \Phi(X^{\bar{x}} Z^{\bar{z}} \rho X^{\bar{x}} Z^{\bar{z}}) &= \sum_i p_i \Pi_i X^{\bar{x}} Z^{\bar{z}} \rho X^{\bar{x}} Z^{\bar{z}} \Pi_i = \sum_i p_i \Pi_{S_i + \bar{x}} Z^{\bar{z}} \rho Z^{\bar{z}} \Pi_{S_i + \bar{x}} \\ &= Z^{\bar{z}} \sum_i p_i \Pi_i \rho \Pi_i Z^{\bar{z}} = Z^{\bar{z}} \Phi(\rho) Z^{\bar{z}}. \end{aligned}$$

□

Since these channels are first applying Φ_1 (resp. Φ_2) to half of the EPR pair and then performing some additional measurements as part of the teleportation step (which can only decrease the distance), we only need to worry about how Φ_1, Φ_2 act on halves of EPR pairs. That is, for every bipartite state $\rho_{\mathcal{A}, \mathcal{B}}$,

$$\|(\Phi''_1 \otimes I) \rho_{\mathcal{A}, \mathcal{B}} - (\Phi''_2 \otimes I) \rho_{\mathcal{A}, \mathcal{B}}\|_1 \leq \|(I \otimes \Phi_1) |\text{EPR}\rangle\langle \text{EPR}| - (I \otimes \Phi_2) |\text{EPR}\rangle\langle \text{EPR}|\|_1.$$

¹⁸Note that it is not true in the case of Φ_1 that $\sum_i \Pi_i = I$.

Therefore,

$$\begin{aligned}
\|\Phi_1 - \Phi_2\|_\diamond &= \|\Phi_1'' - \Phi_2''\|_\diamond \\
&= \max_{\rho_{\mathcal{A}, \mathcal{B}}} \|(\Phi_1'' \otimes I)(\rho_{\mathcal{A}, \mathcal{B}}) - (\Phi_2'' \otimes I)(\rho_{\mathcal{A}, \mathcal{B}})\|_1 \\
&\leq \|(I \otimes \Phi_1) |\text{EPR}\rangle\langle\text{EPR}| - (I \otimes \Phi_2) |\text{EPR}\rangle\langle\text{EPR}| \|_1.
\end{aligned}$$

For channels Φ of the form in the proof of Claim 14,

$$\begin{aligned}
I \otimes \Phi(|\text{EPR}\rangle\langle\text{EPR}|) &= \sum_i \sum_{x, x'} p_i (I \otimes \Pi_i) |x, x\rangle\langle x', x'| (I \otimes \Pi_i) \\
&= \sum_i \sum_{x, x' \in S_i} p_i |x, x\rangle\langle x', x'| \\
&= \text{CNOT}(\Phi(|+\rangle\langle+|^{\otimes n}) \otimes |0^n\rangle\langle 0^n|).
\end{aligned}$$

Since the map $\text{CNOT}(\cdot \otimes |0^n\rangle\langle 0^n|)$ preserves the trace distance, we have that

$$\|(I \otimes \Phi_1) |\text{EPR}\rangle\langle\text{EPR}| - (I \otimes \Phi_2) |\text{EPR}\rangle\langle\text{EPR}| \|_1 = \|\Phi_1(|+\rangle\langle+|^{\otimes n}) - \Phi_2(|+\rangle\langle+|^{\otimes n})\|_1,$$

and Lemma 6 completes the proof. $\square$

Lemma 6.

$$\left\| \Phi_1(|+\rangle\langle+|^{\otimes n}) - \Phi_2(|+\rangle\langle+|^{\otimes n}) \right\|_1 \leq \frac{2^k}{2^n}.$$

Proof. For a fixed subspace T of $\dim(T) = k$,

$$\begin{aligned}
\sum_{u \in \text{co}(T)} \Pi_u |+\rangle\langle+|^{\otimes n} \Pi_u &= \sum_{u \in \text{co}(T)} \left(\sum_{t \in T} |t+u\rangle\langle t+u| \right) \left(\frac{1}{2^n} \sum_{x, x' \in \mathbb{F}_2^n} |x\rangle\langle x'| \right) \left(\sum_{t' \in T} |t'+u\rangle\langle t'+u| \right) \\
&= \frac{1}{2^n} \sum_{u \in \text{co}(T)} \sum_{t, t' \in T} \sum_{x, x'} |t+u\rangle\langle t+u| x\rangle\langle x'| t'+u\rangle\langle t'+u| \\
&= \frac{1}{2^n} \sum_{u \in \text{co}(T)} \sum_{t, t' \in T} |t+u\rangle\langle t'+u|.
\end{aligned}$$

Grouping the cosets of T together, this is a block diagonal matrix containing 2^{n-k} blocks, where each block is $\frac{1}{2^n} \mathbf{1} \mathbf{1}^\top 2^k \times 2^k$, where $\mathbf{1} \mathbf{1}^\top$ is the all ones matrix.

Average over all possible subspaces T , we can argue that by symmetry, every diagonal entry must be the same value, say α , and every off-diagonal entry must also be the same value β

$$\Phi_1(|+\rangle\langle+|^{\otimes n}) = \sum_x \alpha |x\rangle\langle x| + \sum_{x \neq x'} \beta |x\rangle\langle x'|.$$

By an averaging argument, we get that $\alpha = \frac{1}{2^n}$ and $\beta = \frac{2^k - 1}{2^{2n} - 2^n}$.

Channel Φ_2 is simply the full standard basis measurement, so

$$\Phi_2(|+\rangle\langle+|^{\otimes n}) = \frac{1}{2^n} \sum_{x \in \mathbb{F}_2^n} |x\rangle\langle x|.$$

Since $\mathbf{1}_d \mathbf{1}_d^\top - I_d$ has eigenvalues $(0, -1, -1, \dots, -1)$, we know that $\|\mathbf{1}_d \mathbf{1}_d^\top - I_d\|_1 = d - 1$, and

$$\left\| \Phi_1(|+\rangle\langle+|^{\otimes n}) - \Phi_2(|+\rangle\langle+|^{\otimes n}) \right\|_1 = \beta \cdot (2^n - 1) = \frac{2^k - 1}{2^n}.$$

$\square$

5 Post-quantum Succinct Ideal Obfuscation

Definition 13 (Post-quantum succinct ideal obfuscation). A (post-quantum) succinct ideal obfuscator in the oracle model consists of the following algorithms.

- $\text{Obf}(1^\lambda, P, T) \rightarrow (O[P], \text{pp})$: The PPT obfuscator takes as input the security parameter 1^λ , the description of a classical Turing machine P with single-bit output, and a time bound $T = \text{poly}(\lambda)$, and outputs the description of a circuit $O[P]$ and public parameters pp .
- $\text{Eval}^{O[P]}(\text{pp}, x) \rightarrow y$: The PPT evaluation algorithm has oracle access to $O[P]$, takes an input x , along with public parameters pp , and produces an output y .

It should satisfy the following properties.

- **Correctness.** For any $1^\lambda, P, T, x$, and $(O[P], \text{pp}) \in \text{Obf}(1^\lambda, P, T)$, it holds that $\text{Eval}^{O[P]}(\text{pp}, x) = P_T(x)$, where $P_T(x)$ is the bit on the output tape of P after T steps.
- **Succinctness.** There exists a fixed polynomial p such that for any program P , time bound T , and $(O[P], \text{pp}) \leftarrow \text{Obf}(1^\lambda, P, T)$, it holds that $|O[P], \text{pp}| = p(\lambda, |P|)$.
- **Ideal obfuscation.** There exists a QPT simulator $\mathcal{S}$ such that for any QPT adversary $\mathcal{A}$, program P , time bound T , and auxiliary information z ,

$$\left\{ \mathcal{A}^{O[P]}(\text{pp}, z) : O[P], \text{pp} \leftarrow \text{Obf}(1^\lambda, P, T) \right\} \approx \left\{ \mathcal{S}^{P_T}(z) \right\}.$$

Classically, the above follows from FHE and SNARKs in the oracle model in a straightforward manner. In this section, we prove that the same construction is also post-quantum secure. Before describing the construction, we will introduce some relevant preliminaries.

5.1 SNARKs

Definition 14. A succinct non-interactive argument of knowledge (SNARK) for NP in the quantum random oracle model (QROM) consists of the following algorithms. Let $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ be a fixed-size random oracle.

- $\text{Prove}^H(1^\lambda, C, x) \rightarrow (z, \pi)$: The p.p.t. prove algorithm takes as input the security parameter 1^λ , the description of a classical deterministic computation C , and an input x , and produces an output z and a proof π .
- $\text{Ver}^H(1^\lambda, C, z, \pi) \rightarrow \{\top, \perp\}$: The deterministic classical verification algorithm takes as input the security parameter 1^λ , the description of a classical deterministic computation C , an output z , and a proof π , and either accepts or rejects.

It should satisfy the following properties.

- **Completeness.** For any λ, C, x, z such that $C(x) = z$,

$$\Pr_H \left[\text{Ver}^H(1^\lambda, C, z, \pi) = \top : (z, \pi) \leftarrow \text{Prove}^H(1^\lambda, C, x) \right] = 1.$$

- **Succinctness.** There exists a fixed polynomial p such that for any time T computation C , the running time of $\text{Ver}^H(1^\lambda, C, z, \pi)$ is $p(\lambda, \log T, |C|, |z|)$. Note that $|x|$ and the time T might be much larger than $p(\lambda, \log T, |C|, |z|)$.
- **Argument of Knowledge.** The argument $(\text{Prove}, \text{Ver})$ is an argument of knowledge with extraction probability κ if there exists a polynomial time quantum extractor $\mathcal{E}$ such that, for every input x and t query quantum oracle algorithm $\tilde{P}$, if, over a random oracle H , for $\pi := \tilde{P}^H$, it holds that $\text{Ver}^H(z, \pi) = 1$ with probability μ , the probability that $\mathcal{E}^{\tilde{P}}(1^t, 1^\lambda)$ outputs a valid witness for π is at least $\kappa(t, \mu, \lambda)$. Here the notation $\mathcal{E}^{\tilde{P}}$ denotes that $\mathcal{E}$ has black box access to $\tilde{P}$.

5.2 Probabilistically Checkable Proofs (PCPs)

Definition 15. A probabilistically checkable proof (PCP) for a relation $\mathcal{R}$ with soundness error ε , proof length ℓ and alphabet Σ is a pair of poly-time algorithms $(\mathbf{P}, \mathbf{V})$ for which the following holds:

- **Completeness:** For every instance-witness pair $(x, w) \in \mathcal{R}$, $\mathbf{P}(x, w)$ outputs a proof string $\Pi : [\ell] \rightarrow \Sigma$ such that $\Pr[\mathbf{V}^\Pi(x) = 1] = 1$.
- **Soundness:** For every instance $x \notin \mathcal{L}(\mathcal{R})$ and proof string $\Pi : [\ell] \rightarrow \Sigma$, $\Pr[\mathbf{V}^\Pi(x) = 1] \leq \varepsilon$.
- **Proof of Knowledge:** The PCP $(\mathbf{P}, \mathbf{V})$ has knowledge error k if there exists a polynomial time extractor $\mathbf{E}$ such that for every witness x and proof string $\Pi : [\ell] \rightarrow \Sigma$, if $\Pr[\mathbf{V}^\Pi(x) = 1] > k$, then $\mathbf{E}(x, \Pi)$ outputs a valid witness w .

5.3 The construction of Micali

Let $(\mathbf{P}, \mathbf{V})$ be the corresponding PCP with prover $\mathbf{P}$ and verifier $\mathbf{V}$. Before describing the construction formally, we will define some notation relevant to Merkle trees. Given a domain Z and $\ell = 2^d$, a *Merkle tree* on a list $\mathbf{v} = (v_i)_{i \in [\ell]} \in Z^\ell$ with respect to the function $H : Z \times Z \rightarrow Z$ is a list of values $(z_{k,i})_{k \in \{0, \dots, d\}, i \in [2^k]} \in Z^{2^\ell}$, where

$$\forall i \in [\ell], z_{d,i} = v_i$$

$$\forall k \in \{0, \dots, d-1\}, \forall i \in [2^k], z_{k+1,i} = H(z_{k+1,2i-1}, z_{k+1,2i})$$

The root of the merkle tree, denoted $\mathbf{rt}$ is $z_{0,1}$. Given $i \in [\ell]$, where $[\ell] = 2^d$, and d is the depth of the tree, the authentication path for the i^{th} leaf is

$$\mathbf{ap} = (a_k)_{k \in [d]},$$

where a_k is the sibling of the k^{th} node on the path from v_i (the i^{th} node) to $\mathbf{rt}$.

To check whether an authentication path is valid, we define

$$\text{CheckPath}^H(\mathbf{rt}, i, v, \mathbf{ap}) = 1 \text{ iff } (\mathbf{x}, \mathbf{y}) \leftarrow \text{Expand}^H(\mathbf{rt}, \mathbf{i}, \mathbf{v}, \mathbf{ap}) \text{ has } \mathbf{y}_0 = \mathbf{rt},$$

where $\text{Expand}^H(\mathbf{rt}, i, v, \mathbf{ap})$ outputs all the input-output pairs, $(\mathbf{x}, \mathbf{y})$, that arise when validating an authentication path.

Extracting Trees: We will recall the algorithm `Extract` that is used to obtain a Merkle tree rooted at a chosen entry in the database corresponding to the compressed oracle for H . This algorithm is used in [CMS19]. In more detail, `Extract` receives a database $D : Z \times Z \rightarrow Z$, a root value $\mathbf{rt} \in Z$, and a height bound d , and outputs a rooted binary tree $T = (V, E)$ of depth at most d where $V \subseteq Z \times \{0, 1\}^{\leq d}$.

`Extract`($D, \mathbf{rt}, d$) :

1. Initialize the vertex set with the root $V := \{(\mathbf{rt}, \emptyset)\}$ and the edge set to be empty $E := \emptyset$.
2. While there exists an unmarked vertex $(u, s) \in V$ such that $u \in \text{Im}(D)$ and $s \in \{0, 1\}^{< d}$:
 - Mark the vertex (u, s) .
 - If $\exists x \neq x' : D(x) = D(x') = u$, return $\perp$.
 - Let x_0, x_1 be the unique values such that $D(x_0, x_1) = u$.
 - Update the vertex set $V := V \cup \{(x_0, s||0), (x_1, s||1)\}$.
 - Update the edge set $E := E \cup \{((u, s), (x_0, s||0)), ((u, s), (x_1, s||1))\}$.

Given a tree $T := \text{Extract}(D, \mathbf{rt}, d)$ we can define a string $\text{leaves}(T)$ where for each $i \in \{0, 1\}^d$, if there exists a vertex $(x, i) \in T$ then $\text{leaves}(T)_i := x$, else $\text{leaves}(T)_i := \perp$.

The formal construction of the Micali SNARK is now as follows:

- **SNARK.Prove**(x, w) $\rightarrow \pi$:
 - Prover P uses H to commit to the proof string $\Pi \leftarrow \mathbf{P}(x, w)$ via a Merkle tree, obtaining a corresponding root $\mathbf{rt}$.
 - Randomness r is derived by applying H to $\mathbf{rt}$.
 - P simulates $\mathbf{V}(\Pi, x, r)$ to deduce the queried locations $(v_i)_{i \in [q]}$ in Π .
 - Output $\pi = \{\mathbf{rt}, (v_i)_{i \in [q]}, (ap_i)_{i \in [q]}\}$.
- **SNARK.Verify**(π, x) $\rightarrow 0/1$:
 - Using H (by making queries to it), check that, for $i \in [q]$,

$$\text{CheckPath}^H(\mathbf{rt}, i, v, ap) = 1$$
 - Let $r = H(\mathbf{rt})$.
 - Run $\mathbf{V}(x, r)$:
 - * For every query $j_i \in [\ell]$, where j_i is the location of the i^{th} query made by $\mathbf{V}$, V runs $\text{CheckPath}^H(\mathbf{rt}, ap_i, v, j)$, and returns v_i if $\text{CheckPath}^H(\mathbf{rt}, ap_i, v, j) = 1$.
 - Output the same answer as $\mathbf{V}(x, r)$.

Theorem 8. [CMS19] Let $(\mathbf{P}, \mathbf{V})$ be a PCP for relation $\mathcal{R}$ that has soundness error ε , proof length ℓ and query complexity q . The Micali construction when based on $(\mathbf{P}, \mathbf{V})$ is a non-interactive argument for $\mathcal{R}$ that has soundness error $O(t^2\varepsilon + t^3/2^\lambda)$ against quantum attackers $\tilde{P}$ that make at most $t - O(q \log \ell)$ queries to H .

Now we will describe the knowledge extractor for the Micali SNARK in the QROM for any t' query algorithm $\tilde{P}$.

$\mathcal{E}^{*\tilde{P}}(1^{t'}, 1^\lambda)$:

1. Set $t = t' + O(q \log \ell)$.
2. Compute the quantum state $|\text{Sim}^*(\tilde{P})\rangle$ by simulating $\tilde{P}$, and recording its queries to the random oracle H with compressed oracles.
3. Measure the database register to get a database D .
4. For each $\mathbf{rt} \in D$:
 - Run $T \leftarrow \text{Extract}(D, \mathbf{rt}, d)$.
 - Set Π_T equal to leaves(T) on all points where leaves(T) $\neq \perp$, and 0 otherwise.
 - Run $x \leftarrow \mathbf{E}(\Pi_T)$.
 - Output x .

Lemma 7. [CMS19] The Micali construction, when based on a PCP of knowledge with knowledge error k , has the following guarantee:

- For any quantum attacker $\tilde{P}$ that makes at most t' queries to H , the following holds with probability $\Omega(\mu - t^2k - t^3/2^\lambda)$:
 - For all inputs (ct', π) such that $\text{SNARK.Verify}(\pi) = 1$ with probability $\geq \mu$, $\mathcal{E}^{*\tilde{P}}(1^{t'}, 1^\lambda)$ extracts a valid witness x .

5.4 Construction

Let $U_T(\cdot, \cdot)$ be the universal circuit that takes as input a Turing machine description P and an input x and outputs $P_T(x)$ (i.e. the output of the Turing machine after running for T steps). Let $U_{T,x} := U_T(\cdot, x)$ be the same universal circuit except that the input is hardcoded and fixed to x , so it only takes as input a Turing machine description P .

Let H be a random oracle. We describe the construction of succinct ideal obfuscation in Figure 1.

Succinct Ideal Obfuscation

- $\text{Obf}^H(1^\lambda, P, T)$:
 - $(\text{pk}, \text{sk}) \leftarrow \text{FHE.Gen}(1^\lambda)$
 - $\text{ct} \leftarrow \text{FHE.Enc}(\text{pk}, P)$
 - Define the circuit $O_{\text{ct}, \text{sk}}^H(\text{ct}', \pi)$ to take as input a ciphertext ct' and proof π . It runs $\text{SNARK.Verify}^H((\text{ct}, \text{ct}'), \pi)$ to check that π is a valid proof of the following NP statement:

$$\exists x : \text{ct}' = \text{FHE.Eval}(\text{pk}, U_{T,x}, \text{ct}).$$
 If $\text{SNARK.Verify}^H((\text{ct}, \text{ct}'), \pi)$ accepts, output $y \leftarrow \text{FHE.Dec}(\text{sk}, \text{ct}')$ and otherwise output $\perp$.
 - Output the obfuscation $(O[P], \text{pp})$ where $O[P] := O_{\text{ct}, \text{sk}}^H$ and $\text{pp} := \text{ct}$.
- $\text{Eval}^{O[P], H}(\text{pp}, x)$:
 - Parse $\text{pp} = \text{ct}$ and evaluate $\text{ct}' \leftarrow \text{FHE.Eval}(\text{pk}, U_x, \text{ct})$.
 - Prove that ct' was honestly generated: run $\pi \leftarrow \text{SNARK.Prove}^H((\text{ct}, \text{ct}'), x)$ to get a SNARK π of the statement

$$\exists x : \text{ct}' = \text{FHE.Eval}(\text{pk}, U_{T,x}, \text{ct}).$$
 - Output $O[P](\text{ct}', \pi)$.

Figure 1: Succinct Ideal Obfuscation using Classical Oracles and FHE

5.5 Correctness

Theorem 9. *The construction in Figure 1 satisfies correctness as defined in Definition 13.*

Proof. Correctness of the succinct obfuscation scheme in Figure 1 follows from the completeness of the SNARK and the correctness of the FHE scheme. $\square$

5.6 Succinctness

Theorem 10. *The construction in Figure 1 satisfies succinctness as defined in Definition 13.*

Proof. For a Turing machine with description P , its obfuscation consists of a circuit $O[P]$ that's accessible as an oracle and a FHE ciphertext $\text{ct} \leftarrow \text{FHE.Enc}(\text{pk}, P)$. The ciphertext ct encrypts the description of P , so its size is bounded $|\text{ct}| \leq \text{poly}(\lambda, |P|)$, which is succinct. The circuit $O[P](\text{ct}', \pi)$ consists of two circuits: (1) the SNARK verification circuit and (2) the FHE decryption circuit. By the succinctness of the SNARK scheme and compactness of the FHE scheme, both these circuits have size bounded by $\text{poly}(\lambda, |P|)$. Therefore the construction in Figure 1 satisfies succinctness. $\square$

5.7 Security

Theorem 11. *The construction in Figure 1 satisfies ideal obfuscation security as defined in Definition 13.*

Proof. We will start by formally describing the simulator $\mathcal{S}$.

Before describing our simulator, we need some terminology. We define a procedure **FindWitness**. Very roughly, **FindWitness** is the witness extractor from Lemma 7.4 in [CMS19], with the crucial difference being that the database register is not measured by **FindWitness**.

FindWitness

- **Input:**

- Database D .

- **Procedure:**

- Run $x \leftarrow \mathbf{E}(0^\ell)$, where 0^ℓ denotes the all 0s proof, and $\ell = 2^d$ for some $d \in \mathbb{N}$ is the proof length. If x is a valid witness, output x , otherwise continue.
- For each $\text{rt} \in D$:
 - * Run $T \leftarrow \text{Extract}(D, \text{rt}, d)$.
 - * Set Π_T equal to $\text{leaves}(T)$ on all points where $\text{leaves}(T) \neq \perp$, and 0 otherwise.
 - * Run $x \leftarrow \mathbf{E}(\Pi_T)$.
 - * Output x .

Figure 2: FindWitness

Now we are ready to define our simulator. On (superposition) query (ct', π) to $O[P]$, simulator $\mathcal{S}$ does the following:

- $\mathcal{S}$ outputs $\text{pp} = \text{FHE.Enc}(pk, 0)$.
- The joint state of the distinguisher $\mathcal{D}$ and H is given by a superposition over $|ct', \pi\rangle |y\rangle \otimes |D\rangle |\text{aux}\rangle$, where $|ct', \pi\rangle, |y\rangle$ are the query input and query output registers respectively, and $|D\rangle, |\text{aux}\rangle$ are the database D (recording the queries to H) and $\mathcal{D}$'s auxiliary registers respectively. $|D\rangle$ will update according to the Ctso procedure.
- On query (ct', π) to $O[P]$, $\mathcal{S}$ performs the following operation on the state $|ct'\rangle |\pi\rangle \otimes |D\rangle$:
 - Initialize ancilla register $|0\rangle_b$. Run $\text{SNARK.Verify}^H(\pi)$ and XOR the output in register b . If output is 0, output $\perp$.
 - Perform the map $|ct'\rangle |\pi\rangle \otimes |D\rangle \rightarrow |ct'\rangle |\pi\rangle \otimes |D\rangle |\text{FindWitness}(D)\rangle_{\mathcal{W}}$.
- Evaluate P on the input from register $\mathcal{W}$, and XOR in the query output register.
- Uncompute the $\mathcal{W}$ register by evaluating **FindWitness**(D) and XOR-ing in the $\mathcal{W}$ register.

Hybrid 0: Sample $(O[P], \text{pp}) \leftarrow \text{Obf}^H(1^\lambda, P)$.

Hybrid 1: Same as **Hybrid 0**, but the random oracle H is simulated as a compressed standard oracle for a database D .

Claim 15. $\Pr[\text{Hybrid 0} = 1] = \Pr[\text{Hybrid 1} = 1]$.

Proof. This follows directly from Lemma 4 in [Zha19a]. $\square$

Hybrid 2: Same as **Hybrid 1**, but we add an abort condition for $\text{FindWitness}(D)$ outputting invalid witnesses. More precisely,

- On query (ct', π) to $O[P]$, the joint state of the distinguisher $\mathcal{D}$ and H is $|ct', \pi\rangle |y\rangle \otimes |D\rangle |aux\rangle$
 - Initialize ancilla register $|0\rangle_b$. Run $\text{SNARK.Verify}^H(\pi)$ and XOR the output in register b . If output is 0, output $\perp$.
 - Measure the database register D , and run $\text{FindWitness}(D)$.
 - If $x \leftarrow \text{FindWitness}(D)$ is not a valid witness, output $\perp$.
 - Otherwise, compute $y \leftarrow \text{FHE.Dec}(sk, ct')$ and XOR into the query output register of $\mathcal{D}$.

Claim 16. $\Pr[\text{Hybrid 2} = 1] - \Pr[\text{Hybrid 1} = 1] \leq \text{negl}(\lambda)$

Proof. Assume otherwise, then there must exist some query made by $\mathcal{D}$ with noticeable amplitude on $|ct', \pi\rangle \otimes |D\rangle$ such that $\text{FindWitness}(D)$ does not output a valid witness, but $\text{SNARK.Verify}^H(\pi) = 1$. Formally, define the following set B of inputs of the form

$$B = \{(ct', \pi, D, b) : x \leftarrow \text{FindWitness}(D) \text{ is not valid but } b = 1\}$$

Let $|\psi\rangle$ be the joint state of $\mathcal{D}$ and H right before measuring the database D . Then our assumption implies that

$$\|\Pi[B] |\psi\rangle\|^2 = \varepsilon(\lambda)$$

for some non negligible function ε . For a database D , define $B[D] := \{(\cdot, D) : (\cdot, D) \in B\}$. We can conclude that there exists a database D' such that

$$\|\Pi[B[D']] |\psi\rangle\|^2 = \varepsilon'(\lambda),$$

where $\varepsilon'(\cdot)$ is also a non negligible function. Now define the state

$$|\psi'\rangle = \langle x | \text{FindWitness}(D') |\psi\rangle, |\psi^*\rangle := \frac{|\psi'\rangle}{\|\psi'\rangle\|}$$

So $|\psi^*\rangle$ is the result of running $\text{FindWitness}(D')$ coherently, and post selecting on the witness being invalid. From the gentle measurement lemma, we can conclude that, the probability that $\text{FindWitness}(D')$ finds an invalid witness on $\Pi[B[D']] |\psi^*\rangle$ is non negligible as well, but this is a contradiction to Lemma 7. $\square$

Hybrid 3: Same as **Hybrid 2**, but queries to $O[P]$ are answered as follows:

- On query (ct', π) to $O[P]$, $\mathcal{S}$ performs the following operation on the state $|ct'\rangle |\pi\rangle \otimes |D\rangle$:
 - Initialize ancilla register $|0\rangle_b$. Run $\text{SNARK.Verify}^H(\pi)$ and XOR the output in register b . If output is 0, output $\perp$.
 - Perform the map $|ct'\rangle |\pi\rangle \otimes |D\rangle \rightarrow |ct'\rangle |\pi\rangle \otimes |D\rangle |\text{FindWitness}(D)\rangle_{\mathcal{W}}$.
 - If $x \leftarrow \text{FindWitness}(D)$ is not a valid witness, that is, if $P(x) \neq \text{FHE.Dec}(sk, ct')$, output $\perp$.
 - Otherwise, evaluate P on the input from register $\mathcal{W}$, and XOR in the query output register.
- Uncompute the $\mathcal{W}$ register by evaluating $\text{FindWitness}(D)$ and XOR-ing in the $\mathcal{W}$ register.

Claim 17. $\Pr[\text{Hybrid 2} = 1] = \Pr[\text{Hybrid 3} = 1]$

Proof. First observe that **FindWitness** is essentially the extractor $\mathcal{E}^*$ for the Micali SNARK, with the only (but crucial) difference being that the database D is not measured. Since **Extract** aborts if there exists a $u : D(x) = D(x') = u, x \neq x'$, by construction, this implies that, for a fixed database D , **FindWitness**(D) has a unique solution. Thus, when all invalid solutions are aborted, it follows from the correctness of $\mathcal{E}^*$ that queries are answered identically in **Hybrid 2** and **Hybrid 3**. $\square$

Hybrid 4: Same as **Hybrid 3**, but the abort condition for **FindWitness** outputting invalid witnesses is removed, and queries to $O[P]$ are answered by $\mathcal{S}$ as follows:

- On query (ct', π) to $O[P]$, $\mathcal{S}$ performs the following operation on the state $|ct'\rangle |\pi\rangle \otimes |D\rangle$:
 - Initialize ancilla register $|0\rangle_b$. Run $\text{SNARK.Verify}^H(\pi)$ and XOR the output in register b . If output is 0, output $\perp$.
 - Perform the map $|ct'\rangle |\pi\rangle \otimes |D\rangle \rightarrow |ct'\rangle |\pi\rangle \otimes |D\rangle |\text{FindWitness}(D)\rangle_{\mathcal{W}}$.
 - Evaluate P on the input from register $\mathcal{W}$, and XOR in the query output register.
- Uncompute the $\mathcal{W}$ register by evaluating **FindWitness**(D) and XOR-ing in the $\mathcal{W}$ register.

Claim 18. $\left| \Pr[\text{Hybrid 3} = 1] - \Pr[\text{Hybrid 4} = 1] \right| \leq \text{negl}(\lambda)$

Proof. The proof of this claim is identical to Claim 16. $\square$

Hybrid 5: This is the ideal world.

- Queries to $O[P]$ are answered as in the description of $\mathcal{S}$ as follows:
 - Initialize ancilla register $|0\rangle_b$. Run $\text{SNARK.Verify}^H(\pi)$ and XOR the output in register b . If output is 0, output $\perp$.
 - Perform the map $|ct'\rangle |\pi\rangle \otimes |D\rangle \rightarrow |ct'\rangle |\pi\rangle \otimes |D\rangle |\text{FindWitness}(D)\rangle_{\mathcal{W}}$.
 - Evaluate P on the input from register $\mathcal{W}$, and XOR in the query output register.
- Uncompute the $\mathcal{W}$ register by evaluating **FindWitness**(D) and XOR-ing in the $\mathcal{W}$ register.
- Output $\text{pp} \leftarrow \text{FHE.Enc}(pk, 0^{|P|})$.

Claim 19. $\left| \Pr[\text{Hybrid 5} = 1] - \Pr[\text{Hybrid 4} = 1] \right| \leq \text{negl}(\lambda)$

Proof. This follows directly from the security of the FHE scheme. $\square$

$\square$

6 Publicly-verifiable QFHE

We define publicly-verifiable QFHE in the oracle model. We consider both leveled and unleveled QFHE.

6.1 Definition

Definition 16 (Publicly-verifiable QFHE). *A leveled publicly-verifiable quantum fully-homomorphic encryption scheme for pseudo-deterministic circuits consists of the following algorithms (Gen, Enc, Eval, Ver, Dec).*

- $\text{Gen}(1^\lambda, D, N, S) \rightarrow (\text{pk}, \text{sk}, \text{PP})$: The key generation algorithm is a PPT algorithm that takes as input the security parameter 1^λ , an upper bound on circuit depth D , an upper bound on input length N , and an upper bound on circuit description size S , and returns a public key pk , a secret key sk , and the description of a classical deterministic circuit PP .
- $\text{Enc}(\text{pk}, x) \rightarrow \text{ct}$: The encryption algorithm is a PPT algorithm that takes as input the public key pk and a classical plaintext x , and outputs a ciphertext ct .
- $\text{Eval}^{\text{PP}}(\text{ct}, Q) \rightarrow (\tilde{\text{ct}}, \pi)$: The evaluation algorithm is a QPT algorithm that has oracle access to PP , takes as input a ciphertext ct , a quantum circuit Q , and outputs an evaluated ciphertext $\tilde{\text{ct}}$ and proof π .
- $\text{Ver}^{\text{PP}}(\text{ct}, P_Q, \tilde{\text{ct}}, \pi) \rightarrow \{\top, \perp\}$: The verification algorithm is a PPT algorithm that has oracle access to PP , takes an input ciphertext ct , a quantum program P_Q (i.e. a potentially succinct description of the quantum circuit Q), an output ciphertext $\tilde{\text{ct}}$ and a proof π , and accepts ($\top$) or rejects ($\perp$).
- $\text{Dec}(\text{sk}, \text{ct}) \rightarrow x$: The decryption algorithm is a PPT algorithm that takes as input the secret key sk and a classical ciphertext ct , and outputs plaintext x .

These algorithms should satisfy the following properties. Recall that pseudo-deterministic circuits Q are defined to have a single bit of output, without loss of generality.

- **Correctness.** For any polynomials $D = D(\lambda), N = N(\lambda), S = S(\lambda)$, any pseudo-deterministic program P_Q where Q has circuit depth at most D and $|P_Q| \leq S$, and any input x with $|x| \leq N$,

$$\Pr \left[\begin{array}{l} \text{Ver}^{\text{PP}}(\text{ct}, P_Q, \tilde{\text{ct}}, \pi) = \top \wedge \\ \text{Dec}(\text{sk}, \tilde{\text{ct}}) = Q(x) \end{array} : \begin{array}{l} (\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{Gen}(1^\lambda, D, N, S) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x) \\ (\tilde{\text{ct}}, \pi) \leftarrow \text{Eval}^{\text{PP}}(\text{ct}, Q) \end{array} \right] = 1 - \text{negl}(\lambda).$$

- **Privacy.** For any QPT adversary $\mathcal{A} = \{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$, polynomials $D = D(\lambda), N = N(\lambda), S = S(\lambda)$, and messages x_0, x_1 ,

$$\left| \Pr \left[\mathcal{A}^{\text{PP}}(\text{pk}, \text{ct}) = 1 : \begin{array}{l} (\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{Gen}(1^\lambda, D, N, S) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x_0) \end{array} \right] - \Pr \left[\mathcal{A}^{\text{PP}}(\text{pk}, \text{ct}) = 1 : \begin{array}{l} (\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{Gen}(1^\lambda, D, S) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x_1) \end{array} \right] \right| = \text{negl}(\lambda).$$

- **Soundness.** For any QPT adversary $\mathcal{A} = \{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$, polynomials $D = D(\lambda), N = N(\lambda), S = S(\lambda)$, pseudo-deterministic program P_Q where Q has circuit depth at most D and $|P_Q| \leq S$, and input x with $|x| \leq N$,

$$\Pr \left[\begin{array}{l} \text{Ver}^{\text{PP}}(\text{ct}, P_Q, \tilde{\text{ct}}, \pi) = \top \wedge \\ \text{Dec}(\text{sk}, \tilde{\text{ct}}) \neq Q(x) \end{array} : \begin{array}{l} (\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{Gen}(1^\lambda, D, N, S) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x) \\ (\tilde{\text{ct}}, \pi) \leftarrow \mathcal{A}^{\text{PP}}(\text{ct}) \end{array} \right] = \text{negl}(\lambda).$$

- **Compactness.** There exists a fixed polynomial p such that the following holds. Fix any polynomials $D = D(\lambda), N = N(\lambda), S = S(\lambda)$, any pseudo-deterministic program P_Q where Q has circuit depth at most D and $|P_Q| \leq S$, and any x with $|x| \leq N$, and let $(\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{Gen}(1^\lambda, D, N, S)$, $\text{ct} \leftarrow \text{Enc}(\text{pk}, x)$, and $(\tilde{\text{ct}}, \pi) \leftarrow \text{Eval}^{\text{PP}}(\text{ct}, Q)$. Then it holds that

- $|\mathbf{pk}|, |\mathbf{sk}|, |\mathbf{PP}|, |\mathbf{ct}|, |\tilde{\mathbf{ct}}|, |\pi| \leq p(\lambda, D, N, S)$, and
- the run-times of $\text{Gen}(1^\lambda, D, N, S)$, $\text{Enc}(\mathbf{pk}, x)$, $\text{Ver}^{\text{PP}}(\mathbf{ct}, P_Q, \tilde{\mathbf{ct}}, \pi)$, and $\text{Dec}(\mathbf{sk}, \tilde{\mathbf{ct}})$ are bounded by $p(\lambda, D, N, S)$.

An *unleveled* publicly-verifiable QFHE scheme is defined exactly as above, except that Gen no longer takes a depth parameter D and all properties are defined without the parameter D .

6.2 Upgradable privately-verifiable QFHE

We will construct publicly-verifiable QFHE by *upgrading* a privately-verifiable scheme that satisfies certain properties. These properties are encapsulated in the following definition.

Definition 17 (Upgradable privately-verifiable QFHE). *A leveled privately-verifiable QFHE scheme that is amenable to being upgraded to public-verifiability via our compiler has the following syntax. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}$ be a random oracle. Let $\ell(\lambda, |Q|)$ be the “state size” parameter and $p(\lambda, N, D)$ be the “compactness” parameter.*

- $\text{Gen}(1^\lambda, D) \rightarrow (\mathbf{pk}, \mathbf{sk})$: The PPT key generation algorithm takes as input the security parameter 1^λ and an upper bound on circuit depth D and outputs a public key $\mathbf{pk}$ and secret key $\mathbf{sk}$.
- $\text{Enc}(\mathbf{pk}, x) \rightarrow \mathbf{ct}$: The PPT encryption algorithm takes as input a public key $\mathbf{sk}$ and an input x and outputs a ciphertext $\mathbf{ct}$.
- $\text{VerGen}(1^\lambda, \mathbf{ct}, Q) \rightarrow \mathbf{pp}, \mathbf{sp}$: The PPT verification parameter generation algorithm takes as input the security parameter 1^λ , a ciphertext $\mathbf{ct}$, and quantum circuit Q , and outputs public parameters $\mathbf{pp}$ and secret parameters $\mathbf{sp}$.
- $\text{Eval}^H(\mathbf{pp}, \mathbf{ct}, Q) \rightarrow \pi$: The QPT evaluation algorithm operates as follows.
 - Let $\ell = \ell(\lambda, |Q|)$, initialize an ℓ -qubit state $|\psi_{\mathbf{ct}, Q}\rangle$ on register $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_\ell)$, and initialize two fresh registers $\mathcal{Y}, \mathcal{Z}$.
 - Apply a measurement $M_{\mathbf{pp}}$ on $(\mathcal{M}, \mathcal{Z}, \mathcal{Y})$ that is classically-controlled on $\mathcal{M}$ to obtain a string y .
 - Compute $T = H(y)$, where T is parsed as a subset $T \subseteq [\ell]$.¹⁹
 - Apply a measurement $M_{\mathbf{pp}, y, T}$ on $(\mathcal{M}, \mathcal{Z})$ that is classically-controlled on $\mathcal{M}$ to obtain a string z .
 - For all $i \in [\ell]$, measure $\mathcal{M}_i$ in the standard basis if $i \in T$ or in the Hadamard basis if $i \notin T$ to obtain a string $m \in \{0, 1\}^\ell$.
 - Return $\pi = (m, y, z)$.
- $\text{Ver}^H(\mathbf{sp}, \mathbf{ct}, Q, \pi) \rightarrow \{\perp, \tilde{\mathbf{ct}}\}$: The PPT verification algorithm takes as input secret parameters $\mathbf{sp}$, a ciphertext $\mathbf{ct}$, a quantum circuit Q , and a proof π , and outputs either reject ($\perp$) or a ciphertext $\tilde{\mathbf{ct}}$.
- $\text{Dec}(\mathbf{sk}, \mathbf{ct}) \rightarrow x$: The PPT decryption algorithm takes as input the secret key $\mathbf{sk}$ and a ciphertext $\mathbf{ct}$ and outputs a plaintext x .

It should satisfy the following properties. We leave probabilities taken over the sampling of the random oracle H implicit.

- **Privacy.** For any QPT adversary $\{\mathcal{A}_\lambda\}_\lambda$, depth $D = D(\lambda)$, and messages x_0, x_1 ,

$$\left| \Pr \left[\mathcal{A}(\mathbf{pk}, \mathbf{ct}) = 1 : \begin{array}{l} (\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda, D) \\ \mathbf{ct} \leftarrow \text{Enc}(\mathbf{pk}, x_0) \end{array} \right] - \Pr \left[\mathcal{A}(\mathbf{pk}, \mathbf{ct}) = 1 : \begin{array}{l} (\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda, D) \\ \mathbf{ct} \leftarrow \text{Enc}(\mathbf{pk}, x_1) \end{array} \right] \right| = \text{negl}(\lambda).$$

¹⁹In a slight abuse of notation, we define an ℓ -bit output $H(y) := H(1, y) \parallel \dots \parallel H(\ell, y)$.

- **Completeness.** For any $(pk, sk) \in \text{Gen}(1^\lambda, D)$, $ct \in \text{Enc}(pk, x)$, and Q ,

$$\Pr \left[\text{Dec}(sk, \text{Ver}^H(sp, ct, Q, \pi)) = Q(x) : \begin{array}{l} pp, sp \leftarrow \text{VerGen}(ct, Q) \\ \pi \leftarrow \text{Eval}^H(pp, ct, Q) \end{array} \right] = 1 - \text{negl}(\lambda).$$

- **Soundness.** For any $(pk, sk) \in \text{Gen}(1^\lambda, D)$, $ct \in \text{Enc}(pk, x)$, Q , and QPT adversary $\{\mathcal{A}_\lambda\}_\lambda$,

$$\Pr \left[\tilde{ct} \neq \perp \wedge \text{Dec}(sk, \tilde{ct}) \neq Q(x) : \begin{array}{l} pp, sp \leftarrow \text{VerGen}(ct, Q) \\ \pi \leftarrow \mathcal{A}^H(pp) \\ \tilde{ct} \leftarrow \text{Ver}^H(sp, ct, Q, \pi) \end{array} \right] = \text{negl}(\lambda).$$

- **Incompatible standard-basis measurements.** First, we introduce some notation.

- Given a subset $I \subset [\ell]$, define $\bar{I} := [\ell] \setminus I$.
- Given any $m \in \{0, 1\}^\ell$ and a subset $I \subseteq [\ell]$, we define $m_{I \rightarrow *}$ to be equal to m_i at indices $i \in \bar{I}$ and equal to $*$ at indices $i \in I$.
- Each choice of $(\cdot, sp) \in \text{VerGen}(ct, Q)$ defines²⁰ a set of “standard basis positions” $S \subseteq [\ell]$, which we denote by $S = S[sp]$ (not to be confused with the positions $T \subseteq [\ell]$ that are measured in the standard basis).
- We say that two sets of strings $M_0, M_1 \in \{0, 1, *\}^\ell$ are incompatible if for any $m_0 \in M_0, m_1 \in M_1$, there exists an $i \in [\ell]$ such that $m_{0,i}, m_{1,i} \neq *$ and $m_{0,i} \neq m_{1,i}$.
- Each choice of $(\cdot, sp) \in \text{VerGen}(ct, Q)$ defines²¹ two incompatible sets of strings $M_0, M_1 \in \{0, 1, *\}^\ell$.

Now, we need the following properties.

- For any $(pk, sk) \in \text{Gen}(1^\lambda, D)$, $ct \in \text{Enc}(pk, x)$, Q , $(pp, sp) \in \text{VerGen}(ct, Q)$, and proof $\pi = (m, y, z)$, it holds that

$$\text{Ver}^H(sp, ct, Q, (m, y, z)) = \text{Ver}^H(sp, ct, Q, (m_{S \cap \bar{T} \rightarrow *}, y, z)).$$

That is, Ver ignores all bits of m that are measured in the Hadamard basis, but correspond to “standard basis positions”.

- For any $(pk, sk) \in \text{Gen}(1^\lambda, D)$, $ct \in \text{Enc}(pk, x)$, Q , and QPT adversary $\{\mathcal{A}_\lambda\}_\lambda$,

$$\Pr \left[\tilde{ct} \neq \perp \wedge m_{\bar{S} \cup \bar{T} \rightarrow *} \notin M_0 : \begin{array}{l} pp, sp \leftarrow \text{VerGen}(ct, Q) \\ \pi = (m, y, z) \leftarrow \mathcal{A}^H(pp) \\ T := H(y) \\ \tilde{ct} \leftarrow \text{Ver}^H(sp, ct, Q, \pi) \end{array} \right] = \text{negl}(\lambda).$$

That is, it is hard to find an accepting proof π such that $m_{\bar{S} \cap \bar{T} \rightarrow *}$ (where only the standard basis positions that are measured in the standard basis remain) is in M_0 .

- For any $(pk, sk) \in \text{Gen}(1^\lambda, D)$, $ct \in \text{Enc}(pk, x)$, Q , and QPT adversary $\{\mathcal{A}_\lambda\}_\lambda$,

$$\Pr \left[\tilde{ct} \neq \perp \wedge \text{Dec}(sk, \tilde{ct}) \neq Q(x) \wedge m_{\bar{S} \cup \bar{T} \rightarrow *} \notin M_1 : \begin{array}{l} pp, sp \leftarrow \text{VerGen}(ct, Q) \\ \pi = (m, y, z) \leftarrow \mathcal{A}^H(pp, sp) \\ T := H(y) \\ \tilde{ct} \leftarrow \text{Ver}^H(sp, ct, Q, \pi) \end{array} \right] = \text{negl}(\lambda).$$

That is, it is hard to find an accepting but faulty proof π such that $m_{\bar{S} \cap \bar{T} \rightarrow *}$ (where only the standard basis positions that are measured in the standard basis remain) is in M_1 , even given the secret parameters sp .

²⁰The way that S is defined based on sp is specified in [BKNY23, Section 5.3], though the details will not be important to us here.

²¹As above, this definition is specified in [BKNY23, Section 5.3], and the details will not be important to us here.

- **Parallelizable VerGen.** *There exists a fixed polynomial $k(\lambda)$ and a deterministic classical algorithm $\text{ParVerGen}(\text{ct}, Q, \text{sp}, i) \rightarrow \text{pp}_i$, where $|\text{pp}_i| = k(\lambda)$, such that for any ct, Q , the operation $\text{VerGen}(1^\lambda, \text{ct}, Q)$ is equivalent to:*
 - Sample $\text{sp} \leftarrow \{0, 1\}^\lambda$
 - Run $\text{pp}_i := \text{ParVerGen}(\text{ct}, Q, \text{sp}, i)$ for $i \in [\ell]$, and set $\text{pp} := (\text{pp}_1, \dots, \text{pp}_\ell)$.
- **Compactness.** *Fix any polynomials $N = N(\lambda), D = D(\lambda)$, any input x such that $|x| \leq N$, and any pseudo-deterministic quantum circuit Q with depth at most D , and let $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, D), \text{ct} \leftarrow \text{Enc}(\text{pk}, x), (\text{pp}, \text{sp}) \leftarrow \text{VerGen}(\text{ct}, Q), \pi \leftarrow \text{Eval}^H(\text{pp}, \text{ct}, Q), \tilde{\text{ct}} \leftarrow \text{Ver}^H(\text{sp}, \text{ct}, Q, \pi)$, and $Q(x) \leftarrow \text{Dec}(\text{sk}, \text{ct})$. Then it holds that*
 - $|\text{pk}|, |\text{sk}|, |\text{ct}|, |\tilde{\text{ct}}| \leq p(\lambda, N, D)$, and
 - The run-times of $\text{Gen}(1^\lambda, D), \text{Enc}(\text{pk}, x), \text{Dec}(\text{sk}, \text{ct})$ are bounded by $p(\lambda, N, D)$.

An *unleveled* upgradable privately-verifiable QFHE scheme is defined exactly as above, except that Gen no longer takes the depth parameter D and all properties are defined without the parameter D .

Theorem 12 ([BKNY23], Section 5.3). *Assuming LWE, there exists a leveled upgradable privately-verifiable QFHE in the QROM. Assuming LWE plus an appropriate circular-security assumption [Bra18], there exists unleveled upgradable privately-verifiable QFHE in the QROM.*

6.3 Construction

Publicly-verifiable QFHE

- $\text{Gen}(1^\lambda, N, D, S)$:
 - Sample $\text{pk}, \text{sk} \leftarrow \text{Priv.Gen}(1^\lambda)$.
 - Sample PRF keys $k_{\text{PFC}}, k_{\text{Priv}}, k_H \leftarrow \{0, 1\}^\lambda$.
 - Sample $\text{O} \leftarrow \text{OSS.Setup}(1^\lambda)$.
 - Define $\text{CK}(\text{pk}_{\text{OSS}}, i, v)$ as follows.
 - * Compute $(\text{CK}_{\text{pk}, i}, \text{dk}_{\text{pk}, i}) := \text{PFC.Gen}(1^\lambda, F_{k_{\text{PFC}}}(\text{pk}_{\text{OSS}}, i))$.
 - * Output $\text{CK}_{\text{pk}, i}(v)$.
 - Define $\text{PrivGen}(\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, i)$ as follows.
 - * If $\text{OSS.Ver}^{\text{O}}(\text{pk}_{\text{OSS}}, (\text{ct}, Q, c), \sigma) = \top$, then continue, and otherwise return $\perp$.
 - * Define $\text{sp} := F_{k_{\text{Priv}}}(\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma)$.
 - * Output $\text{pp}_i := \text{Priv.ParVerGen}(\text{ct}, Q, \text{sp}, i)$.
 - Let $\text{H} := F_{k_H}$.
 - Define $\text{PrivVer}(\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$ as follows.
 - * If $\text{OSS.Ver}^{\text{O}}(\text{pk}_{\text{OSS}}, (\text{ct}, Q, c), \sigma) = \top$, then set $\text{sp} := F_{k_{\text{Priv}}}(\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma)$, $S := S[\text{sp}]$, and otherwise return $\perp$.
 - * Parse $u = (u_1, \dots, u_\ell)$, $c = (c_1, \dots, c_\ell)$, and define $T := \text{H}(y)$.
 - * For each $i \in [\ell]$, compute $(\text{CK}_{\text{pk}, i}, \text{dk}_{\text{pk}, i}) := \text{PFC.Gen}(1^\lambda, F_{k_{\text{PFC}}}(\text{pk}_{\text{OSS}}, i))$.
 - * For each $i \in T$, compute $m_i := \text{PFC.DecZ}(\text{dk}_{\text{pk}, i}, c_i, u_i)$, and return $\perp$ if $m_i = \perp$.
 - * For each $i \in \bar{T} \setminus S$, compute $m_i := \text{PFC.DecX}(\text{dk}_{\text{pk}, i}, c_i, u_i)$, and return $\perp$ if $m_i = \perp$.
 - * For each $i \in \bar{T} \cap S$, set $m_i = *$.
 - * Let $m = (m_1, \dots, m_\ell)$, and output $\text{Priv.Ver}^{\text{H}}(\text{sp}, \text{ct}, Q, (m, y, z))$.
 - Output $\text{pk}, \text{sk}, \text{PP} := \text{SObf.Obf}(1^\lambda, (\text{O}, \text{CK}, \text{PrivGen}, \text{H}, \text{PrivVer}), T)$, where T is the size of the circuit required to implement the concatenation of programs $(\text{O}, \text{CK}, \text{PrivGen}, \text{H}, \text{PrivVer})$.
- $\text{Enc}(\text{pk}, x)$: Output $\text{ct} \leftarrow \text{Priv.Enc}(\text{pk}, x)$.
- $\text{Eval}^{\text{PP}}(\text{ct}, Q)$
 - Let $\ell = \ell(\lambda, |Q|)$ and sample $(\text{pk}_{\text{OSS}}, |\text{sk}_{\text{OSS}}|) \leftarrow \text{OSS.Gen}^{\text{O}}(|\text{ct}| + |Q| + \ell \cdot t(\lambda))$.
 - Initialize $|\psi_{\text{ct}, Q}\rangle$ on register $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_\ell)$.
 - For each $i \in [\ell]$ apply $\text{PFC.Com}^{\text{CK}(\text{pk}_{\text{OSS}}, i, \cdot)} : \mathcal{M}_i \rightarrow \mathcal{M}_i, \mathcal{U}_i, c_i$, and set $c := (c_1, \dots, c_\ell)$.
 - Compute $\sigma \leftarrow \text{OSS.Sign}^{\text{O}}(|\text{sk}_{\text{OSS}}\rangle, (\text{ct}, Q, c))$ and, for $i \in [\ell]$, compute $\text{pp}_i \leftarrow \text{PrivGen}(\text{pk}_{\text{OSS}}, \text{ct}, Q, c, \sigma, i)$. Set $\text{pp} := (\text{pp}_1, \dots, \text{pp}_\ell)$.
 - Initialize fresh registers $\mathcal{Y}, \mathcal{Z}$, apply M_{pp} on $(\mathcal{M}, \mathcal{Y}, \mathcal{Z})$ to obtain y , compute $T = \text{H}(y)$, and apply $M_{\text{pp}, y, T}$ on $(\mathcal{M}, \mathcal{Z})$ to obtain z .
 - For each $i \in T$, compute $u_i \leftarrow \text{PFC.OpenZ}(\mathcal{M}_i, \mathcal{U}_i)$ and for each $i \in \bar{T}$, compute $u_i \leftarrow \text{PFC.OpenX}(\mathcal{M}_i, \mathcal{U}_i)$. Set $u := (u_1, \dots, u_\ell)$.
 - Output $\tilde{\text{ct}} := \text{PrivVer}(\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$ and $\pi := (\text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$.
- $\text{Ver}^{\text{PP}}(\text{ct}, Q, \tilde{\text{ct}}, \pi)$: Output $\top$ iff $\text{PrivVer}(\text{ct}, Q, \pi) = \tilde{\text{ct}}$.
- $\text{Dec}(\text{sk}, \text{ct})$: Output $\text{Priv.Dec}(\text{sk}, \text{ct})$.

Figure 3: Our construction of publicly-verifiable QFHE in the classical oracle model.

Ingredients:

- PRF $F_k : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.
- OSS (OSS.Setup, OSS.Gen, OSS.Sign, OSS.Ver) in the classical oracle model (Definition 3).
- PFC (PFC.Gen, PFC.Com, PFC.OpenZ, PFC.OpenZ, PFC.DecZ, PFC.DecX) in the classical oracle model (see Definition 5), where single-qubit commitments are size $t(\lambda)$.
- Upgradable privately-verifiable QFHE (Priv.Gen, Priv.Enc, Priv.VerGen, Priv.Eval^H, Priv.Ver^H, Priv.Dec), in the QROM with state size parameter $\ell(\lambda, |Q|)$ and compactness parameter $p(\lambda, N, D)$ (see Definition 17).
- Post-quantum succinct ideal obfuscation (SObf.Ofb, SObf.Eval) in the classical oracle model (see Definition 13). In an abuse of notation, we will apply SObf.Ofb to a program with multiple bits of output, meaning that we break the program into multiple single-bit output programs, and obfuscate each.

6.4 Proofs

Theorem 13. *The QFHE protocol given in Figure 3 satisfies correctness (see Definition 16).*

Proof. This is a straightforward adaptation of the completeness part of [BKNY23, Theorem 5.12], where we use the correctness of OSS (along with the other primitives) rather than the signature token. $\square$

Theorem 14. *The QFHE protocol given in Figure 3 satisfies privacy (see Definition 16).*

Proof. This follows directly from the privacy of the upgradable privately-verifiable QFHE. $\square$

Theorem 15. *The QFHE protocol given in Figure 3 satisfies compactness (see Definition 16).*

Proof. This follows directly from the compactness of the upgradable privately-verifiable QFHE and succinctness of the post-quantum succinct ideal obfuscator. In particular, note that the output size of the program obfuscated by Gen is bounded by a fixed polynomial, and so PP consists of a bounded polynomial number of obfuscated programs (each with bounded circuit size). $\square$

Theorem 16. *The QFHE protocol given in Figure 3 satisfies soundness (see Definition 16).*

Proof. We follow the soundness part of the proof of [BKNY23, Theorem 5.12], but with a couple of tweaks.

Suppose there exists a program P_{Q^*} , input x , and adversary $\mathcal{A}_1^{\text{PP}}$ that violates the soundness of Definition 16, where we have dropped the indexing by λ for convenience. Our first step will be to appeal to the security of the succinct obfuscation (Definition 13) to replace $\mathcal{A}_1$ with an adversary $\mathcal{A}_2^{\text{O,CK,PrivGen,H,PrivVer}}$ that has direct access to the large-input functionalities. This only has a negligible affect on the output of $\mathcal{A}_1$.

Next, we replace the PRFs $F_{k_{\text{PFC}}}, F_{k_{\text{Priv}}}, F_{k_H}$ with random oracles $H_{\text{PFC}}, H_{\text{Priv}}, H$. Since $\mathcal{A}_2$ only has polynomially-bounded oracle access to these functionalities, this has a negligible effect on the output of $\mathcal{A}_2$ [Zha12]. $\mathcal{A}_2$ can then be used to define an oracle algorithm $\mathcal{A}_3^{\text{H}_{\text{Priv}}}$ that operates as follows.

- Sample $(\text{pk}, \text{sk}, \text{O}, \text{CK}, \text{PrivGen}, \text{H}, \text{PrivVer})$ as in $\text{Gen}(1^\lambda, N, D, S)$, except that H_{PFC} and H are sampled as random oracles.
- Sample $\text{ct}^* \leftarrow \text{Priv.Enc}(\text{pk}, x)$.
- Run $\mathcal{A}_2^{\text{O,CK,PrivGen,H,PrivVer}}(\text{ct}^*)$, forwarding calls to H_{Priv} (which occur as part of calls to PrivGen and PrivVer) to an external random oracle H_{Priv} .
- Measure $\mathcal{A}_2$'s output proof π^* , parse π^* as $(\text{pk}_{\text{OSS}}^*, c^*, \sigma^*, m^*, y^*, z^*)$ and output $a := (\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)$ and $\text{aux} := (m^*, y^*, z^*)$.

Note that $\mathcal{A}_3$ makes $p = \text{poly}(\lambda)$ total queries to H_{Priv} . Now, define V as in Figure 4. Then since $\mathcal{A}_1$ breaks soundness,

$$\Pr \left[V(a, \text{H}_{\text{Priv}}(a), \text{aux}) = 1 : (a, \text{aux}) \leftarrow \mathcal{A}_3^{\text{H}_{\text{Priv}}} \right] = \text{non-negl}(\lambda).$$

Next, since $p = \text{poly}(\lambda)$, by Theorem 3 there exists an algorithm $\mathcal{A}_4 := \text{Sim}[\mathcal{A}_3]$ such that

$$\Pr \left[V((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), s, (m^*, y^*, z^*)) = 1 : \begin{array}{l} ((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), \text{state}) \leftarrow \mathcal{A}_4 \\ s \leftarrow \{0, 1\}^\lambda \\ (m^*, y^*, z^*) \leftarrow \mathcal{A}_4(s, \text{state}) \end{array} \right] = \text{non-negl}(\lambda).$$

Moreover, $\mathcal{A}_4$ operates as follows.

- Sample H_{Priv} as a $2p$ -wise independent function and $(i, d) \leftarrow (\{0, \dots, p-1\} \times \{0, 1\}) \cup \{(p, 0)\}$.
- Run $\mathcal{A}_3$ for i oracle queries, answering each query using the function H_{Priv} .
- When $\mathcal{A}_3$ is about to make its $(i+1)$ 'th oracle query, measure its query register in the standard basis to obtain $a := (\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)$. In the special case that $(i, d) = (p, 0)$, just measure (part of) the final output register of $\mathcal{A}_3$ to obtain a .
- Receive s externally.
- If $d = 0$, answer $\mathcal{A}_3$'s $(i+1)$ 'th query with H_{Priv} . If $d = 1$, answer $\mathcal{A}_3$'s $(i+1)$ 'th query instead with the re-programmed oracle $\text{H}_{\text{Priv}}[(\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*) \rightarrow s]$.
- Run $\mathcal{A}_3$ until it has made all p queries to H_{Priv} . For queries $i+2$ through p , answer with the re-programmed oracle $\text{H}_{\text{Priv}}[(\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*) \rightarrow s]$.
- Measure $\mathcal{A}_3$'s output $\text{aux} := (u^*, y^*, z^*)$.

Recall that $\mathcal{A}_3$ is internally running $\mathcal{A}_2$, who expects oracle access to $\text{O}, \text{CK}, \text{PrivGen}, \text{H}, \text{PrivVer}$. These oracle queries will be answered by $\mathcal{A}_4$. Next, we define $\mathcal{A}_5$ to be the same as $\mathcal{A}_4$, except that after $(\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)$ is measured by $\mathcal{A}_4$, $\mathcal{A}_2$'s queries to PrivVer are answered instead with the functionality $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)]$ from Figure 4.

Claim 20.

$$\Pr \left[V((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), s, (m^*, y^*, z^*)) = 1 : \begin{array}{l} ((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), \text{state}) \leftarrow \mathcal{A}_5 \\ s \leftarrow \{0, 1\}^\lambda \\ (m^*, y^*, z^*) \leftarrow \mathcal{A}_5(s, \text{state}) \end{array} \right] = \text{non-negl}(\lambda).$$

Proof. We can condition on $\text{OSS.Ver}^{\text{O}}(\text{pk}_{\text{OSS}}^*, (\text{ct}^*, Q^*, c^*), \sigma^*) = \top$, since otherwise V would output 0. Then, by the security of OSS (Definition 3), once $(\text{pk}_{\text{OSS}}^*, \text{ct}^*, Q^*, c^*, \sigma^*)$ is measured, $\mathcal{A}_2$ cannot produce any query that has noticeable amplitude on any $(\text{ct}, Q, c, \sigma)$ such that

$$(\text{ct}, Q, c, \sigma) \neq (\text{ct}^*, Q^*, c^*, \sigma^*) \text{ and } \text{OSS.Ver}(\text{pk}_{\text{OSS}}^*, (\text{ct}, Q, c), \sigma) = \top.$$

But after $(\text{pk}_{\text{OSS}}^*, \text{ct}^*, Q^*, c^*, \sigma^*)$ is measured and s is sampled, PrivVer and $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$ can only differ on $(\text{ct}, Q, c, \sigma)$ such that

$$(\text{ct}, Q, c, \sigma) \neq (\text{ct}^*, Q^*, c^*, \sigma^*) \text{ and } \text{OSS.Ver}(\text{pk}_{\text{OSS}}^*, (\text{ct}, Q, c), \sigma) = \top.$$

Thus, since $\mathcal{A}_2$ only has polynomially-many queries, changing the oracle in this way can only have a negligible effect on the final probability, which completes the proof. $\square$

Next, we claim the following, where $V[1]$ is defined in Figure 4.

Claim 21.

$$\Pr \left[V[1]((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), s, (m^*, y^*, z^*)) = 1 : \begin{array}{l} ((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), \text{state}) \leftarrow \mathcal{A}_5 \\ s \leftarrow \{0, 1\}^\lambda \\ (m^*, y^*, z^*) \leftarrow \mathcal{A}_5(s, \text{state}) \end{array} \right] = \text{non-negl}(\lambda).$$

Proof. This is a straightforward adaptation of [BKNY23, Claim 5.14], reducing to the “Incompatible standard-basis measurements” property of Priv (Definition 17). $\square$

Finally, we will define a sequence of hybrids $\{\mathcal{H}_\iota\}_{\iota \in [0, p]}$ based on $\mathcal{A}_5$. Hybrid $\mathcal{H}_\iota$ is defined as follows.

- Run $((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), \text{state}) \leftarrow \mathcal{A}_5$.
- Sample $s \leftarrow \{0, 1\}^\lambda$.
- Run $(m^*, y^*, z^*) \leftarrow \mathcal{A}_5(s, \text{state})$ with the following difference. Recall that at some point, $\mathcal{A}_5$ begins using the oracle $\text{H}_{\text{Priv}}[(\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*) \rightarrow s]$ while answering $\mathcal{A}_2$ ’s queries. For the first ι times that $\mathcal{A}_2$ queries $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$ after this point, respond using the oracle $\text{PrivVer}[\text{pk}_{\text{OSS}}^* \rightarrow \perp]$ that outputs $\perp$ on every input that includes pk_{OSS}^* (and behaves normally otherwise).
- Output $V[1]((\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*), s, (m^*, y^*, z^*))$.

Note that Claim 21 is stating exactly that $\Pr[\mathcal{H}_0 = 1] = \text{non-negl}(\lambda)$. Next, we have the following claim.

Claim 22. $\Pr[\mathcal{H}_p = 1] = \text{negl}(\lambda)$.

Proof. This is a straightforward adaptation of [BKNY23, Claim 5.15], reducing to the soundness of Priv (Definition 17). $\square$

Finally, we prove the following Claim 23. Since $p = \text{poly}(\lambda)$, this contradicts Claim 21 and Claim 22, which completes the proof. $\square$

Claim 23. For any $\iota \in [p]$, $\Pr[\mathcal{H}_\iota = 1] \geq \Pr[\mathcal{H}_{\iota-1} = 1] - \text{negl}(\lambda)$.

Proof. Throughout this proof, when we refer to “query ι ” in some hybrid, we mean the ι ’th query that $\mathcal{A}_2$ makes to $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$ after $\mathcal{A}_5$ has begun using the oracle $\text{H}_{\text{Priv}}[(\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*) \rightarrow s]$ (if such a query exists).

Now, we introduce an intermediate hybrid $\mathcal{H}'_{\iota-1}$ which is the same as $\mathcal{H}_{\iota-1}$ except that query ι is answered with the functionality $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*), M_0]$ defined in Figure 4.

So, it suffices to show that

- $\Pr[\mathcal{H}'_{\iota-1} = 1] \geq \Pr[\mathcal{H}_{\iota-1} = 1] - \text{negl}(\lambda)$, and
- $\Pr[\mathcal{H}_\iota = 1] \geq \Pr[\mathcal{H}'_{\iota-1} = 1] - \text{negl}(\lambda)$.

We note that the only difference between the three hybrids is how query ι is answered:

- In $\mathcal{H}_{\iota-1}$, query ι is answered with $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$.
- In $\mathcal{H}'_{\iota-1}$, query ι is answered with $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*), M_0]$.
- In $\mathcal{H}_\iota$, query ι is answered with $\text{PrivVer}[\text{pk}_{\text{OSS}}^* \rightarrow \perp]$.

Now, the proof is completed by appealing to the following two claims. $\square$

Claim 24. $\Pr[\mathcal{H}'_{\iota-1} = 1] \geq \Pr[\mathcal{H}_{\iota-1} = 1] - \text{negl}(\lambda)$.

Proof. This is a straightforward adaptation of [BKNY23, Claim 5.17], reducing to the “Incompatible standard-basis measurements” property of Priv (Definition 17). $\square$

Claim 25. $\Pr[\mathcal{H}_i = 1] \geq \Pr[\mathcal{H}'_{i-1} = 1] - \text{negl}(\lambda)$

Proof. This is a straightforward adaptation of [BKNY23, Claim 5.18], reducing to the string binding with public decodability of PFC (Definition 8). $\square$

Functionalities used in the proof of Theorem 16

- $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s](\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$
 - Same as PrivVer except that it sets $\text{sp} = s$ if $\text{pk}_{\text{OSS}} = \text{pk}_{\text{OSS}}^*$.
- $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)](\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$
 - Same as $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s]$ except output $\perp$ if $\text{pk}_{\text{OSS}} = \text{pk}_{\text{OSS}}^* \wedge (\text{ct}, Q, c, \sigma) \neq (\text{ct}^*, Q^*, c^*, \sigma^*)$.
- $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*), 0](\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$
 - Same as $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$ except output $\perp$ if $\text{pk}_{\text{OSS}} = \text{pk}_{\text{OSS}}^* \wedge m_{\bar{S} \cup \bar{T}} \notin M_0$, where S, M_0 are defined based on $\text{sp} = s$ as in Definition 17.
- $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*), 1](\text{ct}, Q, \text{pk}_{\text{OSS}}, c, \sigma, u, y, z)$
 - Same as $\text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*)]$ except output $\perp$ if $\text{pk}_{\text{OSS}} = \text{pk}_{\text{OSS}}^* \wedge m_{\bar{S} \cup \bar{T}} \notin M_1$, where S, M_1 are defined based on $\text{sp} = s$ as in Definition 17.
- $V(a, s, \text{aux})$:
 - Parse $a := (\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)$ and $\text{aux} := (u^*, y^*, z^*)$.
 - Compute $\tilde{\text{ct}} := \text{PrivVer}[\text{pk}_{\text{OSS}}^*, s](\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*, u^*, y^*, z^*)$.
 - Output 1 iff $\tilde{\text{ct}} \neq \perp$ and $\text{Priv.Dec}(\text{sk}, \tilde{\text{ct}}) \neq Q^*(x)$.
- $V[1](a, s, \text{aux})$:
 - Parse $a := (\text{ct}^*, Q^*, \text{pk}_{\text{OSS}}^*, c^*, \sigma^*)$ and $\text{aux} := (u^*, y^*, z^*)$.
 - Compute $\tilde{\text{ct}} := \text{PrivVer}[\text{pk}_{\text{OSS}}^*, s, (\text{ct}^*, Q^*, c^*, \sigma^*), 1](\text{ct}^*, Q^*, (\text{pk}_{\text{OSS}}^*, c^*, \sigma^*, u^*, y^*, z^*))$.
 - Output 1 iff $\tilde{\text{ct}} \neq \perp$ and $\text{Priv.Dec}(\text{sk}, \tilde{\text{ct}}) \neq Q^*(x)$.

Figure 4: Helper functionalities for the proof of Theorem 16.

7 (Succinct) Classical Obfuscation of Pseudo-deterministic Quantum Circuits

Definition 18 (Classical Ideal obfuscation for Pseudodeterministic Quantum Circuits). *A classical ideal obfuscator for a family of pseudo-deterministic quantum circuits is a pair of QPT algorithms (Obf, Eval) with the following syntax.*

- $\text{Obf}(1^\lambda, P_Q) \rightarrow P_{\tilde{Q}}$: Obf is a classical p.p.t. algorithm that takes as input the security parameter 1^λ and the classical description P_Q of a quantum circuit Q , and outputs the classical description of an obfuscated circuit $P_{\tilde{Q}}$.
- $\text{Eval}(P_{\tilde{Q}}, x) \rightarrow b$: Eval is a (quantum) polynomial-time algorithm that takes as input the description $P_{\tilde{Q}}$ of an obfuscated circuit $\tilde{Q}$ and an input x , and outputs a bit $b \in \{0, 1\}$.

A VBB obfuscator should satisfy the following properties for any pseudo-deterministic family of circuits $Q = \{Q_\lambda\}_{\lambda \in \mathbb{N}}$ with input length $n = n(\lambda)$.

- **Correctness**: It holds with probability $1 - \text{negl}(\lambda)$ over $\tilde{Q} \leftarrow \text{Obf}(1^\lambda, P_Q)$ that for all $x \in \{0, 1\}^n$, $\Pr[\text{Eval}(P_{\tilde{Q}}, x) \rightarrow Q(x)] = 1 - \text{negl}(\lambda)$.
- **Security**: There exists a QPT simulator $\{\mathcal{S}_\lambda\}_{\lambda \in \mathbb{N}}$, such that for any QPT adversary $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$, pseudodeterministic programs P_Q ,

$$\{\mathcal{A}_\lambda(\text{Obf}(1^\lambda, P_Q))\} \approx_c \mathcal{S}_\lambda^{O[Q]}$$

where $O[Q]$ is the oracle that computes the map $x \rightarrow Q(x)$.

In addition, the ideal obfuscator could optionally satisfy a succinctness criterion.

- **Succinctness**: The size of the obfuscation $P_{\tilde{Q}} \leftarrow \text{Obf}(1^\lambda, P_Q)$ is polynomially bounded by the size of the description P_Q of the input circuit, rather than the size of the circuit (in terms of number of gates) itself. That is, $|P_{\tilde{Q}}| \leq \text{poly}(\lambda, |P_Q|)$, rather than $|P_{\tilde{Q}}| \leq \text{poly}(\lambda, |Q|)$.

7.1 Construction

- A publicly-verifiable QFHE for pseudo-deterministic circuits in the oracle model $\text{pvQFHE} = (\text{Gen}, \text{Enc}, \text{Eval}, \text{Ver}, \text{Dec})$ (Definition 16).
- A post-quantum input-succinct obfuscator ($\text{SObf}, \text{SEval}$), which we constructed in Section 5.

Given the above building blocks, we give the following construction of classical obfuscation for pseudo-deterministic quantum circuits.

Classical obfuscation of pseudo-deterministic quantum circuits

Let U be the universal quantum circuit that takes as input the description P_Q of a circuit of input of size N , where N is the length of an input to Q . Let D be the depth of U , and let S be a bound on the size of U . Let U_x be the quantum circuit that takes as input the description of a circuit P_Q and runs the universal quantum circuit U on $P_Q\|x$.

- $\text{Obf}(1^\lambda, P_Q)$:
 - Sample $(\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{pvQFHE.Gen}(1^\lambda, D, |P_Q|, S)$, and $\text{ct} \leftarrow \text{pvQFHE.Enc}(\text{pk}, P_Q)$.
 - Let $\text{DK}(x, \text{ct}', \pi)$ be the following functionality. First, run $\text{pvQFHE.Ver}^{\text{PP}}(\text{ct}, U_x, \text{ct}', \pi)$. If the output was $\perp$, then output $\perp$, and otherwise output $\text{pvQFHE.Dec}(\text{sk}, \text{ct}')$.
 - Sample $\widetilde{\text{DK}} \leftarrow \text{SObf}(1^\lambda, \text{DK})$.
 - Output $\widetilde{Q} := (\text{ct}, \text{PP}, \widetilde{\text{DK}})$.
- $\text{Eval}(P_{\widetilde{Q}}, x)$:
 - Parse $P_{\widetilde{Q}}$ as $(\text{ct}, \text{PP}, \widetilde{\text{DK}})$.
 - Compute $(\text{ct}', \pi) \leftarrow \text{Eval}^{\text{PP}}(\text{ct}, U_x)$.
 - Output $b := \widetilde{\text{DK}}(x, \text{ct}', \pi)$.

Figure 5: Succinct classical obfuscation of pseudo-deterministic quantum circuits

Theorem 17. *The construction in Figure 5 satisfies correctness, security and succinctness definitions in Definition 18.*

The proofs of correctness and security are almost identical to the proof of Theorem 6.1 of [BKNY23].

Proof. First, correctness follows immediately from the correctness of the VBB obfuscator and the correctness of the publicly-verifiable QFHE scheme (Definition 16). Note that even though the evaluation procedure may include measurements, an evaluator could run coherently, measure just the output bit b , and reverse. By the Gentle Measurement lemma, this implies the ability to run the obfuscated program on any $\text{poly}(\lambda)$ number of inputs.

Next, we show security. For any QPT adversary $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$, we define a simulator $\{\mathcal{S}_\lambda\}_{\lambda \in \mathbb{N}}$ as follows, where $\{\widetilde{\mathcal{A}}_\lambda\}_{\lambda \in \mathbb{N}}$ is the simulator for the classical obfuscation scheme (CObf, CEval), defined based on $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$.

- Sample $(\text{pk}, \text{sk}, \text{PP}) \leftarrow \text{pvQFHE.Gen}(1^\lambda, D, |Q|, S)$, $\text{ct} \leftarrow \text{pvQFHE.Enc}(\text{pk}, 0^{|Q|})$.
- Run $\widetilde{\mathcal{A}}_\lambda^{\text{PP}, \text{DK}}(\text{ct})$, answering PP calls honestly, and DK calls as follows.
 - Take (x, ct', π) as input.
 - Run $\text{Ver}^{\text{PP}}(x, \text{ct}', \pi)$. If the output was $\perp$ then output $\perp$.
 - Otherwise, forward x to the external oracle $O[Q]$, and return the result $b = O[Q](x)$.
- Output $\widetilde{\mathcal{A}}_\lambda$'s output.

Now, for any circuit Q , we define a sequence of hybrids.

- Hybrid 0: Sample $\widetilde{Q} \leftarrow \text{Obf}(1^\lambda, Q)$ and run $\mathcal{A}_\lambda(1^\lambda, \widetilde{Q})$.
- Hybrid 1: Sample $(\text{ct}, \text{PP}, \text{DK})$ as in $\text{Obf}(1^\lambda, Q)$, and run $\widetilde{\mathcal{A}}_\lambda^{\text{PP}, \text{DK}}(\text{ct})$.

- Hybrid 2: Same as hybrid 1, except that calls to DK are answered as in the description of $\mathcal{S}_\lambda$.
- Hybrid 3: Same as hybrid 2, except that we sample $\text{ct} \leftarrow \text{Enc}(\text{pk}, 0^{|Q|})$. This is $\mathcal{S}_\lambda$.

We complete the proof by showing the following.

- $|\Pr[1 \leftarrow \text{Hybrid 0}] - \Pr[1 \leftarrow \text{Hybrid 1}]| = \text{negl}(\lambda)$. This follows from the security of the classical obfuscation scheme (CObf, CEval).
- $|\Pr[1 \leftarrow \text{Hybrid 1}] - \Pr[1 \leftarrow \text{Hybrid 2}]| = \text{negl}(\lambda)$. Suppose otherwise. Then there must exist some query made by $\tilde{\mathcal{A}}_\lambda$ to DK with noticeable amplitude on $(x, \tilde{\text{ct}}, \pi)$ such that DK does not return $\perp$ but $\text{Dec}(\text{sk}, \tilde{\text{ct}}) \neq Q(x)$. Thus, we can measure a random one of the $\text{poly}(\lambda)$ many queries made by $\tilde{\mathcal{A}}_\lambda$ to obtain such an $(x, \tilde{\text{ct}}, \pi)$, which violates the soundness of the publicly-verifiable QFHE scheme (Definition 16).
- $|\Pr[1 \leftarrow \text{Hybrid 2}] - \Pr[1 \leftarrow \text{Hybrid 3}]| = \text{negl}(\lambda)$. Since sk is no longer used in hybrid 2 to respond to DK queries, this follows directly from the security of the publicly-verifiable QFHE scheme (Definition 16).

Finally, the scheme is succinct. This follows from the compactness of the pvQFHE scheme and the input-succinct obfuscation (SObf, SEval) scheme. $\square$

8 Acknowledgements

Aparna Gupte would like to thank Yael Kalai and Vinod Vaikuntanathan for valuable discussions. Aparna Gupte was supported in part by DARPA under Agreement No. HR00112020023, NSF CNS-2154149 and a Simons Investigator Award. This work was done in part while Aparna Gupte was at the Simons Institute and participating in the Challenge Institute for Quantum Computation at UC Berkeley, and while Omri Shmueli was a research fellow at the Simons Institute for the Theory of Computing at UC Berkeley.

References

- [AC12] Scott Aaronson and Paul Christiano. Quantum money from hidden subspaces. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pages 41–60, 2012.
- [AGKZ20] Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Mark Zhandry. One-shot signatures and applications to hybrid quantum/classical authentication. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 255–268, 2020.
- [AGQY22] Prabhanjan Ananth, Aditya Gulati, Luowen Qian, and Henry Yuen. Pseudorandom (function-like) quantum state generators: New definitions and applications. In *Theory of Cryptography Conference*, pages 237–265. Springer, 2022.
- [APM20] Shweta Agrawal and Alice Pellet-Mary. Indistinguishability obfuscation without maps: Attacks and fixes for noisy linear fe. In *Advances in Cryptology – EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I*, page 110–140, Berlin, Heidelberg, 2020. Springer-Verlag.
- [Bar21] James Bartusek. Secure quantum computation with classical communication. In *Theory of Cryptography Conference*, pages 1–30. Springer, 2021.
- [BBV24] James Bartusek, Zvika Brakerski, and Vinod Vaikuntanathan. Quantum state obfuscation from classical oracles. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, pages 1009–1017, 2024.

- [BCP14] Elette Boyle, Kai-Min Chung, and Rafael Pass. On extractability (a.k.a. differing-inputs) obfuscation. page TCC, 2014.
- [BDGM23] Zvika Brakerski, Nico Döttling, Sanjam Garg, and Giulio Malavolta. Candidate io from homomorphic encryption schemes. *Journal of Cryptology*, 36(3):27, 2023.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im) possibility of obfuscating programs. In *Annual international cryptology conference*, pages 1–18. Springer, 2001.
- [BGL⁺15] Nir Bitansky, Sanjam Garg, Huijia Lin, Rafael Pass, and Sidharth Telang. Succinct randomized encodings and their applications. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, page 439–448, New York, NY, USA, 2015. Association for Computing Machinery.
- [BGMZ18] James Bartusek, Jiaxin Guan, Fermi Ma, and Mark Zhandry. Return of ggh15: provable security against zeroizing attacks. In *Theory of Cryptography Conference*, pages 544–574. Springer, 2018.
- [BK21] Anne Broadbent and Raza Ali Kazmi. Constructions for quantum indistinguishability obfuscation. In *Progress in Cryptology – LATINCRYPT 2021: 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6–8, 2021, Proceedings*, page 24–43, Berlin, Heidelberg, 2021. Springer-Verlag.
- [BKL⁺22] James Bartusek, Yael Tauman Kalai, Alex Lombardi, Fermi Ma, Giulio Malavolta, Vinod Vaikuntanathan, Thomas Vidick, and Lisa Yang. Succinct classical verification of quantum computation. In *Annual International Cryptology Conference*, pages 195–211. Springer, 2022.
- [BKNY23] James Bartusek, Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. Obfuscation of pseudo-deterministic quantum circuits. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1567–1578, 2023.
- [BM22] James Bartusek and Giulio Malavolta. Indistinguishability Obfuscation of Null Quantum Circuits and Applications. In Mark Braverman, editor, *13th Innovations in Theoretical Computer Science Conference (ITCS 2022)*, volume 215 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:13, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [Bra18] Zvika Brakerski. Quantum fhe (almost) as secure as classical. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 67–95, Cham, 2018. Springer International Publishing.
- [CLLW22] Kai-Min Chung, Yi Lee, Han-Hsuan Lin, and Xiaodi Wu. Constant-round blind classical verification of quantum sampling. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 707–736. Springer, 2022.
- [CLLZ21] Andrea Coladangelo, Jiahui Liu, Qipeng Liu, and Mark Zhandry. Hidden cosets and applications to unclonable cryptography. In *Annual International Cryptology Conference*, pages 556–584. Springer, 2021.
- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *Theory of Cryptography*, pages 1–29, Cham, 2019. Springer International Publishing.
- [CVW18] Yilei Chen, Vinod Vaikuntanathan, and Hoeteck Wee. Ggh15 beyond permutation branching programs: proofs, attacks, and candidates. In *Annual International Cryptology Conference*, pages 577–607. Springer, 2018.

- [DFM20] Jelle Don, Serge Fehr, and Christian Majenz. The measure-and-reprogram technique 2.0: Multi-round fiat-shamir and more. In *Advances in Cryptology – CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III*, page 602–631, Berlin, Heidelberg, 2020. Springer-Verlag.
- [DFMS19] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the fiat-shamir transformation in the quantum random-oracle model. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II*, page 356–383, Berlin, Heidelberg, 2019. Springer-Verlag.
- [DH22] Whitfield Diffie and Martin E Hellman. New directions in cryptography. In *Democratizing cryptography: the work of Whitfield Diffie and Martin Hellman*, pages 365–390. 2022.
- [GGH15] Craig Gentry, Sergey Gorbunov, and Shai Halevi. Graph-induced multilinear maps from lattices. In *Theory of Cryptography Conference*, pages 498–527. Springer, 2015.
- [GGH⁺16] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM Journal on Computing*, 45(3):882–929, 2016.
- [GKNV25] Sam Gunn, Yael Kalai, Anand Natarajan, and Ági Villányi. Classical commitments to quantum states. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC ’25, page 234–244, New York, NY, USA, 2025. Association for Computing Machinery.
- [GR07] Shafi Goldwasser and Guy N Rothblum. On best-possible obfuscation. In *Theory of Cryptography Conference*, pages 194–213. Springer, 2007.
- [HT25] Mi-Ying Huang and Er-Cheng Tang. Obfuscation of unitary quantum programs. *arXiv preprint arXiv:2507.11970*, 2025.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *Proceedings of the 53rd annual ACM SIGACT symposium on theory of computing*, pages 60–73, 2021.
- [Mah18] Urmila Mahadev. Classical verification of quantum computations. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 259–267, 2018.
- [Mah20] Urmila Mahadev. Classical homomorphic encryption for quantum circuits. *SIAM Journal on Computing*, 52(6):FOCS18–189, 2020.
- [Mic94] Silvio Micali. Computationally sound proofs. 52(6):FOCS, 1994.
- [MNZ24] Tony Metger, Anand Natarajan, and Tina Zhang. Succinct Arguments for QMA from Standard Assumptions via Compiled Nonlocal Games . In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1193–1201, Los Alamitos, CA, USA, October 2024. IEEE Computer Society.
- [NZ23] Anand Natarajan and Tina Zhang. Bounding the Quantum Value of Compiled Nonlocal Games: From CHSH to BQP Verification . In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1342–1348, Los Alamitos, CA, USA, November 2023. IEEE Computer Society.
- [Shm22a] Omri Shmueli. Public-key quantum money with a classical bank. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 790–803, 2022.
- [Shm22b] Omri Shmueli. Semi-quantum tokenized signatures. In *Annual International Cryptology Conference*, pages 296–319. Springer, 2022.

- [Sho99] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.
- [SZ25] Omri Shmueli and Mark Zhandry. On one-shot signatures, quantum vs classical binding, and obfuscating permutations. *Cryptology ePrint Archive*, 2025.
- [WW21] Hoeteck Wee and Daniel Wichs. Candidate obfuscation via oblivious lwe sampling. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 127–156. Springer, 2021.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, pages 679–687, 2012.
- [Zha13] Mark Zhandry. A note on the quantum collision and set equality problems. *arXiv preprint arXiv:1312.1027*, 2013.
- [Zha19a] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In *Annual International Cryptology Conference*, pages 239–268. Springer, 2019.
- [Zha19b] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In *Annual International Cryptology Conference*, 2019.
- [Zha21] Mark Zhandry. How to construct quantum random functions. *Journal of the ACM (JACM)*, 68(5):1–43, 2021.
- [Zha22] Mark Zhandry. New constructions of collapsing hashes. In *Annual international cryptology conference*. Springer, 2022.

A Remaining proofs from Section 4

A.1 Correctness

Proof of Theorem 4. This proof is almost identical to the proof of Theorem 4.6 of [BK~~NY~~23], but we repeat it here for completeness. The only difference is that we are using one-shot signatures instead of signature tokens; this is what allows us to make the commitment keys entirely classical.

We will assume that the one-shot signature schemes $\text{OSS}, \overline{\text{OSS}}$ are perfectly correct. If they are statistically correct, then the Pauli functional commitment scheme we obtain is still correct, because we allow for a negligible statistical distance.

We will first show that the map applied by $\text{Com}^{\text{CK}}(b)$ in the case that the measurement of the first qubit of S' is $1 - b$ successfully takes $|S_{y,1-b}\rangle \rightarrow |S_{y,b}\rangle$. Since we are assuming perfect correctness from $\overline{\text{OSS}}$, it suffices to show that for any balanced affine subspace $|S_y\rangle$,

$$H^{\otimes n} \text{Phase}^{O[S_y^\perp]} H^{\otimes n} |S_{y,1-b}\rangle \rightarrow |S_{y,b}\rangle,$$

where $\text{Phase}^{O[S_y^\perp]}$ is the map $|s\rangle \rightarrow (-1)^{O[S_y^\perp](s)} |s\rangle$, and $O[S_y^\perp]$ is the oracle that outputs 0 if $s \in S^\perp$ and 1 if $s \notin S^\perp$. This was also shown in [AGKZ20].

Let S_y^* be the subspace parallel to affine subspace S_y , and let $S_{y,b}^*$ be the coset of S_y^* whose first coordinate is b . We will use the facts that $S_{y,1}^* = S_{y,0}^* + w$ for some w , and that $S_{y,0} = S_{y,0}^* + v_0$ and $S_{y,1} = S_{y,1}^* + v_1$ for some v_0, v_1 such that $v_0 + v_1 = w$. Also note that for any $s \in S_y^\perp$, $s \cdot w = 0$, and for any $s \in (S_{y,0}^*)^\perp \setminus (S_y^*)^\perp$, $s \cdot w = 1$.

$$\begin{aligned}
& H^{\otimes n} \text{Ph}^{O[S_y^\perp]} H^{\otimes n} |S_{y,1-b}\rangle \\
&= H^{\otimes n} \text{Ph}^{O[S_y^\perp]} H^{\otimes n} \frac{1}{\sqrt{2^{n/2-1}}} \left(\sum_{s \in S_{y,0}^*} |s + v_{1-b}\rangle \right) \\
&= H^{\otimes n} \text{Ph}^{O[S_y^\perp]} \frac{1}{\sqrt{2^{n/2+1}}} \left(\sum_{s \in (S_{y,0}^*)^\perp} (-1)^{s \cdot v_{1-b}} |s\rangle \right) \\
&= H^{\otimes n} \text{Ph}^{O[S_y^\perp]} \frac{1}{\sqrt{2^{n/2+1}}} \left(\sum_{s \in (S_y^*)^\perp} (-1)^{s \cdot w + s \cdot v_b} |s\rangle + \sum_{s \in (S_{y,0}^*)^\perp \setminus (S_y^*)^\perp} (-1)^{s \cdot w + s \cdot v_b} |s\rangle \right) \\
&= H^{\otimes n} \text{Ph}^{O[S_y^\perp]} \frac{1}{\sqrt{2^{n/2+1}}} \left(\sum_{s \in (S_y^*)^\perp} (-1)^{s \cdot v_b} |s\rangle + \sum_{s \in (S_{y,0}^*)^\perp \setminus (S_y^*)^\perp} (-1)^{1 + s \cdot v_b} |s\rangle \right) \\
&= H^{\otimes n} \frac{1}{\sqrt{2^{n/2+1}}} \left(\sum_{s \in S^\perp} (-1)^{s \cdot v_b} |s\rangle + \sum_{s \in (S_{y,0}^*)^\perp \setminus (S_y^*)^\perp} (-1)^{s \cdot v_b} |s\rangle \right) \\
&= H^{\otimes n} \frac{1}{\sqrt{2^{n/2+1}}} \left(\sum_{s \in (S_{y,0}^*)^\perp} (-1)^{s \cdot v_b} |s\rangle \right) \\
&= |S_{y,b}\rangle.
\end{aligned}$$

Now consider applying Com^{CK} to a pure state. By the correctness of OSS, Com can produce a valid signature σ_0 of 0, and is able to access the $\bar{O} = (P, P^{-1}, D)$ oracles of OSS through $\text{CK}_P(\text{vk}, \cdot)$, $\text{CK}_{P^{-1}}(\text{vk}, \cdot)$ and $\text{CK}_D(\text{vk}, \sigma_0)$. These oracles $\bar{O} = (P, P^{-1}, D)$ are an instance of $\bar{\text{OSS}}$ generated using randomness determined by $\text{RO}(\text{vk})$. By the correctness of $\bar{\text{OSS}}.\text{Gen}^{P, P^{-1}, D}$, Com is able to generate a $(\overline{vk}, \overline{sk})$ pair $(y, |S_y\rangle)$.

Applying Com^{CK} to pure state $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$ produces (up to negligible trace distance) the state

$$|\psi_{\text{Com}}\rangle = \alpha_0 |0\rangle |S_{y,0}\rangle + \alpha_1 |1\rangle |S_{y,1}\rangle,$$

and a signature c on the bit 1.

We claim that measuring and decoding $|\psi_{\text{Com}}\rangle$ in the standard (resp. Hadamard) basis produces the same distribution as directly measuring $|\psi\rangle$ in the standard (resp. Hadamard) basis. As a mixed state is a probability distribution over pure states, this will complete the proof of correctness.

First, it is immediate that measuring $|\psi_{\text{Com}}\rangle$ in the standard basis produces a bit b with probability $|\alpha_b|^2$ along with a vector s such that $s \in (S + v)_b$.

Next, note that applying Hadamard to each qubit of $|\psi_{\text{Com}}\rangle$ except the first results in the state

$$\alpha_0 |0\rangle \left(\sum_{s \in (S_{y,0}^*)^\perp} (-1)^{s \cdot v_0} |s\rangle \right) + \alpha_1 |1\rangle \left(\sum_{s \in (S_{y,0}^*)^\perp} (-1)^{s \cdot v_1} |s\rangle \right),$$

and thus, measuring each of these qubits (except the first) in the Hadamard basis produces a vector s and a single-qubit state

$$(-1)^{s \cdot v_0} \alpha_0 |0\rangle + (-1)^{s \cdot v_1} \alpha_1 |1\rangle = \alpha_0 |0\rangle + (-1)^{s \cdot w} \alpha_1 |1\rangle.$$

So, measuring this qubit in the Hadamard basis is equivalent to measuring $|\psi\rangle$ in the Hadamard basis and masking the result with $s \cdot w$. Recalling that $s \cdot w = 0$ if $s \in (S_y^*)^\perp$ and $s \cdot w = 1$ if $s \in (S_{y,0}^*)^\perp \setminus (S_y^*)^\perp$ completes the proof of correctness, since $S_y^\perp = (S_y^*)^\perp$ and $S_{y,0}^\perp = (S_{y,0}^*)^\perp$. $\square$

A.2 Binding

Lemma 8. *The collapse-binding definition of Definition 9 implies the single-bit binding definition of Definition 7.*

Proof. We will first define the following notion of binding, and then establish that the collapse-binding definition of Definition 9 implies this collapse binding definition.

Definition 19 (Single bit collapse binding). *For any adversary $\mathcal{A} := \{(\mathcal{A}_1, \mathcal{A}_2)_\lambda\}_{\lambda \in \mathbb{N}}$, where each of $\mathcal{A}_1$ and $\mathcal{A}_2$ are oracle-aided quantum operations that make at most $\text{poly}(\lambda)$ oracle queries, define the single-bit collapse binding experiment $\text{SingleBitCollapseBindingExpt}^{\mathcal{A}}(1^\lambda)$ as follows.*

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$, do nothing, and if $b = 1$ measure $(\mathcal{B}, \mathcal{U})$ with $\{\Pi_{\text{dk}, c, 0}, \Pi_{\text{dk}, c, 1}\}$
- Run $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' .
- The experiment outputs 1 if $b = b'$.

We say that adversary $\mathcal{A}$ is valid if the state on $(\mathcal{B}, \mathcal{U})$ output by $\mathcal{A}_1$ is in the image of $|0\rangle\langle 0| \otimes \Pi_{\text{dk}, c, 0} + |1\rangle\langle 1| \otimes \Pi_{\text{dk}, c, 1}$. Then, we say that a Pauli functional commitment $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies publicly-decodable single bit collapse-binding if it holds that for all valid adversaries $\mathcal{A}$,

$$\left| \Pr \left[\text{SingleBitCollapseBindingExpt}^{\mathcal{A}}(1^\lambda) = 1 \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

Lemma 9. *The collapse binding definition of Definition 9 implies the single bit collapse binding definition of Definition 19.*

Proof. We will establish the proof of this claim through a sequence of hybrids.

Hybrid 0: This hybrid describes the following experiment

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Measure $(\mathcal{B}, \mathcal{U})$ with $\{\Pi_{\text{dk}, c, 0}, \Pi_{\text{dk}, c, 1}\}$.
- Run $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' .

Hybrid 1 This hybrid is exactly the same as the previous hybrid, except that the computational basis measurement is performed instead of the single bit measurement $\{\Pi_{\text{dk}, c, 0}, \Pi_{\text{dk}, c, 1}\}$

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- Run $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' .

Claim 26. *If $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies the collapse-binding definition of Definition 9, then*

$$\left| \Pr[1 \leftarrow \text{Hybrid } 0] - \Pr[1 \leftarrow \text{Hybrid } 1] \right| \leq \text{negl}(\lambda)$$

Proof. Assume for contradiction that there exists an adversary $(\mathcal{A}_1, \mathcal{A}_2)$ such that $\left| \Pr[1 \leftarrow \text{Hybrid } 1] - \Pr[1 \leftarrow \text{Hybrid } 0] \right| \geq \varepsilon(\lambda)$, where $\varepsilon(\cdot)$ is some non negligible function. Then, we can construct an adversary $(\mathcal{B}_1, \mathcal{B}_2)$ that breaks the collapse binding property of $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$.

Description of $(\mathcal{B}_1, \mathcal{B}_2)$:

- $\mathcal{B}_1$ runs $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- $\mathcal{B}_1$ measures $(\mathcal{B}, \mathcal{U})$ with $\{\Pi_{dk, c, 0}, \Pi_{dk, c, 1}\}$.
- Sample $b \leftarrow \{0, 1\}$. If $b = 0$ do nothing, and if $b = 1$, measure $(\mathcal{B}, \mathcal{U})$ in the computational basis.
- $\mathcal{B}_2$ runs $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs b' .
- $\mathcal{B}_2$ outputs b' .
- The experiment outputs 1 if $b' = b$.

Observe that when $b = 0$, this experiment is identical to **Hybrid 0**, and when $b = 1$, the experiment is identical to **Hybrid 1**. Thus, $(\mathcal{B}_1, \mathcal{B}_2)$ breaks collapse binding (Definition 9) with non negligible advantage. $\square$

Hybrid 2: This is exactly the same as the previous hybrid, except no measurement is performed.

- Sample $(\text{CK}, \text{dk}) \leftarrow \text{Gen}(1^\lambda)$.
- Run $\mathcal{A}_1^{\text{CK}, \text{DecZ}[\text{dk}], \text{DecX}[\text{dk}]}(1^\lambda)$ until it outputs a commitment c and a state on registers $(\mathcal{B}, \mathcal{U}, \mathcal{R})$. Here $\mathcal{B}$ is a single-qubit register, $\mathcal{U}$ is the opening register and $\mathcal{R}$ holds the internal state of $\mathcal{A}$.
- Run $\mathcal{A}_2^{\text{CK}, \text{DecZ}[\text{dk}]}(\mathcal{B}, \mathcal{U}, \mathcal{R})$ until it outputs a bit b' .

Claim 27. *If $(\text{Gen}, \text{Com}, \text{OpenZ}, \text{OpenX}, \text{DecZ}, \text{DecX})$ satisfies the collapse-binding definition of Definition 9, then*

$$\left| \Pr[1 \leftarrow \text{Hybrid } 2] - \Pr[1 \leftarrow \text{Hybrid } 1] \right| \leq \text{negl}(\lambda)$$

Proof. This follows directly from the collapse binding definition of Definition 9. $\square$

$\square$

Now, Lemma A.1 of [BKRY23] shows that single-bit collapse binding implies a notion of binding called “unique-message binding”, and also shows that this notion implies string-binding. String-binding implies single-bit binding, and therefore we are done. $\square$